

Java 27, Java'ya Yeni Başlayan Programcılar İçin

- [Java 27, Java'ya Yeni Başlayan Programcılar İçin](#)
 - [Burada "Java 27" ne demek](#)
 - [Başlarken](#)
 - [Müfredat](#)
 - [Bunu nasıl kullanmalı](#)
 - [Örnekleri çalıştırma](#)
 - [Her şeyi doğrulama](#)
 - [PDF](#)
 - [Proje dokümanları](#)
 - [Bunun nasıl inşa edildiğine dair bir not](#)
- [Çeviri Notları](#)
 - [Neyin çevrildiği, neyin çevrilmediği](#)
 - [Terim tercihleri](#)
 - [Yapı](#)
 - [PDF](#)
- [Spesifikasyon: Java 27 Öğrenim Müfredatı](#)
 - [Yaptığım Varsayımlar](#)
 - [Amaç](#)
 - [Teknoloji Yığını](#)
 - [Proje Yapısı](#)
 - [Kod Stili](#)
 - [Test Stratejisi](#)
 - [Sınırlar](#)
 - [Başarı Kriterleri](#)
 - [Açık Sorular, Çözüldükleri Hâliyle](#)
 - [Uygulama Sırasında Yapılan Düzeltmeler](#)
 - [Ek 1 — PDF çıktısı ve Türkçe çeviri](#)
- [Modül 01: Başlarken](#)
 - [Neler Öğreneceksiniz](#)
 - [Başka Bir Dilden Geliyorsanız](#)
 - [Ders](#)
 - [Çalıştırın](#)
 - [Sık Yapılan Hatalar](#)
 - [Özet Çıkarımlar](#)
 - [Ödev](#)
- [Ödev 01: Hakkımda](#)

- Görev
- Sonra Bozun
- Kabul Kriterleri
- İleri Seviye
- İpucu, yaşta takıldıysanız
- Modül 02: Değişkenler ve Tipler
 - Neler Öğreneceksiniz
 - Başka Bir Dilden Geliyorsanız
 - Ders
 - Çalıştırın
 - Sık Yapılan Hatalar
 - Özet Çıkarımlar
 - Ödev
- Ödev 02: Hesabı Tutan Fiş
 - Görev
 - Para Kuralı
 - Yuvarlama Kararı
 - Sonra İki Tuzağı Yeniden Üretin
 - Kabul Kriterleri
 - İleri Seviye
 - İpucu, biçimlendirme sizi zorluyorsa
- Modül 03: Akış Kontrolü
 - Neler Öğreneceksiniz
 - Başka Bir Dilden Geliyorsanız
 - Ders
 - Çalıştırın
 - Sık Yapılan Hatalar
 - Özet Çıkarımlar
 - Ödev
- Ödev 03: Notlar ve İzgaralar
 - Birinci Bölüm: Notlar
 - İkinci Bölüm: İzgarada Arama
 - Kabul Kriterleri
 - İleri Seviye
 - İpucu, switch derlenmiyorsa
- Modül 04: Metotlar
 - Neler Öğreneceksiniz
 - Başka Bir Dilden Geliyorsanız
 - Ders

- Çalıştırın
- Sık Yapılan Hatalar
- Özet Çıkarımlar
- Ödev
- Ödev 04: Bir Metot Neyi Değiştirebilir, Neyi Değiştiremez
 - Birinci Bölüm: Envanter
 - İkinci Bölüm: Remove Tuzağı
 - Kabul Kriterleri
 - İleri Seviye
 - İpucu, birinci bölüm zaten çalışıyor görünüyorsa
- Modül 05: Sınıflar ve Nesneler
 - Neler Öğreneceksiniz
 - Başka Bir Dilden Geliyorsanız
 - Ders
 - Çalıştırın
 - Sık Yapılan Hatalar
 - Özet Çıkarımlar
 - Ödev
- Ödev 05: Bozulamayan Bir Nesne
 - Görev
 - Kanıtlayın
 - Ayrıca İçermeli
 - Kabul Kriterleri
 - İleri Seviye
 - İpucu, dördüncü kural elinizden kaçıyor
- Modül 06: Kalıtım ve Çok Biçimlilik
 - Neler Öğreneceksiniz
 - Başka Bir Dilden Geliyorsanız
 - Ders
 - Çalıştırın
 - Sık Yapılan Hatalar
 - Özet Çıkarımlar
 - Ödev
- Ödev 06: Bir Şeyleri Kaybeden Küme
 - Birinci Bölüm: Bir HashSet Kırın
 - İkinci Bölüm: Karışık Raf
 - Kabul Kriterleri
 - İleri Seviye
 - İpucu, birinci bölüm yanlış davranmıyorsa

- Modül 07: Arayüzler
 - Neler Öğreneceksiniz
 - Başka Bir Dilden Geliyorsanız
 - Ders
 - Çalıştırın
 - Sık Yapılan Hatalar
 - Özet Çıkarımlar
 - Ödev
- Ödev 07: Yetenekler, Çakışmalar ve Kırılgan Bir Temel
 - Birinci Bölüm: Akıllı Cihazlar
 - İkinci Bölüm: Çözmek Zorunda Olduğunuz Bir Çakışma
 - Üçüncü Bölüm: Bir Alt Sınıfı Kırın
 - Kabul Kriterleri
 - İleri Seviye
 - İpucu, üçüncü bölüm hemen 3 veriyorsa
- Modül 08: Record'lar ve Enum'lar
 - Neler Öğreneceksiniz
 - Başka Bir Dilden Geliyorsanız
 - Ders
 - Çalıştırın
 - Sık Yapılan Hatalar
 - Özet Çıkarımlar
 - Ödev
- Ödev 08: Bir Sipariş Sistemi
 - Birinci Bölüm: Money
 - İkinci Bölüm: İki Sızıntı
 - Üçüncü Bölüm: Kargo ve Durum
 - Kabul Kriterleri
 - İleri Seviye
 - İpucu, ikinci bölümün ikinci saldırısı işe yaramıyorsa
- Modül 09: Mühürlü Tipler ve Desen Eşleme
 - Neler Öğreneceksiniz
 - Başka Bir Dilden Geliyorsanız
 - Ders
 - Çalıştırın
 - Sık Yapılan Hatalar
 - Özet Çıkarımlar
 - Ödev
- Ödev 09: Bir İfade Değerlendirici

- Birinci Bölüm: Model
- İkinci Bölüm: Üç İşlem
- Üçüncü Bölüm: Derleyicinin İzlediğini Kanıtlayın
- Dördüncü Bölüm: Sadeleştirme
- Kabul Kriterleri
- İleri Seviye
- İpucu, iç içe desen derlenmiyorsa
- Modül 10: İstisnalar
 - Neler Öğreneceksiniz
 - Başka Bir Dilden Geliyorsanız
 - Ders
 - Çalıştırın
 - Sık Yapılan Hatalar
 - Özet Çıkarımlar
 - Ödev
- Ödev 10: Doğruyu Söyleyen Bir Config Yükleyici
 - Birinci Bölüm: Yükleyici
 - İkinci Bölüm: Üç Yanlış Kullanım
 - Üçüncü Bölüm: Kaynaklar
 - Kabul Kriterleri
 - İleri Seviye
 - İpucu, bastırılan liste boş geliyorsa
- Modül 11: Jenerikler
 - Neler Öğreneceksiniz
 - Başka Bir Dilden Geliyorsanız
 - Ders
 - Çalıştırın
 - Sık Yapılan Hatalar
 - Özet Çıkarımlar
 - Ödev
- Ödev 11: Tipli Bir Önbellek
 - Birinci Bölüm: Önbellek
 - İkinci Bölüm: Sınırlar ve PECS
 - Üçüncü Bölüm: Yığını Kırletin
 - Kabul Kriterleri
 - İleri Seviye
 - İpucu, üçüncü bölüm fırlatmıyorsa
- Modül 12: Koleksiyonlar
 - Neler Öğreneceksiniz

- Başka Bir Dilden Geliyorsanız
- Ders
- Çalıştırın
- Sık Yapılan Hatalar
- Özet Çıkarımlar
- Ödev
- Ödev 12: Bir Kelime Dizini
 - Metin
 - Birinci Bölüm: Sayma
 - İkinci Bölüm: Dizinleme
 - Üçüncü Bölüm: Bir Girdiyi Kaybedin
 - Kabul Kriterleri
 - İleri Seviye
 - İpucu, ilk üçünüz değişip duruyorsa
- Modül 13: Lambda'lar
 - Neler Öğreneceksiniz
 - Başka Bir Dilden Geliyorsanız
 - Ders
 - Çalıştırın
 - Sık Yapılan Hatalar
 - Özet Çıkarımlar
 - Ödev
- Ödev 13: Bir Kural Motoru
 - Veri
 - Birinci Bölüm: Birleştirilmiş Predicate'ler
 - İkinci Bölüm: Dört Metot Referansı Biçimi
 - Üçüncü Bölüm: Yakalama Kuralı
 - Kabul Kriterleri
 - İleri Seviye
 - İpucu, `compose` tip denetiminden geçmiyorsa
- Modül 14: Stream'ler
 - Neler Öğreneceksiniz
 - Başka Bir Dilden Geliyorsanız
 - Ders
 - Çalıştırın
 - Sık Yapılan Hatalar
 - Özet Çıkarımlar
 - Ödev
- Ödev 14: Log Analizi

- Veri
- Birinci Bölüm: Gruplama
- İkinci Bölüm: Gatherer'lar
- Üçüncü Bölüm: Yarıştırın
- Dördüncü Bölüm: Tembellik ve Tek Kullanım
- Kabul Kriterleri
- İleri Seviye
- İpucu, farklarınız bir eleman fazla çıkıyorsa
- Modül 15: Optional
 - Neler Öğreneceksiniz
 - Başka Bir Dilden Geliyorsanız
 - Ders
 - Çalıştırın
 - Sık Yapılan Hatalar
 - Özet Çıkarımlar
 - Ödev
- Ödev 15: Bir Rehber
 - Veri
 - Birinci Bölüm: Kutudan Çıkarmadan Arama
 - İkinci Bölüm: Tuzağı Kanıtlayın
 - Üçüncü Bölüm: Dört Yanlış Kullanımı Düzeltin
 - Dördüncü Bölüm: Sizi Neye Karşı Korumadığı
 - Kabul Kriterleri
 - İleri Seviye
 - İpucu, birinci bölümün `flatMap` noktası oturmuyorsa
- Modül 16: Dosyalar ve Giriş/Çıkış
 - Neler Öğreneceksiniz
 - Başka Bir Dilden Geliyorsanız
 - Ders
 - Çalıştırın
 - Sık Yapılan Hatalar
 - Özet Çıkarımlar
 - Ödev
- Ödev 16: Bir Log Aracı
 - Birinci Bölüm: Akışla Okuyun, Yutmayın
 - İkinci Bölüm: Filtrelenmiş Bir Kopya Yazın
 - Üçüncü Bölüm: Yol Doğrulayıcı
 - Dördüncü Bölüm: Bir Şey Söyleyen İstisnalar
 - Kabul Kriterleri

- İleri Seviye
- İpucu, `/etc/passwd` elinizden kaçıyor
- Modül 17: Standart Kütüphane Turu
 - Neler Öğreneceksiniz
 - Başka Bir Dilden Geliyorsanız
 - Ders
 - Çalıştırın
 - Sık Yapılan Hatalar
 - Özet Çıkarımlar
 - Ödev
- Ödev 17: Mart'tan Sağ Çıkan Bir Planlayıcı
 - Birinci Bölüm: Aylık Faturalama
 - İkinci Bölüm: Yaz Saatini Aşan Bir Toplantı
 - Üçüncü Bölüm: Dört Math Tuzağı
 - Dördüncü Bölüm: Tekrarlanabilir Örneklem ve Biçimlendirme
 - Kabul Kriterleri
 - İleri Seviye
 - İpucu, çıktınız çalıştırmalar arasında değişiyorsa
- Modül 18: Eşzamanlılık Temelleri
 - Neler Öğreneceksiniz
 - Başka Bir Dilden Geliyorsanız
 - Ders
 - Çalıştırın
 - Sık Yapılan Hatalar
 - Özet Çıkarımlar
 - Ödev
- Ödev 18: Sayamayan Bir Sayaç
 - Birinci Bölüm: Dört Sayaç
 - İkinci Bölüm: Hiç Bitmeyen Bir Döngü
 - Üçüncü Bölüm: Denetle Sonra Davran
 - Dördüncü Bölüm: Kimsenin Duymadığı Başarısızlık
 - Kabul Kriterleri
 - İleri Seviye
 - İpucu, güvensiz sayaç doğru çıkıyorsa
- Modül 19: Sanal İş Parçacıkları
 - Neler Öğreneceksiniz
 - Başka Bir Dilden Geliyorsanız
 - Ders
 - Çalıştırın

- Sık Yapılan Hatalar
- Özet Çıkarımlar
- Ödev
- Ödev 19: Bir İndirme Servisi
 - Birinci Bölüm: Ölçün
 - İkinci Bölüm: Kaldırdığınız Sınır
 - Üçüncü Bölüm: Kapsam Değerleri
 - Dördüncü Bölüm: Değişmeyen Şey
 - Kabul Kriterleri
 - İleri Seviye
 - İpucu, sayacınızın tepe değeri düşük görünüyorsa
- Modül 20: Paketler ve Modüller
 - Neler Öğreneceksiniz
 - Başka Bir Dilden Geliyorsanız
 - Ders
 - Çalıştırın
 - Sık Yapılan Hatalar
 - Özet Çıkarımlar
 - Ödev
- Ödev 20: Kendiniz Kurun
 - Birinci Bölüm: İki Paket
 - İkinci Bölüm: Paketleyin
 - Üçüncü Bölüm: Üç Başlangıç Hatası
 - Dördüncü Bölüm: Pratikte Modüller
 - Kabul Kriterleri
 - İleri Seviye
 - İpucu, ikinci başarısızlık başarısız olmuyorsa
- Modül 21: Kodunuzu Test Etmek
 - Neler Öğreneceksiniz
 - Başka Bir Dilden Geliyorsanız
 - Ders
 - Çalıştırın
 - Sık Yapılan Hatalar
 - Özet Çıkarımlar
 - Ödev
- Ödev 21: Aksi Hâlde İçer Aktaracağınız Çatıyı Kendiniz Kurun
 - Birinci Bölüm: Daha İyi Bir Çatı
 - İkinci Bölüm: Yalıtım
 - Üçüncü Bölüm: `assert` İfadesinin Gizlediği Hata

- Dördüncü Bölüm: Hangi Denetim, Nereye
- Kabul Kriterleri
- İleri Seviye
- İpucu, üçüncü bölümde silme gerçekleşiyorsa
- Modül 22: Yenilikler, Dürüstçe
 - Neler Öğreneceksiniz
 - Başka Bir Dilden Geliyorsanız
 - Ders
 - Çalıştırın
 - Sık Yapılan Hatalar
 - Özet Çıkarımlar
 - Ödev
- Ödev 22: Kendiniz Araştırın
 - Birinci Bölüm: Üç Özellik, Araştırılmış
 - İkinci Bölüm: Üç Soru
 - Üçüncü Bölüm: Bir Geçmiş Bölümü Okuyun
 - Dördüncü Bölüm: Sürümü Düzgün Denetleyin
 - Kabul Kriterleri
 - İleri Seviye
 - İpucu, History bölümünü bulamıyorsanız
- Uygulama Planı: Java 27 Öğrenim Müfredatı
 - Doğrulanmış Ön Koşullar
 - Açık Sorularda Alınan Kararlar
 - Bileşenler
 - Bu Sıra Neden
 - Riskler ve Önlemler
 - Paralel ve Ardışık
 - Doğrulama Kontrol Noktaları
- Görev Listesi: Java 27 Öğrenim Müfredatı
 - scaffold — iskelet
 - template — format referansı
 - core-language — çekirdek dil
 - type-system — tip sistemi
 - functional — fonksiyonel
 - platform — platform
 - release-literacy — sürüm okuryazarlığı
 - integration — bütünleştirme

Java 27, Java'ya Yeni Başlayan Programcılar İçin

Başka bir dilde kod yazan ve şimdi Java'ya ihtiyacı olan geliştiriciler için kendi hızında ilerleyen bir müfredat. Değişkenin ya da döngünün ne olduğunu yeniden anlatmaz. Java'nın bu işleri nasıl yaptığını ve sizi nerede şaşırtacağını anlatır.

Yirmi iki modül. Her birinde bir ders, yorumları *nedenini* açıklayan çalıştırılabilir örnekler, özet çıkarımlar ve referans çözümüyle birlikte bir ödev var.

Bu depodaki her örnek JDK 27 üzerinde çalıştırıldı. Derslerde alıntılanan her derleyici hatası, bozuk kodu gerçekten derleyerek üretildi ve her sayı hatırlanarak değil ölçülerek yazıldı.

Bu, müfredatın Türkçe sürümüdür. İngilizce özgün metinler bir üst dizindedir. Neyin çevrilip neyin çevrilmediği [CEVIRI-NOTLARI.md](#) dosyasında açıklanmıştır. Kısaca: kod, komutlar ve derleyici çıktıları İngilizce bırakıldı.

Burada "Java 27" ne demek

[JDK 27](#) 15 Eylül 2026'da genel kullanıma açıldı. LTS **değil**, ve kesinleşen özellikleri neredeyse tamamen çalışma zamanı seviyesinde: varsayılan çöp toplayıcı olarak G1, küçültülmüş nesne başlıkları, post-kuantum TLS anahtar değişimi, uçuş kaydedici maskeleme. Bu listede Java yazma biçiminizi değiştiren hiçbir şey yok.

Bu yüzden müfredat kendi başlığı konusunda dürüst:

JDK 27 çalışma ortamı ve araç zinciridir. Öğrendiğiniz dil ise [JDK 25 LTS](#) sürümüne kadar birikmiş kararlı Java'dır.

Modül 22, JDK 26 ve 27'nin gerçekte neyi değiştirdiğini, LTS olmayan sürümlerin neden sürüm numarasının düşündüğünden daha az önemli olduğunu ve bir JEP'i kendi başınıza nasıl okuyacağınızı anlatır.

Başlarken

Tek gereksinim JDK 27. Maven yok, Gradle yok, bu depoda hiçbir üçüncü parti kütüphane yok.

```
java -version # görmeniz gereken: java version "27"
```

Kurulum

Platform	Komut
macOS (Homebrew)	<code>brew install openjdk@27</code>
Linux (SDKMAN)	<code>sdk install java 27-open</code>
Windows (winget)	<code>winget install Microsoft.OpenJDK.27</code>
Hepsi	Adoptium veya Oracle üzerinden indirin

Ona yönlendirme

```
# macOS
export JAVA_HOME=$(/usr/libexec/java_home -v 27)

# Linux
export JAVA_HOME=/usr/lib/jvm/jdk-27
export PATH="$JAVA_HOME/bin:$PATH"

# Windows PowerShell
$env:JAVA_HOME = "C:\Program Files\Microsoft\jdk-27"
$env:PATH = "$env:JAVA_HOME\bin;$env:PATH"
```

İlk çalıştırmanız

```
java modules/01-getting-started/examples/HelloWorld.java
```

Derleme adımı yok, build dosyası yok. Modül 01 gerçekte ne olduğunu anlatıyor.

Müfredat

Sırayla ilerleyin. Her modülün metni, öncekileri okuduğunuzu varsayar.

#	Modül	Konusu
01	getting-started	JVM, JRE ve JDK; dosyayı doğrudan çalıştırma; derleyici hatası okuma
02	variables-and-types	İlkel tipler ve referanslar, otomatik kutulama tuzakları, <code>var</code> , metin blokları
03	control-flow	Doğruluk kavramının olmayışı, switch ifadeleri, kapsayıcılık, etiketli break
04	methods	Nesneler için de geçerli olan değer geçişi, aşırı yükleme çözümlemesi, varargs
05	classes-and-objects	Klasik <code>main</code> , kapsülleme, statik fabrikalar, esnek kurucular
06	inheritance-and-polymorphism	Dinamik gönderim, <code>abstract</code> ile <code>final</code> , <code>equals</code> / <code>hashCode</code> sözleşmesi
07	interfaces	Yetenekler, varsayılan metotlar, elmas çakışması, kırılğan temel sınıflar
08	records-and-enums	Record'lar, kompakt kurucular, yüzeysel değişmezlik, davranış taşıyan enum'lar
09	sealed-and-pattern-matching	Mühürlü tipler, record desenleri, koşullar, kapsayıcılığın neden kazanç olduğu
10	exceptions	Denetlenen ve denetlenmeyen istisnalar, try-with-resources, hataların kaybolduğu üç yol
11	generics	Tip silme ve yasakladıkları, yığın kirlenmesi, PECS, değişmezlik
12	collections	Uygulama seçimi, sıralı koleksiyonlar, anahtarların neden değişmemesi gerektiği
13	lambdas	Metot referansı biçimleri, yakalama ve etkin finallik, standart arayüzler
14	streams	Tembellik, collector'lar, gatherer'lar ve paralel veri yarışı
15	optional	Ne işe yaradığı, <code>orElse</code> ile <code>orElseGet</code> farkı, beş yanlış kullanım
16	files-and-io	<code>Path</code> ile <code>Files</code> , yol geçişi saldırısı, akış halinde okuma, <code>java.io.File</code> neden değil
17	standard-library-tour	String'ler, <code>java.time</code> , yaz saati, <code>Math</code> tuzakları, tekrarlanabilir rastgelelik
18	concurrency-basics	Yarış ve görünürlük ayrı tehlikeler, <code>synchronized</code> , atomikler, executor'lar
19	virtual-threads	Taşıyıcılar ve inme, kıyaslama, bugünkü sabitleme, kapsam değerleri
20	packages-and-modules	Dizin kuralı, sınıf yolu, jar'lar ve <code>--add-opens</code> neden var
21	testing-your-code	<code>assert</code> neden test aracı değil; 39 satırda bir test çatısı
22	whats-new	JDK 26 ve 27 gerçekte ne değiştirdi, LTS, ve JEP nasıl okunur

Bunu nasıl kullanmalı

Modül README'sini okuyun, örnekleri çalıştırın, sonra ödevi çözümü açmadan **önce** yapın.

```
modules/<nn>--<ad>/
├─ README.md      ders, özet çıkarımlarla biter
├─ examples/      çalıştırılabilir dosyalar, bolca yorumlu
├─ homework/      ödev ve kabul kriterleri

solutions/<nn>--<ad>/
├─ referans çözümler, bilerek ders ağacının dışında tutuldu
```

Çözümler kendinizi karşılaştırmanız içindir, önce okumanız için değil. Her biri neden o kararları verdiğini açıklayan yorumlar taşır, dolayısıyla kendi cevabınızı ürettikten sonra okumaya değer, öncesinde ise neredeyse değersizdir.

Her ödevin açık kabul kriterleri var. Sizin sürümünüz bunları karşılıyorsa, referanstan farklı olsa bile doğrudur.

Örnekleri çalıştırma

```
# sıradan bir örnek
java modules/02-variables-and-types/examples/TheAutoboxingTrap.java

# assert gerektiren biri
java -ea modules/21-testing-your-code/examples/UsingAssertions.java

# preview özellik gerektiren biri
java --enable-preview --source 27 modules/22-whats-
new/examples/PreviewStructuredConcurrency.java
```

Bazı örnekler **bilerek** başarısız olur, çünkü kendi ürettiğiniz bir derleyici hatası, size alıntılanandan daha çok şey öğretir. Her biri ilk on satırında ne beklediğini bildirir:

İşaretleyici	Anlamı
(yok)	Derlenir ve çalışır, sıfırla çıkar
// EXPECT: compile-error	Derlenmesi başarısız olmalı ; bir derleme hatası öğretiyor
// EXPECT: runtime-error	Derlenmeli, sonra çalışma zamanında çökmeli
// EXPECT: compile-only	Derlenmeli; tek başına çalıştırılmaz
// EXPECT: assertions	-ea ile çalıştırılır, assert ifadeleri etkin olur
// EXPECT: preview	--enable-preview --source 27 ile çalıştırılır

Her şeyi doğrulama

```
./scripts/verify-examples.sh # tüm depo
./scripts/verify-examples.sh modules/09-sealed-and-pattern-matching # tek modül
```

Script her örneği ve her referans çözümü çalıştırır, her birinin işaretleyicisinde bildirdiği gibi davrandığını denetler, ve JDK 27 dışında bir sürümde hiç çalışmayı reddeder. Çünkü araç zinciri yüzünden başarısız olan bir örnek size hiçbir şey öğretmez.

Bu deponun güncel durumu:

Modül	22
Örnek dosya	105
Referans çözüm	25
Doğrulan dosya	130
Tam doğrulama süresi	yaklaşık 70 saniye

PDF

Her ders ve ödev PDF olarak da mevcut, ayrıca tüm müfredat tek bir kitap halinde.

```
./scripts/build-pdfs.sh --tr # yalnızca Türkçe
./scripts/build-pdfs.sh # her iki dil
```

Türkçe PDF'ler `pdf-tr/` klasörüne yazılır.

Proje dokümanları

- [SPEC.md](#) — bu projenin ne inşa etmeye çalıştığı ve başarı kriterleri
- [tasks/plan.md](#) — uygulama planı
- [tasks/todo.md](#) — görev dökümü, issue'lara bağlı
- [CEVIRI-NOTLARI.md](#) — çeviri kararları

Bunun nasıl inşa edildiğine dair bir not

Bu derslerdeki her olgusal iddia hatırlanarak değil üretilerek yazıldı. Derleyici hataları bozuk kod derlenerek üretildi. Kıyaslamalar ölçüldü. JEP numaraları, yayın tarihleri ve LTS bilgileri openjdk.org

üzerinden okundu.

Bu disiplin yazım sırasında gerçek hatalar yakaladı: kendi iddiasını çürüten bir kompozisyon örneği, seçilen sayıların hatayı tamamen gizlediği bir kova indeksi gösterimi, ve varsayılandan iki tur fazlası çıkan bir "yenilikler" geçmişi. Her düzeltme git geçmişinde kayıtlı.

Burada yanlış bir şey bulursanız, JEP dizini bu dokümandan uzun yaşar. Kaynağa bakın.

Çeviri Notları

Bu klasör, müfredatın Türkçe sürümüdür. İngilizce özgün metinler bir üst dizinde durur ve ikisi aynı yapıyı paylaşır.

Neyin çevrildiği, neyin çevrilmediği

Öge	Karar
Ders metni, başlıklar, tablolar	Çevrildi
Kod blokları	Aynen bırakıldı
Java anahtar kelimeleri ve API adları	İngilizce (<code>record</code> , <code>sealed</code> , <code>Optional.orElseGet</code>)
Derleyici çıktısı ve hata mesajları	Aynen bırakıldı
Dosya ve izin adları	Aynen bırakıldı
Komutlar	Aynen bırakıldı

Kod neden çevrilmedi? Kod blokları `examples/` altındaki gerçek dosyalardan alıntıdır. Yorumları çevirseydim, alıntı artık çalıştırdığınız dosyayla eşleşmezdi. Ayrıca projenin en baştaki kuralı kodun ve kod yorumlarının İngilizce olmasıydı, çünkü okuyacağınız her gerçek Java kod tabanı öyle.

Hata mesajları neden çevrilmedi? Derleyici size İngilizce konuşur. Çevrilmiş bir hata mesajı aratılamaz ve terminalde gördüğünüzle eşleşmez.

Terim tercihleri

Türkçe yazılım camiasında yerleşmiş İngilizce terimleri olduğu gibi kullandım, ilk geçtikleri yerde Türkçe karşılığını parantez içinde verdim. Amaç, okuduğunuz kelimeyi arattığınızda sonuç bulabilmeniz.

İngilizce	Bu metinde
stream	stream (akış)
thread	iş parçacığı
virtual thread	sanal iş parçacığı
record	record
sealed	mühürlü (sealed)
immutable	değişmez
race condition	yarış koşulu
exception	istisna
checked exception	denetlenen istisna
garbage collector	çöp toplayıcı
classpath	sınıf yolu (classpath)
pattern matching	desen eşleme
boxing / autoboxing	kutulama / otomatik kutulama
generics	jenerikler
type erasure	tip silme
pinning	sabitlenme

`stream`, `record`, `lambda` gibi terimleri çevirmemek bilinçli bir karardır. Türkçe bir Java geliştiricisi bunları İngilizce kullanır, ve dokümantasyon ile hata mesajlarında da böyle geçerler.

Yapı

tr/	
├─ CEVIRI-NOTLARI.md	bu dosya
├─ README.md	genel bakış
├─ modules/<nn>--<ad>/	
│ └─ README.md	dersin çevirisi
│ └─ homework/README.md	ödevin çevirisi
└─ tasks/	proje dokümanları

Örnek kodlar ve çözümler çevrilmez; her iki dil de aynı `modules/*/examples/` ve `solutions/` dosyalarını kullanır.

PDF

```
./scripts/build-pdfs.sh --tr # yalnızca Türkçe  
./scripts/build-pdfs.sh    # her iki dil
```

Türkçe PDF'ler pdf-tr/ klasörüne yazılır.

Spesifikasyon: Java 27 Öğrenim Müfredatı

Durum: Teslim edildi, sonra eklendi — aşağıdaki *Ek 1* bölümüne bakın **Özgün kapsam:** 24 görevin hepsi tamam, 2026-09-23'te doğrulandı **Tarih:** 2026-09-23 **Aşama:** Specify → Plan → Tasks → Implement dizisinin 1. aşaması (Specify)

Yaptığım Varsayımlar

- Öğrenci başka bir dilde (Python, JavaScript, C#) zaten programlıyor ve **özellikle Java'ya** yeni. Modüller değişkenin ya da döngünün ne olduğunu yeniden anlatmıyor; Java'nın bunu nasıl yaptığını ve nerede ayrıştığını anlatıyor.
- Özgün istekteki "takeout" kelimesi **Özet Çıkarımlar** anlamına geliyor: her modülü kapatan kısa bir toparlama.
- İçerik ve tüm kod **İngilizce**. Bu spesifikasyon bir depo dokümanı, o da İngilizce.
- Müfredat yalnızca **çekirdek dili ve JDK standart kütüphanesini** kapsıyor. Spring yok, JDBC yok, build çatısı yok.
- Dersler ve örnek kodları **her modül klasörünün içinde birlikte** duruyor. Ödev çözümleri ise öğrenci kazara cevabı görmesin diye ayrı bir üst düzey `solutions/` ağacında.
- Build aracı yok. Örnekler tek dosya kaynak başlatma ile çalışıyor: `java Example.java`.

→ Bunlardan herhangi birini şimdi düzeltin, yoksa böyle devam ediyorum.

Amaç

Başka bir dil bilen bir programcıyı alıp JDK 27 üzerinde modern Java'da akıcı kılan, kendi hızında ilerleyen bir Java müfredatı kurmak.

Kullanıcı: Genel programlama okuryazarlığı olan ve sıfır Java deneyimi olan bir geliştirici.

Başarı neye benziyor: Öğrenci numaralı modüllerde sırayla ilerliyor. Her modül bir alanı hafif ve sade bir tonda anlatıyor, yorumları sözdizimini tekrarlamak yerine *nedenini* öğreten çalıştırılabilir örneklerle destekliyor. Her modül Özet Çıkarımlar ve bir ödevle kapanıyor; referans çözümler var ama dersin dışında duruyor.

Bu müfredatı şekillendiren bir bulgu

JDK 27, **15 Eylül 2026**'da genel kullanıma açıldı. **LTS değil** (güncel LTS JDK 25). Teslim ettiği JEP'ler neredeyse tamamen çalışma zamanı seviyesinde:

JEP	Başlık	Durum
523	Make G1 the Default Garbage Collector in All Environments	Final
527	Post-Quantum Hybrid Key Exchange for TLS 1.3	Final
534	Compact Object Headers by Default	Final
536	JFR In-Process Data Redaction	Final
531	Lazy Constants	Üçüncü Preview
532	Primitive Types in Patterns, instanceof, and switch	Beşinci Preview
533	Structured Concurrency	Yedinci Preview
537	Vector API	On İkinci İnkübatör
538	PEM Encodings of Cryptographic Objects	Üçüncü Preview

Sonuç: "Java 27 öğrenmek", dokuz yeni dil özelliği öğrenmek demek değil. O listedeki hiçbir şey yeni başlayanın kod yazma biçimini değiştirmiyor. Dürüst çerçeve, ve bu müfredatın benimsediği çerçeve şu: *JDK 27 çalışma ortamı ve araç zinciridir; öğrendiğiniz dil ise JDK 25 LTS'e kadar birikmiş kararlı Java'dır.* Kapanış modülü 26 ve 27'nin gerçekte neyi değiştirdiğini, LTS olmayan sürümlerin neden düşündürdüğünden az önem taşıdığını ve bir JEP'in nasıl okunacağını anlatıyor.

Giriş kararı

JEP 512 (Compact Source Files and Instance Main Methods) **JDK 25'te kesinleşti**, dolayısıyla 27'de kararlı. Bu yüzden Modül 01 şöyle açılıyor:

```
void main() {  
    IO.println("Hello, World!");  
}
```

`public class` yok, `static` yok, `String[] args` yok, `import` yok. Modül 05 ise öğrenciyi bilinçli biçimde klasik forma "mezun ediyor", çünkü karşılaşacağı her gerçek kod tabanı, öğretici ve Stack Overflow cevabı onu kullanıyor. Kolay biçimi öğretip klasik olanı hiç açıklamamak, onu gerçek Java okuyamaz hâlde bırakırdı.

Teknoloji Yığını

Öge	Seçim
JDK	OpenJDK 27 (GA 2026-09-15)
Dil seviyesi	JDK 25'e kadar kararlı özellikler; preview özellikler yalnızca son modülde, açıkça etiketli
Build aracı	Yok
Bağımlılıklar	Yok
Ders biçimi	Markdown
Örnek biçimi	Tek dosya <code>.java</code> , doğrudan çalıştırılabilir

Proje Yapısı

Java-27-edu/ ├─ README.md ├─ SPEC.md ├─ scripts/ │ └─ verify-examples.sh bildirir ├─ modules/ │ ├── 01-getting-started/ │ │ ├── README.md │ │ ├── examples/ │ │ └─ homework/ │ └─ ... └─ solutions/ └─ 01-getting-started/	→ İçindekiler, nasıl başlanır, JDK kurulumu → Bu doküman → Her örneği derleyip çalıştırır, başarısızlıkları bildirir → Ders metni + Özet Çıkarımlar
---	--

Modül kimlikleri kebab-case, sıfır dolgulu ve bir kez atandıktan sonra sabit. Proje ortasında asla yeniden adlandırılmıyorlar.

Kod Stili

Dosya başına tek kavram. Yorumlar *nedenini* öğretiyor; asla sözdizimini anlatmıyor.

```
// Module 02 – examples/AutoboxingTrap.java
//
// Java has two parallel worlds for numbers: primitives (int) and objects
// (Integer). Most of the time Java quietly converts between them, which is
// convenient right up until the moment it bites you. Here is the bite.

void main() {
    Integer a = 127;
    Integer b = 127;
    IO.println(a == b);    // true – and this is the misleading one
}
```

Gelenekler: standart Java adlandırma (`PascalCase` tipler, `camelCase` üyeler, `UPPER_SNAKE` sabitler), 4 boşluk girinti, 100 karakteri aşmayan satırlar, her örnek argümentsiz tek başına çalıştırılabilir.

Test Stratejisi

Burada birim testi yapılacak bir uygulama yok; teslim edilen şey öğretim materyali. Doğruluk, **her örneğin gerçekten çalışması** demek.

- `scripts/verify-examples.sh`, `modules/**/examples/*.java` ve `solutions/**/*.java` dosyalarını gezer, her birini çalıştırır, sıfır olmayan her çıkışta ya da derleme hatasında başarısız olur.
 - Bilerek derleme hatası gösteren örnekler `// EXPECT: compile-error` başlığı taşır ve atlanmak yerine başarısız olmaları denetlenir.
 - Preview bayrağı gerektirenler `// EXPECT: preview` taşır ve `--enable-preview --source 27` ile çalıştırılır.
 - Bir modül, örnekleri, ödev metni ve referans çözümünü scriptten geçene kadar "bitti" sayılmaz.
 - Modül 21 testi `assert` ve elle yazılmış bir çatıyla öğretir, sıfır bağımlılık kuralını bozmadan, ve gerçek bir JUnit kurulumunun ne katacağını anlatır.
-

Sınırlar

Her zaman

- Tüm ders metnini, kodu, yorumları ve tanımlayıcıları İngilizce yazın.
- Her modüle bir Özet Çıkarımlar bölümü ve bir ödev verin.
- Bir modülü bitmiş saymadan önce her örneği çalıştırın.
- Yaygın bir hatayı anlatırken öğrencinin göreceği gerçek hata mesajını gösterin.
- Bir özellik preview, inkübatör ya da JDK sürümüne bağlıysa bunu açıkça belirtin.

Önce sorun

- JUnit dahil herhangi bir üçüncü parti bağımlılık eklemek.

- Maven ya da Gradle getirmek.
- Onaylanmış modül kimliklerini, sıralarını ya da müfredat taslağını değiştirmek.
- Kapsamı çatılara, Android'e ya da web geliştirmeye genişletmek.

Asla

- Bilerek olduğu işaretlenmeden derlenmeyen bir örnek yayımlamayın.
- JEP numarası, yayın tarihi ya da özellik iddiası uydurmayın. openjdk.org üzerinden doğrulayın.
- Tanımlamadan önce jargon kullanmayın.
- Kullanımdan kaldırılmış ya da önerilmeyen bir pratiği öyle olduğunu belirtmeden sunmayın.
- Ödev çözümlerini modül klasörünün içine koymayın.

Başarı Kriterleri

Sekizinin hepsi 2026-09-23'te teslim edilen depoya karşı denetlendi. Her satır yalnızca sağlandığını değil, **nasıl doğrulandığını** kaydediyor.

- ✓ **1. `./scripts/verify-examples.sh` tüm depoda sıfırla çıkıyor.** 130 dosya, hepsi bildirdiği gibi davranıyor, yaklaşık 70 saniyede.
- ✓ **2. 22 modül klasörünün hepsi var, her biri `README.md`, en az iki çalıştırılabilir dosya içeren `examples/` ve `homework/README.md` içeriyor.** Script ile her modül dizininde denetlendi. Toplam 105 örnek dosya, modül başına en az 4.
- ✓ **3. Her modül `README`'si yedi şablon bölümünün hepsini, Özet Çıkarımlar dahil içeriyor.** 22 dosyanın her birinde başlıklar grep ile denetlendi. Son geçişte `22-whats-new` içinde bir eksik bulundu ve düzeltildi.
- ✓ **4. Her ödevin `solutions/<modül-kimliği>/` altında temiz çalışan eşleşen bir referans çözümü var.** 22 modül için 22 çözüm dizini, 25 çözüm dosyası, hepsi yukarıdaki doğrulama çalıştırmasına dahil.
- ✓ **5. Kök `README.md` her modülü sırayla bağlıyor ve JDK 27 kurulumunu belgeliyor.** 22 bağlantının hepsi script ile doğrulandı. Kurulum macOS, Linux ve Windows için, her biri `JAVA_HOME` talimatıyla belgelendi.
- [~] **6. Hiçbir dosya İngilizce olmayan metin, tanımlayıcı ya da yorum içermiyor. Ek 1 ile geçersiz kılındı.** İngilizce müfredat ve tüm kod için doğru, ki kriterin koruduğu şey buydu. Depo bütünü için artık doğru değil, çünkü `tr/` altında her dersin Türkçe çevirisi duruyor. Yerine 9 ile 12 arası kriterler geçti.
- ✓ **7. Depoda hiçbir yerde üçüncü parti bağımlılık yok.** `pom.xml`, `build.gradle` ya da `.jar` yok. 130 dosyadaki her import `java.*`, `javax.*` ya da `jdk.*` altında çözülüyor, artı bir bağımlılık değil dil özelliği olan JEP 511 `import module java.base` bildirimi.

****Ek 1 notu.**** `scripts/build-pdfs.sh`, pandoc ve Chromium tabanlı bir tarayıcı gerektiriyor. Bunlar dokümantasyon üretmek için çağrılan dış araçlar*, müfredatın kodunun bağlandığı kütüphaneler değil. Öğrencinin derlediği ya da çalıştırdığı hiçbir şey onlara bağlı değil, ve PDF'ler isteğe bağlı çıktı. Kriter Java kodu için yazıldığı gibi geçerli, ve bu not ayrımın varsayılmak yerine belirtilmesi için var.

- ✓ **8. Bir JDK sürümü hakkındaki her olgusal iddia openjdk.org yayın verisine dayanıyor.** Modül ve kök README'lerde 32 ayrı openjdk.org bağlantısı. JDK 26 ve 27 JEP tabloları, JDK 25 kesinleşme iddiaları ve JEP 491'in sanal iş parçacığı sabitletmesini kaldırması, yazım sırasında hatırlanmak yerine kaynağından alındı.

Açık Sorular, Çözöldükleri Hâliyle

- JDK 27 kurulumu.** Çözöldü. Uygulama başlamadan önce JDK 27 (derleme 27+35-2325) kuruldu ve her örnek ona karşı doğrulandı.
- Modöl 21 ve JUnit.** Sıfır bağımlılık kuralı lehine çözöldü. Test, çıplak `assert` artı 39 satırlık elle yazılmış bir çatıyla öğretiliyor, ve modöl gerçek bir projede JUnit benimsemenin gerekçesini kuruyor. Kısıt dersi iyileştirdi: çalışan bir çatının tamamını görmek, bir framework'ün ne yaptığını onu kullanmaktan daha iyi açıklıyor.
- Preview özellikler.** Önerildiği gibi çözöldü. Yapılandırılmış eşzamanlılık (JEP 533) ve ilkel tip desenleri (JEP 532) yalnızca Modöl 22'de, JEP numarası ve preview turuyla etiketli olarak, `--enable-preview` arkasında görünüyor. Modöl preview özellik yayımlamaya açıkça karşı çıkıyor, çünkü bir preview sınıf dosyası bayrakla bile farklı bir sürüm tarafından reddediliyor.

Uygulama Sırasında Yapılan Düzeltmeler

Kaydedildi, çünkü spesifikasyonun sınırı iddia uydurmayı yasaklıyordu ve buna bağlı kalmak yanlış olan birkaç şeyi açığa çıkardı:

- Bir kompozisyon örneği, `ArrayList` alt sınıflamanın eklemeleri iki kez sayacağını iddia ediyordu. Çalıştırmak bunu çürüttü: modern `ArrayList.addAll` toplu kopyalıyor. Hatayı gösteren koleksiyon `HashSet`, ve örnek artık ikisini de gösteriyor, ki bu kırılğan temel sınıf noktasını özgününün yapacağından daha iyi kuruyor.
- Alıntılanan üç derleyici hatası satır numarası yanlış. Hepsi hatalar üretilerek düzeltildi.
- Bir kova indeksi gösterimi 8 kova kullanıyordu, ve `Integer.MIN_VALUE` 8'e tam bölünüyor, dolayısıyla hem bozuk hem doğru biçim `0` döndürüyordu ve hata görünmezdi. 7'ye değiştirildi.
- Bir ödevin belirtilen çıktısı referans çözümünün yazdığınıyla çelişiyordu. Ödev çıktıları artık gözle değil çözüm çıktısına karşı `diff` ile karşılaştırılıyor.

- Yapılandırılmış eşzamanlılık preview geçmişi hafızadan yedi tur olarak yazılmıştı. JEP 533'ün History bölümünü okumak, öncesinde iki inkübatör turu daha olduğunu gösterdi, yani dokuz sürüm.

Ek 1 — PDF çıktısı ve Türkçe çeviri

Tarih: 2026-09-23, özgün 24 görev teslim edildikten sonra.

Spesifikasyon onaylandıktan sonra iki yetenek eklendi, ve hiçbiri inşa edildiği sırada herhangi bir spesifikasyonda ya da görev listesinde yer almıyordu. `spec-driven-development` becerisi tam olarak bu durumu bir kırmızı bayrak olarak isimlendiriyor: "*Hiçbir spesifikasyonda ya da görev listesinde geçmeyen özellikleri uygulamak.*" Bu ek, dokümanı gerçekte teslim edilenle yeniden hizalıyor.

Eklenen ne

Yetenek	Ne olduğu
PDF çıktısı	<code>scripts/build-pdfs.sh</code> her Markdown dosyasını PDF'e, artı dil başına birleşik bir kitaba dönüştürüyor. <code>scripts/pdf.css</code> onları baskı için biçimlendiriyor.
İki dilli içerik	<code>tr/</code> altında 48 Markdown dosyasının tamamının Türkçe çevirisi, artı çeviri kurallarını kaydeden <code>tr/CEVIRI-NOTLARI.md</code> .

Çevirinin bilerek çevirmediği

Burada kaydedildi çünkü bu bir tercih değil bir kısıt, ve özgün spesifikasyonun Sınırlar bölümünde ikinci bir dili kapsayan hiçbir kural yoktu:

- Kod blokları aynen İngilizce kalır.** `examples/` altındaki gerçek dosyaları alınıyorlar, dolayısıyla yorumlarını çevirmek öğrencinin gerçekten çalıştığı şeyle örtüşmeyi bozardı.
- Derleyici çıktısı aynen kalır.** Çevrilmiş bir hata mesajı aratılamaz ve terminalde yazılanla eşleşmez.
- Komutlar, dosya yolları, Java API adları ve JEP numaraları olduğu gibi kalır.**
- `tr/` altında hiç Java kaynağı yok.** Her iki dil aynı `modules/*/examples/` ve `solutions/` ağaçlarını paylaşıyor.

Bu ekin yakaladığı bir ihlal

Yukarıdaki kuralı yazmak onu denetlemeyi zorunlu kıldı, ve denetim başarısız oldu. **On sekiz Türkçe dosyadaki otuz beş Java kod bloğunun yorumları çevrilmişti**, ki bu çevirinin kendi belgelediği kuralın doğrudan ihlali. İngilizce özgünlerine geri alındılar, ve 10 numaralı kriter artık tekrarına karşı koruyor.

Kuralı akılda tutmak yerine yazmanın değeri bu: kural açıktı, yazılmıştı, ve herhangi bir şey denetlemeden önce yine de kırk beş kez çiğnendi.

Güncellenmiş başarı kriterleri

1 ile 5 arası, 7 ve 8 numaralı kriterler değişmeden geçerli. 6 numaralı kriter geçersiz kılındı.

- ✓ **9. Her İngilizce Markdown dosyasının `tr/` altında aynı yolda bir Türkçe karşılığı var.** 48'inin hepsi script ile doğrulandı.
- ✓ **10. Hiçbir Türkçe dosyadaki hiçbir `java` kod bloğu İngilizce olmayan metin içermiyor.** 35 tanesi bulunup geri alındıktan sonra sıfır ihlal.
- ✓ **11. Java kod blokları iki dil arasında birebir hizalı.** Eşleşen 44 dosyanın her birinde sayı aynı, dolayısıyla okuyucu her iki sürümü de aynı koda karşı takip edebiliyor.
- ✓ **12. `tr/` altında hiç Java kaynak dosyası yok.** Sıfır. Her iki dil tek bir örnek ve çözüm kümesini paylaşıyor, dolayısıyla doğrulama scripti ikisini birden kapsıyor.
- ✓ **13. `scripts/build-pdfs.sh` her iki dili de üretiyor.** `pdf/` altında 49 İngilizce PDF, `pdf-tr/` altında 50 Türkçe, her birinde birleşik bir kitap, yaklaşık 100 saniyede.
- ✓ **14. Çeviri örnek doğrulamasını etkilemiyor.** `./scripts/verify-examples.sh` hâlâ 130 dosyanın bildirdiği gibi davrandığını raporluyor.

Süreç notu

Doğru sıra, her iki yeteneği de inşa etmeden **önce** bu spesifikasyonu güncellemek, sonra plan ve görevleri ondan türetmek olurdu. Bu olmadı: ikisi de doğrudan istek üzerine inşa edildi ve spesifikasyon bayat kaldı, 6 numaralı kriter yanlış olduğu hâlde `[x]` olarak duruyordu.

Eki yazmak, aksi hâlde yayımlanmış olacak kod bloğu ihlalini açığa çıkaran şey oldu. Ders, bu müfredatın Modül 21'inin testler hakkında söylediğiyle aynı: kimsenin denetlemediği bir kural zaten çığnemiş bir kuraldır.

Modül 01: Başlarken

Kod yazmayı zaten biliyorsunuz. Bu modül makinenizde Java'yı çalıştırır, bir iş yapan en küçük programı gösterir ve onu çalıştırdığınızda gerçekte ne olduğunu anlatır. Yarım saat sürer ve JDK, JRE ile JVM arasındaki farkı bir daha asla karıştırmazsınız.

Neler Öğreneceksiniz

- JDK 27'yi nasıl çalışır hale getireceğiniz ve doğru sürümde olduğunuzu nasıl doğrulayacağınız
- JVM, JRE ve JDK'nın her birinin ne olduğu ve insanların neden karıştırdığı
- Sınıf bildirimi ya da build aracı olmadan Java programı yazıp çalıştırma
- Derleyicinin dosyanızla ne yaptığı, iddia edilerek değil gösterilerek
- Bir derleyici hatasını okuyup şikâyet ettiği tam noktayı bulma

Başka Bir Dilden Geliyorsanız

Python ya da JavaScript'te bir dosya yazıp çalıştırırsınız. Tek adım. Java geleneksel olarak size iki adım yaptırdı: kaynağı bytecode'a derlemek, sonra bytecode'u çalıştırmak. "Java hantaldır" ününün başladığı yer bu ek adımdır.

Bu iki şeyin ikisi de değişti ve bu modül değişimle tanıştığınız yer.

	Python / JavaScript	Java, geleneksel olarak	Java 27
Script çalıştırma	<code>python app.py</code>	<code>javac App.java</code> sonra <code>java App</code>	<code>java App.java</code>
En küçük program	<code>print("hi")</code>	<code>sınıf + static main + String[] args</code>	<code>void main() + IO.println</code>
Hataların bulunması	çalışma zamanında	derleme zamanında	derleme zamanında

Çalışma biçiminizi asıl değiştirecek olan son satır. Python, nadiren girilen bir dalda yazım hatası olduğunu size kullanıcı üretimde çarpınca söyler. Java program daha başlamadan söyler. Yaptığınız takas budur: baştan biraz daha tören, sonrasında çok daha az sürpriz.

Ders

Üç harfli kelimeler

İnsanlar JVM, JRE ve JDK'yı birbirinin yerine kullanıyor ve sonra birbirlerini karıştırıyorlar. Bunlar üst üste duran üç ayrı şey:

- **JVM**, Java Virtual Machine. Bytecode'u çalıştıran şey. Derlenmiş Java'nın macOS, Linux ve Windows'ta değişmeden çalışmasının sebebi. Kodunuz bir işlemciyi değil, bunu hedefler.
- **JRE**, Java Runtime Environment. JVM artı standart kütüphane. Java'yı *çalıştırmaya* yeter, derlemeye yetmez. Artık nadiren ayrı kurulur.
- **JDK**, Java Development Kit. JRE artı araçlar: derleyici `javac`, sökücü `javap`, `jar`, `javadoc`. Kurduğunuz şey budur.

Siz bir JDK kurdunuz. Doğrulayın:

```
java -version
```

`java version "27"` görmelisiniz. Başka bir şeyse buradaki örnekler dersle hiç ilgisi olmayan sebeplerle başarısız olabilir.

En küçük program

`examples/HelloWorld.java` dosyasını açın. Dosyanın tamamı:

```
void main() {  
    IO.println("Hello, World!");  
}
```

Daha önce Java gördüyseniz, orada olmayanları fark edin. `public class` yok, `static` yok, `String[] args` yok, yazdırma için `import` yok. Bu tören yirmi beş yıl boyunca Java'nın en çok alay edilen özelliği idi ve JEP 512, JDK 25'te onu yazma zorunluluğunu kaldırdı. Preview değil, kararlı.

`IO` `import` olmadan erişilebilir çünkü `java.lang` içinde yaşar ve her Java dosyası onu otomatik olarak içe aktarır.

Tören kaybolmadı

Burada yavaşlamaya değer, çünkü Modül 05'te sizi ısıracak bir yanlış anlamayı önler.

Java yeni başlayanlar için ikinci ve daha basit bir dil edinmedi. Töreni sizin yerinize derleyici yazıyor. Kanıtlayın:

```
javac -d /tmp/java27 modules/01-getting-started/examples/WhatTheCompilerWrites.java  
javap -cp /tmp/java27 WhatTheCompilerWrites
```

Dönen çıktı:

```
final class WhatTheCompilerWrites {  
    WhatTheCompilerWrites();  
    void main();  
}
```

İşte sınıfınız. Derleyici onu dosyadan sonra adlandırdı, `final` işaretledi, argümansız bir kurucu verdi ve `main` 'i bir **örnek** metodu olarak bıraktı. Özelliğin adının "instance main methods" olmasının sebebi bu: çalışma zamanı bu sınıftan bir nesne kurar ve `main` 'i onun üzerinde çağırır.

Yani baştan sona sıradan Java yazıyorsunuz, sadece daha azını yazarak. Buradaki her şey Modül 05'teki klasik forma aktarılır, çünkü zaten o formun kendisi.

Çalıştırmak gerçekte ne yapar

`java HelloWorld.java` çalıştırdığınızda sırayla üç şey olur:

1. `javac` kaynağınızı JVM'in anladığı bir komut kümesi olan **bytecode**'a derler. Java 11'den beri bu bellekte olabiliyor, `.class` dosyası görmemenizin sebebi bu.
2. JVM o bytecode'u yükler ve doğrular.
3. JVM onu çalıştırır, sıcak yolları ilerledikçe yerel makine koduna derler.

Birinci adım yazım hatalarınızın yakalandığı yer. Derlenen, statik tipli bir dilin tüm pazarlığı budur: derleyici, hiçbir çalışmadan önce her satırı okur.

Uzun yoldan da yapabilirsiniz, ve bazen isteyeceksiniz:

```
javac Greet.java    # Greet.class üretir  
java Greet          # dikkat: .java yok, .class yok, sadece sınıf adı
```

Derleyicinin size söylediğini okumak

Yeni bir dildeki ilk gerçek beceriniz kod yazmak değil. Hatayı okuyup nereye bakacağınızı bilmektir.

[examples/ReadTheError.java](#) bilerek bozuk. Çalıştırın:

```
ReadTheError.java:29: error: ';' expected  
    IO.println("I am missing something")  
                                ^  
1 error
```

Şapka işaretinden geriye doğru okuyun:

- `^` tam sütunu gösterir
- `:29` satır
- `';' expected` derleyicinin orada bulmayı beklediği şey

Java niyetinizi tahmin etmiyor. Kodunuzu o şapkaya kadar ayrıştırdı ve o konumda tek geçerli belirteç noktalı virgüldü. Şapkanın gösterdiği yeri düzeltin, tüm satırı değil.

Çalıştırın

```
# En küçük program
java modules/01-getting-started/examples/HelloWorld.java

# Derleyicinin sizin için ürettiğini görün
java modules/01-getting-started/examples/WhatTheCompilerWrites.java

# Etkileşimli, bunu kendiniz çalıştırın
java modules/01-getting-started/examples/AskingForInput.java

# Bilerek bozuk. Hatayı okuyun.
java modules/01-getting-started/examples/ReadTheError.java

# Bu modüldeki her örneğin hâlâ doğru davrandığını kontrol edin
./scripts/verify-examples.sh modules/01-getting-started
```

Sık Yapılan Hatalar

Elinizde yalnızca `Greet.java` varken `java Greet` çalıştırmak.

```
Error: Could not find or load main class Greet
Caused by: java.lang.ClassNotFoundException: Greet
```

`java Greet`, `Greet` adında derlenmiş bytecode arar. `java Greet.java` kaynağı çalıştırır. Fark `.java` uzantısında, ve hata mesajı bunu size söylemiyor.

`I0.println` yerine `io.println` yazmak.

```
error: cannot find symbol
  io.println("hi");
  ^
symbol:   variable io
location: class E3
```

Java büyük küçük harfe duyarlıdır ve `I0` bir sınıf adıdır. "Cannot find symbol" ilk ayınızda en sık göreceğiniz hatadır. Neredeyse her zaman bir yazım hatası, eksik bir import ya da henüz var olmayan bir ad demektir.

Sol taraf metin olduğunda `+` işaretinin toplayacağını sanmak.

```
"Next year is " + year + 1    // "Next year is 2026"
"Next year is " + (year + 1)  // "Next year is 2027"
```

`+` soldan sağa değerlendirilir. Bir taraf `String` olduğu anda ondan sonraki her `+` "topla" değil "sona ekle" demektir. Aritmetiği parantez içine alın.

`System.out.println` artık geçersiz sanmak. Değil. `IO.println`, kompakt kaynak dosyalarında bulunan bir kısaltma. Okuyacağınız her gerçek kod tabanı `System.out.println` ya da bir loglama çatısı kullanıyor, ve ikisi de burada çalışıyor.

Özet Çıkarımlar

- **Kurduğunuz şey JDK'dır.** Bytecode çalıştıran JVM'i ve onu üreten araçları içerir. Bu müfredat için `java -version` çıktısı 27 demelidir.
- **`java File.java` bir kaynak dosyasını doğrudan çalıştırır.** Build aracı yok, ayrı derleme adımı yok, geride `.class` dosyası kalmaz.
- **Kompakt kaynak dosyaları gerçek Java'dır.** Sınıf bildirimini, kurucuyu ve `final` niteleyicisini derleyici sizin için yazar. `javap` size tam olarak ne yazdığını gösterir.
- **Buradaki `main` bir örnek metodudur,** dolayısıyla `this` vardır. Klasik `public static void main` bunu taşımaz, Modül 05 nedenini anlatıyor.
- **Derleme hataları belirsiz değil, konumludur.** Şapka sütunu, numara satırı verir, mesaj da derleyicinin ne beklediğini söyler.
- **Hiçbir şey değiştirilmedi.** `System.out.println`, sınıflar ve `static main` hepsi hâlâ çalışıyor. Farklı bir dil değil, daha az yazmayı öğreniyorsunuz.

Ödev

[homework/README.md](#)

Az önce gördüklerinizden küçük etkileşimli bir program kurun, sonra bilerek bozup derleyicinin ne dediğini okuyun. Referans çözüm [solutions/01-getting-started/](#) altında, ama önce kendiniz deneyin. Cevabı okumak, hatayı alıp düzeltmekten çok daha az şey öğretir.

Ödev 01: Hakkımda

Modül 01'in kapsadıklarını kullanarak küçük etkileşimli bir program kurun, sonra bilerek bozup derleyicinin size ne söylediğini okuyun.

Görev

Bu deponun dışında herhangi bir yerde, örneğin bir karalama klasöründe, `AboutMe.java` adında bir dosya oluşturun. Onu **kompakt kaynak dosyası** olarak yazın, yani `class` bildirimi ve `static` olmadan.

Şunları yapmalı:

1. Kullanıcının adını sorsun.
2. Doğduğu yılı sorsun.
3. İçinde bulunulan yılı sorsun.
4. Adını ve yaklaşık yaşını içeren küçük bir profil yazdırsın; yaş, içinde bulunulan yıldan doğum yılının çıkarılmasıdır.

Örnek çalıştırma, yazılan girdiler istemlerin ardından gösterilmiştir:

```
What is your name? Ada
What year were you born? 1990
What year is it now? 2026

--- About Ada ---
Name: Ada
Age this year: 36
```

Sonra Bozun

Çalıştıktan sonra:

1. Dosyanın ortasında bir yerden noktalı virgül silin.
2. Tekrar çalıştırın.
3. Dosyanın altına, satır numarası ve şapka satırı dahil, tam hata mesajını kaydeden bir yorum ekleyin.
4. Noktalı virgülü geri koyun.

Ödevin asıl amacı bu adım. Kariyeriniz boyunca binlerce derleyici hatası okuyacaksınız ve ilki baskı altındayken olmamalı.

Kabul Kriterleri

- ☐ Dosya ayrı bir `javac` adımı olmadan `java AboutMe.java` ile çalışıyor.

- ☐ `class` bildirimi ve `static` anahtar kelimesi yok.
- ☐ Soru sormak için `IO.readLine`, yazdırmak için `IO.println` kullanıyor.
- ☐ Yaş, kullanıcı tarafından yazılarak değil aritmetikle hesaplanıyor.
- ☐ Yaş birleştirilmiş metin olarak değil sayı olarak yazdırılıyor. `"Age: " + year - birth` derlenmez, nedenini çözmek alıştırmanın bir parçası.
- ☐ Dosyanın altındaki bir yorum, ürettiğiniz ve okuduğunuz gerçek bir derleyici hatasını kaydediyor.

İleri Seviye

Kendi dosyanıza `javap` çalıştırın ve derleyicinin uydurduğu sınıf adını bulun:

```
javac -d /tmp/hw01 AboutMe.java
javap -cp /tmp/hw01 AboutMe
```

Üretilen sınıf, Modül 01'in söylediğiyle eşleşiyor mu? `main` metodunun imzası nedir ve statik mi?

İpucu, yaşta takıldıysanız

`IO.readLine` kullanıcı rakam yazsa bile size her zaman bir `String` verir. Java `"1990"` değerini sessizce sayı gibi kullanmaz. `AskingForInput.java` dosyasının metni `int` değerine nasıl çevirdiğine tekrar bakın.

Modül 02: Değişkenler ve Tipler

Değişkenin ne olduğunu biliyorsunuz. Java'dan ihtiyacınız olan şey daha dar: hangi tipler var, bunların hangisi nesne hangisi değil, ve bu ikisi arasındaki sınır sizi nerede yanıltacak. O sınır bu modülün tamamı.

Neler Öğreneceksiniz

- İlkel tipler ile referanslar arasındaki ayrım ve Java'nın bunu neden hâlâ koruduğu
- İnsanları şaşırtan ama hiçbir hata olmayan beş sayısal davranış
- Otomatik kutulama ve ısırdığı iki yer
- `==` operatörünün nesneler hakkında neden yalan söylediği ve yerine ne kullanılacağı
- Değişmez nesneler olarak String'ler ve metin blokları
- `var`, ne yaptığı ve nerede yardımı kesildiği

Başka Bir Dilden Geliyorsanız

Python ve JavaScript'te sayılar dahil her şey nesnedir. Java iki ayrı dünyayı korur ve dikişi hissedeceksiniz.

	Python / JS	Java
<code>3</code>	bir nesne	ilkel tip, 32 bit, tahsis yok
Tanımlama	<code>x = 3</code>	<code>int x = 3;</code> ya da <code>var x = 3;</code>
Sonradan tip değiştirme	sorun değil	imkânsız, tip derleme zamanında sabitlenir
Tam sayı taşması	sayıyı büyütür	sessizce başa sarar
<code>"a" == "a"</code>	içeriği karşılaştırır	kimliği karşılaştırır, <code>.equals</code> kullanın
Doğruluk kavramı	<code>if (items)</code> çalışır	yalnızca <code>boolean</code> kabul edilir

Son satır bir not hak ediyor. Java'da doğruluk kavramı yok. `if (someString)` derlenmez. `if (!someString.isEmpty())` yazarsınız, ve bu uzunluk bilinçlidir: hangi değerlerin yanlış sayıldığına dair ezberlenecek bir kural yoktur.

Ders

İki tür değer

Sekiz ilkel tip var ve bu liste sonsuza dek kapalı:

```
boolean    int    long    double
char       byte   short   float
```

İlkel tipli bir değişken değerin kendisini tutar. Java'daki diğer her şey bir **referanstır**: değişken, başka bir yerde yaşayan bir nesnenin adresini tutar.

Fark, bir şeyi kopyalayana kadar görünmez. `TwoKindsOfValue.java` dosyasına bakın:

```
int[] original = {1, 2, 3};
int[] alias = original; // copies the reference, not the array
alias[0] = 999;
// original[0] is now 999
```

İlkel tipi kopyalamak değeri kopyalar. Referansı kopyalamak etiketi kopyalar ve iki etiket de aynı nesneyi gösterir. Modül 04, metod argümanlarına baktığımızda buna geri dönüyor.

Pratikte `int`, `long`, `double`, `boolean` ve `char` kullanacaksınız. Diğer üçü çoğunlukla karşılaşmanız pek olası olmayan bellek kısıtlı durumlar için var.

Sayılar sizi şaşırtacak

`NumbersWillSurpriseYou.java` dosyasını çalıştırın. Gerçek çıktı:

```
7 / 2 = 3
7 / 2.0 = 3.5
Integer.MAX_VALUE + 1 = -2147483648
Math.addExact says: integer overflow
0.1 + 0.2 = 0.30000000000000004
7 % -2 = 1
-7 % 2 = -1
Math.floorMod(-7, 2) = 1
'A' + 1 = 66
```

Orada beş şey oluyor:

1. **Tam sayı bölmesi kalanı atar.** `7 / 2` sonucu `3`. Bir işleneni `double` yapın, `3.5` alırsınız. İki işlenen de değişkenken bunu göremezsiniz, tehlikeli olan yanı bu.
2. **Taşma sessizce başa sarar.** `int` 32 bittir ve en büyüğüne bir eklemek istisna olmadan en küçüğe döner. İstedığınızde `Math.addExact` şikâyet eder.
3. **`double` çoğu ondalığı tam tutamaz.** `0.1` ikilik tabanda devirli bir kesirdir. Parayı asla `double` içinde saklamayın. Kuruş cinsinden bir `long` ya da `BigDecimal` kullanın.
4. **% sol işlenenin işaretini korur.** `-7 % 2` sonucu `-1`, `1` değil. Matematiksel modülo istiyorsanız o `Math.floorMod`.
5. **`char` bir sayıdır.** `'A' + 1` sonucu `66`, çünkü `+` `char`'ı `int`'e yükseltti. `B` almak için `(char)` ile geri dönüştürün.

Otomatik kutulama ve iki ısırığı

Her ilkel tipin bir nesne ikizi var: `int` / `Integer`, `double` / `Double` ve devamı. Java bunlar arasında otomatik dönüşüm yapar. Buna otomatik kutulama denir ve şu iki âna kadar gerçekten kullanışlıdır.

Birinci ısırık, kutulanmış değerlerde `==`. `TheAutoboxingTrap.java` dosyasından:

```
127 == 127 : true
128 == 128 : false
```

Aynı kod, aynı türden değer, zıt cevap. Java `-128..127` aralığı için `Integer` nesnelerini ön belleğe alır, dolayısıyla bu aralıkta `==` bir nesneyi kendisiyle karşılaştırır. Aralığın dışında iki nesne alırsınız ve `==` doğru biçimde farklı nesneler olduklarını bildirir.

Bozuk olan bir şey yok. Nesnelerde `==` her zaman "aynı nesne" anlamına geliyordu, ön bellek bunu sizden gizliyordu.

Kural: ilkel tiplerde `==`, nesnelerde `.equals()`.

İkinci ısırık, null kutudan çıkarma. `NullUnboxing.java` bilerek çöker:

```
Exception in thread "main" java.lang.NullPointerException:
  Cannot invoke "java.lang.Integer.intValue()" because the return value of
  "java.util.Map.get(Object)" is null
```

`Map.get` olmayan bir anahtar için `null` döndürür ve bunu bir `int` değişkene atamak Java'yı hiçliğin üzerinde `intValue()` çağırma zorlar. Mesajın ne kadar belirgin olduğuna dikkat edin: `null` döndüren çağrıyı ve onun üzerinde başarısız olan çağrıyı isimlendiriyor. Modern Java NPE'leri hata ayıklamanın çoğunu sizin için yapar.

String'ler

String'ler nesnedir ve **değişmezdir**. Hiçbir metot, üzerinde çağırdığınız string'i değiştirmez.

`StringsAndTextBlocks.java` dosyasından:

```
String name = "ada";
name.toUpperCase();           // computes "ADA" and throws it away
name = name.toUpperCase();    // you have to keep the result
```

İlk satır insanların gerçekten yaptığı bir hatadır ve hiçbir şey sizi uyarmaz.

`==` tuzağı burada da geçerli, `Integer` ile aynı sebepten:

```
literal == literal : true      // derleyici aynı sabitleri havuzlar
literal == new      : false    // new String() ayrı bir nesne zorlar
equals              : true
```

Hemen edinilmeye değer bir alışkanlık: sabiti sola koyun.

```
"java".equals(maybeNull)    // false
maybeNull.equals("java")    // NullPointerException
```

Metin blokları kaçış karakteri olmadan çok satırlı string verir:

```
String json = """
{
    "name": "Ada"
}""";
```

Java ortak baştaki girintiyi, en az girintili satırdan ölçerek siler, böylece blok kodunuzla hizalanır ama o boşluk değere karışmaz.

var

`var`, derleyiciden tipi sağ taraftan çıkarmasını ister. Bu kısmı dikkatle okuyun, çünkü insanların yanlış anladığı nokta burası:

`var` dinamik tiplere değildir. Değişkenin hâlâ tam olarak bir tipi vardır, sonsuza dek sabitlenmiş, derleme zamanında denetlenir. Siz sadece yazmadınız.

```
var x = "hello";
x = 42;    // error: incompatible types: int cannot be converted to String
```

Tip uzun ve zaten görünür olduğunda hakkını verir:

```
var scores = new HashMap<String, List<Integer>>();
```

Sağ taraf ne aldığınızı söylemiyorsa zarar verir:

```
var result = service.process(input);    // now go look up process()
```

Yapamadığı üç şey, hepsi derleme hatası:

```
var a;                // nothing to infer from
var b = null;          // null belongs to every reference type
var c = () -> 1;        // a lambda needs a target type
```

`VarCannotInferNull.java` ikincisini gösteriyor. Ayrıca yalnızca yerel değişkenler içindir. Alanlar, parametreler ve dönüş tipleri hâlâ yazılmalıdır, bu da API'leri okunabilir tutar.

Çalıştırın

```
java modules/02-variables-and-types/examples/TwoKindsOfValue.java
java modules/02-variables-and-types/examples/NumbersWillSurpriseYou.java
java modules/02-variables-and-types/examples/TheAutoboxingTrap.java
java modules/02-variables-and-types/examples/StringsAndTextBlocks.java
java modules/02-variables-and-types/examples/VarAndInference.java

# Bu ikisi bilerek başarısız olur. Ne dediklerini okuyun.
java modules/02-variables-and-types/examples/NullUnboxing.java
java modules/02-variables-and-types/examples/VarCannotInferNull.java

./scripts/verify-examples.sh modules/02-variables-and-types
```

Sık Yapılan Hatalar

String karşılaştırmak için `==` kullanmak. Testinizde kısa sabitlerle çalışır, üretimde çalışma zamanında oluşturulan string'lerle başarısız olur. Yeni başlayanların yazdığı en yaygın Java hatası budur. `.equals` kullanın.

Parayı `double` içinde saklamak. `0.1 + 0.2` sonucu `0.3` değildir. Kuruş cinsinden `long` ya da `BigDecimal` kullanın. Muhasebe sistemleri yanlış yazılmıştır ve tutarsızlık aylar sonra ortaya çıkar.

String metotlarının yeni bir String döndürdüğünü unutmak. Tek başına bir satırda `name.strip();` hiçbir şey yapmaz.

`Map.get` sonucunun bir `int` içine kutudan çıkmasına izin vermek. Eksik her anahtar `NullPointerException` olur. `getOrDefault` kullanın ya da `Integer` olarak tutup kontrol edin.

Sıcak döngü içinde kutulama.

```
Long sum = 0L; // a million allocations
for (long i = 0; i < 1_000_000; i++) sum += i;
```

`Long` yerine `long` yazmak her tahsisi kaldırır. İki sürüm neredeyse aynı görünür, bunu bilmeye değmesinin sebebi de tam olarak bu.

`long big = 9000000000L;` yazmak, `L` olmadan. Sabit önce `int` olarak ayrıştırılır, sığmaz, dolayısıyla derlenmez.

Özet Çıkarımlar

- **İki dünya.** İlkel tipler değer tutar, referanslar adres tutar. Referansı kopyalamak nesneyi değil etiketi kopyalar.
- `==` "aynı nesne mi" diye sorar, `.equals` "aynı içerik mi" diye. İlkel olmayan her şeyde istediğiniz `.equals`.

- **Integer önbelleği** `==` işaretini 128'in altında doğru gösterir. Onu tehlikeli yapan sınır değil, bu.
- **Sayılar sabit genişliktedir.** Tam sayı bölmesi keser, taşma başa sarar ve `double` çoğu ondalığı tam tutamaz.
- **String'ler asla değişmez.** Her metot yenisini döndürür, sonucu saklayın.
- `var` çıkarımdır, dinamik tipleme değil. Tip hâlâ sabit ve hâlâ denetleniyor.
- **Java'da doğruluk kavramı yoktur.** `if` içine yalnızca `boolean` girer, ve bu bir özelliktir.

Ödev

[homework/README.md](#)

Para işlemeyi doğru yapan küçük bir fiş hesaplayıcı kurun, sonra bu modülün iki tuzağını bilerek yeniden üretin. Referans çözüm [solutions/02-variables-and-types/](#) altında.

Ödev 02: Hesabı Tutan Fiş

Bir dükkânın kasası yanlış yuvarlıyor ve kimse sebebini bulamıyor. Siz yuvarlamayan birini yazacaksınız, bu da para tipini doğru seçmek demek.

Görev

`Receipt.java` adında bir kompakt kaynak dosyası oluşturun. Şunları yapmalı:

- Üç ürünü ad ve fiyatla tutsun: `19.99`, `4.05` ve `0.99`.
- Her ürünü kendi satırında yazdırsın.
- Ara toplamı, yüzde 20 vergiyi ve toplamı yazdırsın.
- Aritmetiği tam doğru yapsın, ve *biçimlendirmeyi* de tam doğru yapsın.

Hedef çıktı:

Bread	19.99
Milk	4.05
Gum	0.99

Subtotal	25.03
Tax (20%)	5.01
Total	30.04

Para Kuralı

Fiyatlar için `double` **kullanmayın**. Modül 02 nedenini gösterdi: `0.1 + 0.2` sonucu `0.3` değildir ve her satırda bir kuruşun yüzde biri kadar sapan bir kasa eninde sonunda denetimden geçemez.

Her fiyatı **kuruş** sayan bir `long` olarak saklayın, yani `19.99` değeri `1999` olur. Tüm aritmetiği tam kuruşlarla yapın ve yalnızca yazdırma anında metne çevirin.

Kuruşu para birimi olarak biçimlendirmek için tam sayı bölmesi ve kalan size iki yarıyı da verir:

```
long cents = 405;
String shown = (cents / 100) + "." + (cents % 100);
```

Bunu süt için çalıştırın ve ne aldığınıza bakın. Yanlış, ve *neden* yanlış olduğunu çözmek bu alıştırmamanın en değerli kısmı. Bu fişin altı satırından dördü aynı hataya yakalanıyor, yani savuşturabileceğiniz bir uç durum değil.

Yuvarlama Kararı

2503 kuruşun yüzde 20'si tam olarak 500,6 kuruş. Bir kuruşun onda altısını tahsil edemezsiniz, dolayısıyla ona ne olacağına karar vermelisiniz ve kodunuz bu kararı kazara değil görünür biçimde vermelidir.

Ne seçerseniz seçin, ne seçtiğinizi ve nedenini söyleyen bir yorum bırakın.

Sonra İki Tuzağı Yeniden Üretin

`Traps.java` adında ikinci bir dosya ekleyin ve şunları gösterip açıklayın:

- Kutulanmış tam sayılarda `==` tuzağı.** Aynı türden değerle `true` veren bir karşılaştırma ve `false` veren bir tane yazdırın, sonra gerçekte ne demek istediğinizi gösteren `.equals` sonucunu yazdırın.
- String değişmezliği.** Bir String metodunu sonucu saklamadan çağırın, değişmeyen değeri yazdırın, sonra doğru şekilde yapın.

Her birinin *nedenini* kendi kelimelerinizle açıklayan bir yorum gerekiyor. Örneklerdeki ifadeleri kopyalamak sayılmaz. Amaç, geri anlatabilmeniz.

Kabul Kriterleri

- ☐ Her iki dosya da `java Receipt.java` ve `java Traps.java` ile çalışıyor
- ☐ `Receipt.java` içinde hiçbir yerde `double` ya da `float` geçmiyor
- ☐ Vergi hesaplanıyor, sabit olarak yazılmıyor
- ☐ `4.05` değeri `4.5` değil `4.05` olarak yazdırılıyor
- ☐ Ara toplam, vergi ve toplam tam olarak iki ondalık basamakla yazdırılıyor
- ☐ Sütunlar hizalı
- ☐ Bir yorum, kuruşun kesrine ne olduğunu ve nedenini belirtiyor
- ☐ `Traps.java` kutulanmış `==` için hem `true` hem `false` veren durum gösteriyor
- ☐ Her tuzak kendi kelimelerinizle bir açıklama taşıyor

İleri Seviye

Kasa artık yüzde 7,5 oranına ihtiyaç duyuyor. 2503 kuruş üzerinden yüzde 7,5 vergi 187,725 kuruş eder.

Bunu uygulayın, kesre ne olacağına karar verin ve seçiminizi bir yorumda savunun. Tek bir doğru cevap yok, tam da bu yüzden düşünmeye değer. Gerçek ödeme sistemlerinin bunun için yayımlanmış kuralları vardır, bazıları toplam yerine satır bazında yuvarlamayı gerektirir ve bu da yine farklı bir toplam verir.

İpucu, biçimlendirme sizi zorluyorsa

`String.format` dolgu ve sıfırla doldurmayı anlar:

```
String.format("%-12s %8s", name, amount)    // left-align 12, right-align 8
String.format("%d.%02d", whole, remainder) // %02d forces two digits
```

İkincisi, daha önce bulduğunuz hatanın çözümü.

Modül 03: Akış Kontrolü

Java'da döngüler ve koşullar, daha önce kod yazmış biri için pek sürpriz içermez. Burada üç şey dikkatinizi hak ediyor: Java'da doğruluk kavramı yok, modern `switch` tanımış olabileceğinizden gerçekten daha iyi, ve etiketli break çoğu dilde bulunmayan bir sözdizimi.

Neler Öğreneceksiniz

- `if (name)` neden derlenmez ve bunun neden bir özellik olduğu
- Kısa devre değerlendirmenin bir optimizasyon değil doğruluk aracı olduğu
- Switch ifadeleri, çok etiketli durumlar ve `yield`
- Kapsayıcılık denetimi ve neden sıklıkla `default` dalı istemediğiniz
- Her döngü biçimi ve hangisine uzanacağınız
- İç içe döngüler için etiketli break

Başka Bir Dilden Geliyorsanız

	Python / JS	Java
Koşul	herhangi bir değer	yalnızca <code>boolean</code>
Boş string yanlış sayılır	evet	yanlış sayılan hiçbir şey yok
String üzerinde <code>switch</code>	JS evet, Python 3.10+ match	evet
Alt duruma düşme	JS evet	yalnızca eski ifade biçiminde
İç içe döngüden çıkma	bayrak değişkeni	<code>break</code> etiket;
<code>for x in xs</code>	yerleşik	<code>for (var x : xs)</code>

En büyük uyum, ilk satır. Java `if (items)` yazmanıza izin vermez ve boş bir listenin ne anlama geldiğine sizin adınıza karar vermez.

Ders

Doğruluk kavramı yok

```
String name = "";
if (name) { } // does not compile
if (0) { }    // does not compile
```

Karşılaştırmayı açıkça yazarsınız:

```
if (!name.isEmpty()) { ... }
```

Daha fazla yazım, karşılığında ezberlenecek yanlış değerler tablosu ve `"0"` ya da `[]` yanlış sayılır mı tartışması yok. Her koşul tam olarak neyi test ettiğini söyler.

Koruma olarak kısa devre

`&&` sol işlenen cevabı zaten belirlediğinde sağ işleneni atlar. Bu yalnızca bir performans ayrıntısı değil, null'a karşı korunma biçiminizdir:

```
if (maybe != null && maybe.length() > 3) { ... }
```

İkisini ters çevirin ve fırlatır. `&&` işlenenlerinin sırası bir doğruluk kararıdır.

`&` ve `|` de boolean üzerinde çalışır ve kısa devre **yapmaz**. Orada onları neredeyse hiç istemezsiniz. Asıl işleri bit düzeyinde:

```
6 & 3 = 2      6 | 3 = 7      6 ^ 3 = 5      6 << 1 = 12
```

Switch, iyi sürüm

Bir switch **ifadesi** değer üretir. [SwitchExpressions.java](#) dosyasına bakın:

```
String kind = switch (day) {  
    case 1, 2, 3, 4, 5 -> "weekday";  
    case 6, 7 -> "weekend";  
    default -> "not a day";  
};
```

`break` yok, dal başına birden fazla etiket var, ve bütünü atayabileceğiniz bir ifade. Bir dal birden fazla ifade gerektirdiğinde blok ve `yield` kullanın:

```
int workload = switch (day) {  
    case 6, 7 -> {  
        int base = 10;  
        yield base * 2;  
    }  
    default -> 8;  
};
```

`yield` var çünkü `return` tüm metottan çıkmaya çalışırdı.

Kapsayıcılık, asıl önemli kısım

Bu metodun `default` dalı yok ve derleyici kabul ediyor:

```
String describe(Light light) {  
    return switch (light) {  
        case RED -> "stop";  
        case AMBER -> "get ready";  
        case GREEN -> "go";  
    };  
}
```

Üç sabit, üç dal, eksiksiz. Şimdi `SwitchMustBeExhaustive.java` dosyasının yaptığı gibi `GREEN` dalını silin:

```
error: the switch expression does not cover all possible input values  
    return switch (light) {  
        ^  
missing patterns:  
    Light.GREEN
```

Derleyici unuttuğunuz durumu ismen söylüyor.

Bu, ilk bakışta tersmiş gibi duran bir tavsiyeye götürür:

Girdi kapalı bir kümeyseniz `default` dalını yazmayın.

`default` olmadan `Light` enum'una dördüncü bir sabit eklemek, güncellenmesi gereken her switch'te derlemeyi kırar ve hata her birini tek tek gösterir. `default` ile hepsi derlenmeye devam eder ve yeni sabiti sessizce yedeğe yönlendirir. Bugün aldığınız bir derleme hatası, üretimde bulacağınız bir hatadan iyidir.

Alt duruma düşme ve neden kaldırıldı

Eski ifade biçimi, bir şey durdurana kadar sonraki durumlara girerek çalışır.

`WhyFallThroughWentAway.java` `n = 2` ile ikisini de gösteriyor:

```
n = 2, old statement form:  
    two  
    three  
n = 2, arrow form:  
    two
```

Eski biçim 2 durumuyla eşleşti, yazdırdı, sonra hiçbir şey onu durdurmadığı için 3 durumuna düştü. Bir eksik anahtar kelime, ve yanlış davranış doğru gibi okunuyor.

Alt duruma düşme ara sıra durumları gruplamak için kullanılırdı. Ok biçimi bunu virgülle ve risk olmadan yapıyor: `case 1, 2, 3 -> handleSmall();`

Döngüler

Dört biçim, ve seçim genellikle apaçık. `Loops.java` dosyasından:

Biçim	Ne zaman
<code>for (var x : xs)</code>	indekse ihtiyacınız yoksa. Çoğu zaman.
<code>for (int i = 0; ...)</code>	indeks, birden farklı adım ya da geriye gitme gerekiyorsa
<code>while</code>	tekrar sayısı baştan bilinmiyorsa
<code>do-while</code>	gövde en az bir kez çalışmalıysa. Nadir.

Etiketli break, çoğu dilde karşılığı olmayan tek döngü sözdizimidir. İçteki bir döngüden dıştakinden çıkar:

```
search:
for (int row = 0; row < grid.length; row++) {
    for (int col = 0; col < grid[row].length; col++) {
        if (grid[row][col] == target) {
            break search; // leaves both loops
        }
    }
}
```

Onsuz bir bayrak değişkeni ve dış döngüde fazladan bir koşul gerekir, ki bu da çürüten türden koddur. Ölçülü kullanın: bir metotta birden fazla etiket genellikle o metodun bölünmek istediği anlamına gelir.

Çalıştırın

```
java modules/03-control-flow/examples/OperatorsAndConditions.java
java modules/03-control-flow/examples/SwitchExpressions.java
java modules/03-control-flow/examples/WhyFallThroughWentAway.java
java modules/03-control-flow/examples/Loops.java

# Bilerek başarısız olur. Hangi durumu isimlendirdiğini okuyun.
java modules/03-control-flow/examples/SwitchMustBeExhaustive.java

./scripts/verify-examples.sh modules/03-control-flow
```

Sık Yapılan Hatalar

&& işlenenlerini ters sırada yazmak. `maybe.length() > 3 && maybe != null` null kontrolü hiç çalışmadan fırlatır.

Enum üzerindeki switch'e alışkanlıkla default eklemek. Bugün derlenir ve yarın eklemeyi unuttuğunuz her durumu gizler.

Eski ifade biçiminde break unutmak. Yeni kod yazıyorsanız ok biçimini kullanın ve sorun ortaya çıkamaz.

Bileşik atamanın yalnızca bir kısaltma olduğunu sanmak.

```
byte b = 10;
b += 300;      // compiles, prints 54
b = b + 300;   // does not compile
```

`+=` sessizce bir dönüşüm ekler. `10 + 300` sonucu `310`, ve bunu byte'a kesmek `54` verir. Açık biçim ise reddeder, ki ikisinden güvenli olanı budur.

Çalışma zamanında oluşturulan String'lerde `==` kullanmak.

```
a1 == a2 : true      // ikisi de derleme zamanı sabiti, havuzlanmış
a1 == a3 : false     // a3 bir StringBuilder ile oluşturuldu
```

Aynı metin, string'in ne zaman var olduğuna bağlı olarak farklı cevaplar.

Taşımaya dayanan bir döngü koşulu yazmak.

```
for (int i = 1; i > 0; i *= 2) { }
```

Bu gerçekten biter, ama yalnızca `i` değeri `Integer.MAX_VALUE` sınırını aşıp negatife sardığı için, ki bu yazdığınız sebep değil.

Özet Çıkarımlar

- `if` içine yalnızca `boolean` girer. Doğruluk kavramı ve yanlış değerler tablosu yok.
- `&&` ve `||` kısa devre yapar, bu da işlenen sırasını bir doğruluk meselesi haline getirir.
- **Switch ifadeleri değer üretir**, dal başına birkaç etiket alır ve asla alt duruma düşmez.
- `yield` bir switch bloğundan değer döndürür; `return` metottan çıkardı.
- **Kapalı kümede `default` yazmayın**, böylece durum eklemek sessizce düşmek yerine derlemeyi kırsın.
- **İndekse ihtiyacınız yoksa gelişmiş `for` kullanın.**
- `break` etiket; bayrak değişkeni olmadan iç içe döngülerden çıkar.

Ödev

[homework/README.md](#)

Switch ifadesiyle küçük bir not verme aracı yazın, sonra alt duruma düşme hatasını derleme hatasına çevirin. Referans çözüm [solutions/03-control-flow/](#) altında.

Ödev 03: Notlar ve Izgaralar

İki küçük program. Birincisi işe uygun koşul yapısını seçmenizi istiyor. İkincisi, etiketli break'in neden var olduğunu önce onsuz yaşatarak gösteriyor.

Birinci Bölüm: Notlar

Grades.java oluşturun. Şu puan listesi verildiğinde:

```
int[] scores = {95, 83, 71, 64, 42, 100, 0};
```

Her puanı harf notu ve kısa bir yorumla yazdırın, sonra sınıf ortalamasını notuyla birlikte yazdırın.

Not sınırları: 90 ve üzeri A, 80 ve üzeri B, 70 ve üzeri C, 60 ve üzeri D, altı F.

Hedef çıktı:

```
95 A    excellent
83 B    solid
71 C    passing
64 D    scraped through
42 F    see me
100 A    excellent
0  F    see me
```

```
Class average: 65 (D)
```

Vermeniz Gereken Karar

Bu iki işten biri switch ifadesine uygun, diğeri değil.

- Bir **puanı** harfe çevirmek
- Bir **harfi** yoruma çevirmek

Hangisinin hangisi olduğunu bulun, switch yalnızca uyduğu yerde kullanın ve diğerinin neden if zinciri olduğunu bir yorumda açıklayın. Uymadığı yere switch zorlamak, hiç kullanmamaktan daha kötü bir cevaptır.

İkinci Bölüm: Izgarada Arama

FindInGrid.java oluşturun. Bu ızgarada bir hedef değer arayın ve satır ile sütununu bildirin, ya da bulunmadığını söyleyin:

```
int[][] grid = {  
    { 7, 12, 3},  
    { 9, 5, 21},  
    {14, 2, 18}  
};
```

Aramayı **iki kez** yazın:

1. `withFlag` — bir `boolean found` değişkeni ve düz `break` kullanarak
2. `withLabel` — etiketli break kullanarak

İkisini de mevcut olan `21` ve olmayan `99` için çalıştırın.

Sonra ikisini karşılaştıran bir yorum yazın. Özellikle: bayraklı sürümün gerektirdiği ve ızgarada aramayla hiç ilgisi olmayan şeyleri sıralayın.

Kabul Kriterleri

- ☐ Her iki dosya da `java Grades.java` ve `java FindInGrid.java` ile çalışıyor
- ☐ `Grades.java` iki işten tam olarak biri için `switch` ifadesi kullanıyor
- ☐ Bir yorum, diğerinin neden switch olmadığını açıklıyor
- ☐ Sütunlar hizalı; `100` ve `0` ikisi de doğru görünüyor
- ☐ `FindInGrid.java` hem `withFlag` hem `withLabel` içeriyor
- ☐ Her iki sürüm de `21` ve `99` için birebir aynı çıktıyı üretiyor
- ☐ Bir yorum, bayraklı sürümün gerektirdiği fazladan işleri sıralıyor

İleri Seviye

Her iki döngüden de çıkmanın, ne bayrak ne etiket gerektiren üçüncü bir yolu var. Bulun, `withExtractedMethod` olarak uygulayın ve etiketli break'e ne zaman tercih edeceğinizi söyleyin.

İpucu, switch derlenmiyorsa

Bir switch ifadesi her olası girdi için bir değer üretmek zorundadır. `int` ya da `String` üzerinde bu, durumları sayarak her şeyi kapsayamayacağınız anlamına gelir, dolayısıyla derleyici `default` ister. Her sabiti listelenmiş bir `enum` üzerinde ise istemez. Bu fark, iki işten birinin neden switch'e daha uygun olduğunun tüm sebebi.

Modül 04: Metotlar

Metot bildirmek sürpriz içermez. Bu modüldeki üç şey içerir: Java'nın argümanları nasıl geçirdiği, aşırı yüklemeler arasında nasıl seçim yaptığı ve `static` kelimesinin gerçekte ne anlama geldiği. Bunlardan ilki dildeki en çok yanlış anlaşılan konudur.

Neler Öğreneceksiniz

- Java'nın nesneler için de neden her zaman değer geçişi yaptığı ve bundan ne çıktığı
- Aşırı yükleme çözümleme sırası ve `List` içinde yol açtığı gerçek hata
- Dönüş tipinin neden metot imzasının parçası olmadığı
- Varargs ve `null` değerinin dönüşüm gerektirdiği tek durum
- `static` ne zaman doğru, ne zaman sizi sonradan engelliyor

Başka Bir Dilden Geliyorsanız

	Python / JS	Java
Aynı ad, farklı parametreler	mümkün değil, varsayılan kullanın	aşırı yükleme
Varsayılan parametre değerleri	<code>def f(x=1)</code>	yok, onun yerine aşırı yükleyin
İsimli argümanlar	<code>f(x=1)</code>	yok
Değişken sayıda argüman	<code>*args</code>	<code>int... nums</code>
Argüman geçişi	nesneye referans, ad yeniden bağlama yerel	aynı semantik, farklı sözcükler

Son satır önemli. Bir nesne geçirdiğinizde Python ve Java **aynı** davranır. İkisi de fonksiyonun nesneyi değiştirmesine izin verir, ikisi de çağırının değişkenini yeniden bağlamasına izin vermez. Tartışma yalnızca iki topluluğun "referans geçişi" ifadesini farklı anlamlarda kullanmasından çıkıyor.

Ders

Değer geçişi, kesin olarak

Metot **değişkenin bir kopyasını** alır. Değişken bir referans tuttuğunda metot referansın kopyasını alır, dolayısıyla ikisi de aynı nesneyi gösterir.

`PassByValue.java` aynı görünen ama olmayan iki durumu gösteriyor. Gerçek çıktı:

```
primitive after the call: 10
list after mutation:      [original, added inside the method]
list after reassignment:  [original]
string after the call:    before
```

Aynı tip, aynı çağrı biçimi, zıt sonuçlar:

```
void mutateTheObject(List<String> items) {
    items.add("added inside the method"); // changes the shared object
}

void reassignTheParameter(List<String> items) {
    items = new ArrayList<>();           // rebinds the local copy only
}
```

Birincide metod, referansının kopyasını takip edip tek paylaşılan nesneye ulaştı ve onu değiştirdi. İkincide kendi kopyasını başka bir yere yöneltti. Sizin değişkeniniz hiç kıpırdamadı, çünkü metodun onu kıpırdatmanın bir yolu yok.

Gerçekten kullanacağınız üç sonuç:

- Bir metodun verinizi değiştirebilmesini istiyorsanız değiştirilebilir nesne geçirin.
- İstemiyorsanız değişmez bir şey, `List.copyOf` ya da savunmacı kopya geçirin.
- Bir metod değişkeninizi asla yeniden yönltemez. Size farklı bir nesne vermesi gerekiyorsa onu **döndürmek** zorundadır.

String'ler bu tartışmayı karıştırır çünkü değişmezdirler, dolayısıyla onlar için değiştirme durumu hiç yoktur. Her String metodu yeni bir nesne döndürür.

Aşırı yükleme çözümlemesi

Hiçbir aşırı yükleme tam eşleşmediğinde derleyici şu sabit sırayı dener:

```
1. genişletme  int -> long -> float -> double
2. kutulama    int -> Integer
3. varargs     int -> int...
```

İlk işe yarayanı alır ve **derleme** zamanında karar verir. `Overloading.java` dosyasından:

```
pick(1)    -> long        genişletme kutulamayı yendi
choose(1)  -> Integer     kutulama varargs'ı yendi
```

Bu sıra geriye dönük uyumluluk için var. Genişletme ve varargs otomatik kutulamadan eskidir, dolayısıyla kutulamayı tercih etmek Java 5 öncesi yazılmış kodun anlamını değiştirdi.

Bunun yol açtığı hata

`List` hem `remove(int index)` hem `remove(Object o)` metoduna sahip:

```
after remove(1): [10, 30]
after remove(Integer.valueOf(30)): [10]
```

Birincisi **indeks 1'deki** elemanı sildi, ki o **20** idi. **1** değerini değil.

Bu gerçek üretim hatalarına yol açmıştır. Bir `List` içinde `Integer` tutulduğunda hangi aşırı yüklemeyi kastettiğinizi her zaman açıkça belirtin.

Dönüş tipi imza değildir

```
int parse(String s) { ... }
double parse(String s) { ... }
```

```
error: method parse(String) is already defined in class ReturnIsNotASignature
```

Derleyici aşırı yüklemeyi çağrı yerindeki argümanlardan seçer. Tek başına bir ifade olarak `parse("1");` yazıldığında seçim yapacağı bir şey yoktur. Standart kütüphanenin `parse` adında iki metot yerine `parseInt` ve `parseDouble` içermesinin sebebi tam olarak budur.

Varargs

Metodun içinde varargs parametresi yalnızca bir dizidir. `Varargs.java` :

```
int count(int... numbers) { return numbers.length; }
```

Üç kural:

1. Son parametre olmalı ve yalnızca bir tane olabilir.
2. Argümansız çağırmak **boş dizi** verir, asla `null`. Sıradan çağrılar için null kontrolü gerekmez.
3. Açıkça verilen `null` belirsizdir ve dönüşüm ister:

```
describe((Object[]) null) -> the array itself was null
describe((Object) null)   -> an array of length 1
```

`String.format` ve `printf` varargs metotlarıdır, herhangi sayıda değer almalarının sebebi bu.

static

`static` sınıfa aittir. Diğer her şey bir nesneye aittir.

Baştan beri fark etmeden örnek metotları yazıyorsunuz: kompakt kaynak dosyasının `main` metodu bir tanesi, içinde `this` çalışmasının sebebi bu.

`static` kullanın: metodun belirli bir nesneden hiçbir şeye ihtiyacı yoksa, `Math.max` gibi argümanlarının saf bir fonksiyonuysa, ya da `List.of` gibi bir fabrikaysa.

Kaçının: metot nesne başına duruma dokunuyorsa, ya da sonradan geçersiz kılmak isteyebilirsiniz.

Statik metotlar çok biçimli değildir ve Modül 06 bunun bedelini anlatıyor.

Faydalı bir test: bir metodu statik yapmak, bir nesnenin zaten bileceği birkaç şeyi parametre olarak geçirmeyi gerektiriyorsa, o metot statik olmamalıdır.

Statik *durum* tüm program için tek paylaşılan kopyadır. Kullanışlı, ve çok iş parçacıklı bir programda patlamayı bekleyen bir yarış koşulu. Modül 18 buna geri dönüyor.

Çalıştırın

```
java modules/04-methods/examples/PassByValue.java
java modules/04-methods/examples/Overloading.java
java modules/04-methods/examples/Varargs.java
java modules/04-methods/examples/StaticVsInstance.java

# Bilerek başarısız olur.
java modules/04-methods/examples/ReturnTypeIsNotASignature.java

./scripts/verify-examples.sh modules/04-methods
```

Sık Yapılan Hatalar

Bir metodun değişkeninizi yeniden yönelteceğini beklemek. Yapamaz. Yeni değeri döndürmesini sağlayın.

`List<Integer>` üzerinde `list.remove(someInt)` çağırmak. Değeri değil konumu sildiniz. Sarın: `list.remove(Integer.valueOf(x))`.

Getter'dan iç koleksiyonunuzu vermek. Çağırın artık nesnenizin durumuna bir referans tutuyor ve arkanızdan değiştirebilir. Önemli olduğunda `List.copyOf(items)` döndürün.

Yalnızca dönüş tipiyle aşırı yüklemeye çalışmak. Metotlara farklı ad verin.

Varsayılan parametre değeri beklemek. Java'da yok. Daha dolu sürümü çağıran bir aşırı yükleme yazın:

```
String greet(String name) { return greet(name, "Hello"); }
String greet(String name, String word) { return word + ", " + name; }
```

"Şu an bir nesneye ihtiyacı yok" diye static yapmak. Bir metodu statik yapmak kolay, çağırınlar ona bağlandıktan sonra geri almak zahmetlidir, özellikle sonradan geçersiz kılmak ya da taklit etmek istediğinizde.

Özet Çıkarımlar

- **Java her zaman değer geçişi yapar.** Kopyalanan değişkendir, ve nesnelerde değişken bir referanstır.
- **Parametrenin nesnesini değiştirmek çağırana görünür. Parametreyi yeniden atamak görünmez.** Bu tek ayırım tüm tartışmayı çözer.
- **Aşırı yüklemeler genişletme, sonra kutulama, sonra varargs sırasıyla çözümlenir,** derleme zamanında.
- `List.remove(int)` indeks alır, `remove(Object)` değer alır. Liste `Integer` tuttuğunda açık olun.
- **Dönüş tipi imzanın parçası değildir.** Farklı dönüş değil, farklı ad.
- **Varargs bir dizidir,** en sonda olmalıdır ve null değil boşdur.
- `static` nesne yok demektir, dolayısıyla `this` yok ve geçersiz kılma yok.

Ödev

[homework/README.md](#)

Kendisine verileni değiştiremeyen bir metot yazın, kanıtlayın, sonra `List.remove` tuzağına bilerek düşün. Referans çözüm [solutions/04-methods/](#) altında.

Ödev 04: Bir Metot Neyi Değiştirebilir, Neyi Değiştiremez

İki program. Birincisi değer geçişi kuralını kendinize güvenle değil kanıtla göstermenizi istiyor. İkincisi sizi bilerek gerçek bir üretim hatasına sokuyor.

Birinci Bölüm: Envanter

`Inventory.java` oluşturun. Değiştirilebilir bir listeye başlayın:

```
var stock = new ArrayList<String>(List.of("bolt", "nut"));
```

Üç metot yazın ve hepsini çağırıp her birinden sonra listeyi yazdırın:

- `addItem(List<String> items, String item)` — bir ürün ekler. Bu çağırana **görünmek zorunda**.
- `replaceAll(List<String> items, List<String> replacements)` — parametreye yeni bir liste atayacak şekilde yazılmış. Bu çağırana **görünmemeli**, ve yazdığınız çıktı görünmediğini göstermeli.
- `replaceAllProperly(...)` — düzeltilmiş sürüm.

Sonra şunu cevaplayan bir yorum yazın: `replaceAll` metodunu düzeltmenin tam olarak iki yolu var. İkisini de isimlendirin ve her birini ne zaman seçeceğinizi söyleyin.

İkinci Bölüm: Remove Tuzağı

`RemoveTrap.java` oluşturun. Bir `List<Integer>` kurun ve her iki aşırı yüklemeyi tetikleyin:

```
var ids = new ArrayList<Integer>(List.of(100, 200, 2, 300));
int idToDrop = 2;
ids.remove(idToDrop);
```

Sonucu yazdırın. Kodun söylüyor görüldüğü şey olmayacak.

Sonra **kendi kelimelerinizle**, derleyicinin neden o aşırı yüklemeyi seçtiğini açıklayan bir yorum yazın. Açıklamanız kutulamanın çözümleme sırasında nerede durduğundan bahsetmeli, çünkü asıl sebep o.

Kodun görüldüğü işi yapmasını sağlayan en az iki yolu göstererek bitirin.

Kabul Kriterleri

- ☐ Her iki dosya da `java Inventory.java` ve `java RemoveTrap.java` ile çalışıyor

- ☐ `Inventory.java` çıktısı `addItem` metodunun çağırana göründüğünü ve `replaceAll` metodunun görünmediğini kanıtlıyor
- ☐ Bir yorum `replaceAll` için iki düzeltmeyi isimlendiriyor ve hangisinin ne zaman geçerli olduğunu söylüyor
- ☐ `RemoveTrap.java` aynı başlangıç listesinden `remove(int)` ve `remove(Object)` çağrılarının farklı sonuç verdiğini gösteriyor
- ☐ Açıklama aşırı yükleme çözümleme sırasından kendi kelimelerinizle bahsediyor
- ☐ En az iki doğru alternatif gösteriliyor

İleri Seviye

`List<String>` alanı tutan ve `getter`'ı alanı doğrudan döndüren küçük bir sınıf ekleyin. `main` içinden listeyi alın, değiştirin ve nesnenin içeriğini yazdırarak iç durumunu izinsiz olarak dışarıdan değiştirdiğinizi gösterin.

Sonra `getter`'ı düzeltin ve düzeltmenin bedelini söyleyin. Birden fazla düzeltme var ve bedelleri aynı değil.

İpucu, birinci bölüm zaten çalışıyor görünüyorsa

`replaceAll` çağırana ait listeyi değiştiriyor görünüyorsa, parametreye atama mı yaptığınızı yoksa üzerinde bir metot mu çağırdığınızı kontrol edin. Bu ikisinden yalnızca biri dışarıdan görünür, ve alıştırmanın tamamı hangisini yazdığınıza bağlı.

Modül 05: Sınıflar ve Nesneler

Bu mezuniyet günü. Şimdiye kadarki her dosya kompakt kaynak dosyasıydı. Buradan itibaren örnekler klasik biçimde yazılıyor, çünkü her gerçek kod tabanı, her öğretici ve şimdiye kadar yazılmış her Stack Overflow cevabı onu kullanıyor.

Dilde yeni olan hiçbir şey yok. Modül 01, derleyicinin sınıfı baştan beri sizin için ürettiğini `javap` ile gösterdi. Şimdi onu kendiniz yazıyorsunuz.

Neler Öğreneceksiniz

- `public static void main(String[] args)` içindeki her kelime ve her birinin nedeni
- Alanlar, dört görünürlük seviyesi ve neden `private` ile başladığınız
- Kurucular, `this(...)` ve bir tane yazınca kaybettiğiniz bedava kurucu
- Statik fabrika metotlarının kuruculardan neden sıklıkla üstün olduğu
- `super()` öncesinde doğrulama yapmanızı sağlayan esnek kurucu gövdeleri

Başka Bir Dilden Geliyorsanız

	Python	Java
Kurucu	<code>__init__</code>	sınıfla aynı adlı metot
Mevcut örnek	açık <code>self</code> parametresi	örtük <code>this</code>
Gizlilik	gelenek olarak <code>_ad</code>	derleyicinin uyguladığı <code>private</code>
Birden fazla kurucu	tek <code>__init__</code> , varsayılan argümanlar	aşırı yükleme ya da statik fabrikalar
Sınıf düzeyinde veri	sınıf niteliği	<code>static</code> alan
Dosya düzeni	dosya başına serbestçe birçok sınıf	dosya başına bir <code>public</code> sınıf, adı eşleşmeli

Asıl fark gizlilik satırı. Python'un alt çizgisi bir ricadır. Java'nın `private` niteleyicisi derleyicinin uyguladığı bir kuraldır ve onu ihlal eden kod derlenmez.

Ders

Klasik biçim, kelime kelime

```
public class TheClassicForm {  
    public static void main(String[] args) {  
        System.out.println("Running the classic way.");  
    }  
}
```

Kelime	Nedeni
public (sınıf)	her yerden kullanılabilir. Dosya adıyla eşleşmeli.
public (main)	JVM'in onu çağırabilmesi gerekiyor
static	asıl önemli olan. Başlangıçta henüz hiçbir nesne yok, dolayısıyla main bir nesneye ait olamaz.
void	hiçbir şey döndürmez; çıkış kodları System.exit ile gelir
main	JVM'in aradığı tam ad
String[] args	komut satırı argümanları. Asla null değil; yoksa boş.

main içinde this yoktur, çünkü statiktir. Örnek durumunu kullanmak için main önce bir nesne kurar, ki gerçek main metodlarının çoğu bunu yapar.

Hangi biçimi kullanmalısınız? Öğrenmek, script'ler ve tek dosyalık araçlar için kompakt biçim daha az gürültülü. Çok dosyalı herhangi bir şey, bir kütüphane ya da başkalarının dokunduğu kod için klasik biçimi kullanın. Aynı dildir; ne kadar yazacağınızı seçiyorsunuz.

Kapsülleme

Dört görünürlük seviyesi, en dardan başlayarak:

Seviye	Erişim
private	yalnızca bu sınıf
(hiçbiri)	bu paket. Paket-özel denir.
protected	bu paket, artı her yerdeki alt sınıflar
public	herkes

private ile başlayın ve sebebiniz olduğunda genişletin. Sonradan genişletmek kolaydır. Sonradan daraltmak ona bağımlı olan herkesi kırar.

Amaç gizlilik değil, **değişmezlerdir**. [Encapsulation.java](#) dosyasından:

```
refused: insufficient funds: have 15000, asked for 100000  
balance is still: 15000
```

`balanceInCents` alanı `private` olduğu için her değişiklik kuralları uygulayan bir metottan geçer, böylece sınıf bakiyenin asla negatif olmayacağına söz verebilir ve sözünü gerçekten tutabilir. `Public` değiştirilebilir alanları olan bir sınıfın hiçbir değişmezi yoktur, çünkü herhangi bir şey onları her an değiştirebilir.

Alanda `final` tam olarak bir kez, kurucuda atanır demektir. Dikkat: bu değişmez olmakla aynı şey değildir, `final` bir alan hâlâ değiştirilebilir bir nesneyi gösterebilir.

Kurucular

Herhangi bir kurucu yazmak bedava olanı kaldırır. Java argümansız kurucuyu yalnızca hiç kurucu bildirmediğinizde sağlar. Mevcut bir sınıfa kurucu eklemenin `new Thing()` yazan çağırıları kırabilmesinin sebebi budur.

`this(...)` başka bir kurucuya devreder:

```
Temperature() {  
    this(0.0);  
}
```

Statik fabrikalar kurucuları sıklıkla yener çünkü adları vardır:

```
Temperature.fromFahrenheit(77.0)  
Temperature.fromKelvin(298.15)
```

Tek bir `double` alan iki kurucu birlikte var olamaz. İki fabrika olabilir. Bir fabrika ayrıca önbelleklenmiş bir örnek ya da bir alt sınıf döndürebilir, kurucu ise her zaman yeni bir şey kurar.

Alanlar varsayılan alır, yereller almaz. Bir alan otomatik olarak `0`, `false`, `null` ya da `0.0` olur. Atanmadan kullanılan yerel değişken derleme hatasıdır. Bu asimetri bilinçlidir: bir alan meşru olarak başka bir metot tarafından sonradan atanabilir, dolayısıyla derleyici hiçbir şey kanıtlayamaz; yerel değişken ise bildirildiği yerden birkaç satır ötede kullanılır, dolayısıyla derleyici denetleyebilir ve denetler.

Esnek kurucu gövdeleri

Eski kural: `super(...)` ilk ifade olmak zorundaydı. Üst nesneyi kurar, sonra argümanların geçersiz olduğunu keşfederdiniz.

JDK 25'te kesinleşen JEP 513, önce ifade çalıştırmaya izin veriyor:

```
Positive(int value) {  
    if (value <= 0) {  
        throw new IllegalArgumentException("must be positive, got " + value);  
    }  
    super(value);  
}
```

`FlexibleConstructorBodies.java` çıktısı:

```
Measurement constructor ran with 5
built: 5

now with an invalid value:
rejected: must be positive, got -1
```

Dikkatle okuyun. Üst sınıf kurucusu geçerli durum için **bir kez** çalıştı. Geçersiz olan için hiç çalışmadı, çünkü denetim önce fırlattı. Eski kural altında bu, denetimi `super()` çağrısının içindeki statik bir yardımcıya gizlemeden imkânsızdı.

`super()` öncesinde `this` hâlâ yasaktır, çünkü üst sınıf çalışmamıştır ve nesne henüz geçerli değildir. Derleyici bunu uygular.

Çalıştırın

```
java modules/05-classes-and-objects/examples/TheClassicForm.java
java modules/05-classes-and-objects/examples/TheClassicForm.java hello world
java modules/05-classes-and-objects/examples/Encapsulation.java
java modules/05-classes-and-objects/examples/Constructors.java
java modules/05-classes-and-objects/examples/FlexibleConstructorBodies.java

./scripts/verify-examples.sh modules/05-classes-and-objects
```

Sık Yapılan Hatalar

Public değiştirilebilir alanlar. Sınıf artık kendi durumu hakkında hiçbir söz veremez.

Kurucu ekleyip başka yerdeki `new Thing()` çağrılarını kırmak. Herhangi bir kurucu bildirdiğiniz anda bedava argümansız kurucu kaybolur.

Getter'dan iç koleksiyonunuzu döndürmek. Çağırان artık nesnenizin durumunu sizden geçmeden değiştirebilir. Modül 04'ün ödevi çözümü kapsıyordu.

`final` kelimesinin değişmez demek olduğunu sanmak.

```
private final List<String> items = new ArrayList<>();
items.add("still allowed"); // fine, the reference never changed
```

`final` referansı sabitler, nesneyi değil.

Gerçek iş yapan bir kurucu. Dosya açan ya da ağ çağrısı yapan kurucuların testi zordur ve çağırının başa çıkamayacağı biçimlerde başarısız olurlar. Nesneyi kurun, sonra bir metod çağırın.

Her alan için refleksle getter ve setter yazmak. Her alan için setter, fazladan adımlı bir public değiştirilebilir alandır. Gerçekten bir şeyin o değeri dışarıdan değiştirmesi gerektiğinde ekleyin.

Özet Çıkarımlar

- `main` üzerindeki `static` , başlangıçta hiçbir nesne olmadığı için vardır.
- **Klasik ve kompakt biçim aynı dildir**, yalnızca yazdığınız şey farklıdır. Çok dosyalı ya da paylaşılan her şey için klasik olanı kullanın.
- **Alanlara `private` ile başlayın.** Genişletmek kolay, daraltmak çağırırları kırar.
- **Kapsülleme size gizlilik değil değişmez satın alır.**
- **Herhangi bir kurucu bildirmek bedava argümansız olanı kaldırır.**
- **Statik fabrikaların adı vardır**, dolayısıyla kuruculardan farklı olarak aşırı yüklenebilirler.
- **Alanlar varsayılan alır, yereller almaz.**
- **İfadeler artık `super()` öncesinde gelebilir**, böylece kötü argümanları üst nesne hiç kurulmadan reddedebilirsiniz.

Ödev

[homework/README.md](#)

Geçersiz duruma sokulamayan bir sınıf kurun, sonra deneyerek kanıtlayın. Referans çözüm

[solutions/05-classes-and-objects/](#) altında.

Ödev 05: Bozulamayan Bir Nesne

Kuralları, çağıranlardan dikkatli olmalarını rica eden bir yorumla değil, derleyici ve kurucu tarafından uygulanan bir sınıf kurun. Sonra onu kırmayı deneyin ve kıramadığınızı gösterin.

Görev

`Booking.java` dosyasını klasik biçimde oluşturun: bir `public class Booking` ve içinde `public static void main(String[] args)`.

İçinde şunları tutan bir `Reservation` sınıfı yazın:

- yolcu adı
- koltuk sırası numarası
- koltuk etiketlerinden oluşan bir liste, örneğin `["A", "B"]`

Garanti Etmesi Gereken Kurallar

Bir `Reservation` asla şu kurallardan birini bozan bir durumda var olamaz:

1. Yolcu adı asla null ve asla boş değildir
2. Sıra numarası her zaman 1 veya daha büyüktür
3. Her zaman en az bir koltuk vardır
4. Koltuk listesi nesnenin dışındaki hiç kimse tarafından değiştirilemez, **listeyi içeri veren dahil**

Dördüncü kural ilginç olanı ve iki yarısı var. Kurucunuzu çağırdıktan sonra çağıranın elinde hâlâ ne kaldığını düşünün.

Kanıtlayın

`main` içinde şunların her birini deneyin ve ne olduğunu yazdırın:

- null yolcu
- boş yolcu adı
- sıra `0`
- boş koltuk listesi
- içeri verdiğiniz listeye bir referans saklayıp sonradan ona ekleme yapmak
- getter'ınızın geri verdiği listede `add` çağırarak

Altı denemenin hepsi başarısız olmalı ya da hiçbir etki yaratmamalı. Çıktınız, nesnenin her seferinde kendini savunduğunu açıkça göstermeli.

Ayrıca İçermeli

- En az bir statik fabrika metodu, artı bunun kurucunun vermediği neyi verdiğini söyleyen bir yorum
- Rezervasyon yazdırmayı okunaklı kılan bir `toString()` geçersiz kılması

Kabul Kriterleri

- ☐ `java Booking.java` ile çalışıyor
- ☐ `public static void main(String[] args)` içeren `public class Booking`
- ☐ `Reservation` içindeki her alan `private` ve `final`
- ☐ Dört kuralın hepsi, herhangi bir alan atanmadan önce kurucuda uygulanıyor
- ☐ Kurulumdan sonra çağırının özgün listesini değiştirmek rezervasyonu değiştirmiyor
- ☐ Getter'ın döndürdüğü listeyi değiştirmek ya başarısız oluyor ya da rezervasyonu değiştirmiyor
- ☐ Bir statik fabrika var ve gerekçesini açıklayan bir yorum taşıyor
- ☐ `toString()` geçersiz kılınmış

İleri Seviye

Koltuk listesini değişmez biri yerine düz bir `ArrayList` alanı yapın ve dördüncü kuralı yine de çalıştırın. Sonra iki yaklaşımı karşılaştıran bir yorum yazın: her biri neye mal oluyor ve bu bedel hangi tarafa düşüyor, kurucuya mı yoksa getter'a yapılan her çağrıya mı?

İpucu, dördüncü kural elinizden kaçıyorsa

İki ayrı sızıntı var ve birini kapatmak diğerini kapatmıyor.

Birincisi: `new Reservation(..., seats)` çağrısından sonra o listeye tam olarak kim hâlâ bir referans tutuyor?

İkincisi: `reservation.seats()` çağrısından sonra az önce dışarı ne verdiniz?

`List.copyOf` ikisine birden çözüm, ve nedenini anlamak alıştırmamanın amacı.

Modül 06: Kalıtım ve Çok Biçimlilik

Kalıtım, Java'nın 1995'te üzerinden pazarlandığı özellik ve modern Java'nın ölçülü kullanmanızı tavsiye ettiği özellik. İkisi de doğru ve bu modül nedenini anlatıyor.

Her gün kullanacağınız kısım aslında `extends` değil. `equals` / `hashCode` sözleşmesi, ve bu modül onu anlatmak yerine kıırarak gösteriyor.

Neler Öğreneceksiniz

- Dinamik gönderim ve kalıtımın asıl amacının bu olduğu
- `super` ve `@Override` yazmanın her seferinde neden değdiği
- Tasarımınız hakkında zıt talimatlar olarak `abstract` ve `final`
- `equals` / `hashCode` sözleşmesi, bir `HashSet` kıırarak gösterilmiş
- Çok biçimli *olmayanlar*: alanlar ve statik metotlar

Başka Bir Dilden Geliyorsanız

	Python	Java
Kalıtım	<code>class Dog(Animal)</code>	<code>class Dog extends Animal</code>
Üst sınıfı çağırma	<code>super().__init__()</code>	<code>super(...)</code> , ve önce gelmeli
Çoklu kalıtım	<code>var</code>	yok, yalnızca tek. Arayüzler boşluğu dolduruyor.
Soyut	<code>abc.ABC</code>	<code>abstract</code> anahtar kelimesi
Alt sınıflamayı engelleme	pek mümkün değil	<code>final</code>
Eşitlik	<code>__eq__</code> ve <code>__hash__</code>	<code>equals</code> ve <code>hashCode</code> , aynı ikili

Java yalnızca bir üst sınıfa izin verir. Arayüzlerin var olma sebebi bu kısıt ve Modül 07 onları anlatıyor.

Ders

Dinamik gönderim

Tek bir çağrı yeri, üç metot. `Inheritance.java` dosyasından:

```
[log] system reboot
[email to ada@example.com] system reboot
[sms to +44 7700 900000] system reboot
```

Döngü `Notification` tipine göre yazıldı. Derleyici yalnızca o tipin bir `deliver()` metodu *olduğunu* denetler. Hangisinin çalışacağına gerçek nesneden, çalışma zamanında karar verilir.

Pratik kazanç: o döngüye hiç dokunmadan dördüncü bir bildirim tipi ekleyebilirsiniz. Temel tipe göre yazılmış kod çalışmaya devam eder.

super ve @Override

`super(...)` üst sınıf kurucusunu çağırır ve Modül 05'in anlattığı istisna dışında önce gelmelidir.

`super.method()` üst sınıfın sürümünü çağırır, davranışı değiştirmek yerine genişletmenin yolu budur.

@Override her zaman yazın. Süs değil, derleyiciye gerçekten bir şeyi geçersiz kıldığınızı denetlemesi için verilen bir talimattır. `OverrideTypo.java` `deliver` yerine `delivar` yazıyor:

```
error: delivar() in Email does not override or implement a method from a supertype
@Override
^
```

Anotasyon olmadan bu kusursuz derlenir, kimsenin çağırmadığı bir metot ekler ve temel uygulamayı çalışmaya devam ettirir. Kod doğru görüldüğü için hata incelemekten sağ çıkar. Tek bir anotasyon tüm kategoriyi ortadan kaldırıyor.

abstract ve final

`abstract` alt sınıflar bunu doldurmalı der. `final` kimse bunu değiştiremez der.

`AbstractAndFinal.java` dosyasından:

```
abstract class Shape {
    abstract double area();

    final boolean isLargerThan(Shape other) {
        return this.area() > other.area();
    }
}
```

Soyut bir sınıf kurulamaz, ki amaç budur: belirli bir şekli olmayan bir şeklin anlamlı bir alanı yoktur. Arayüzden farklı olarak sıradan metotlar ve durum da tutabilir.

Metotta `final` geçersiz kılmayı engeller. Sınıfta `final` genişletmeyi tamamen engeller. `String` ve `Integer` ikisi de final, ve bu yardımseverlikten uzaklık değil: `String` sınıfını alt sınıflayıp `equals` metodunu geçersiz kılabilseydi herhangi biri, hiçbir yerde hiçbir kod bir `String` değerine güvenemezdi.

Sınıfları doğal hissettiğinden daha sık final yapın. Genişletilmek üzere tasarlanmamış bir sınıf genellikle güvenle genişletilemez, çünkü alt sınıf herhangi bir metodu geçersiz kılıp sınıfın dayandığı değişmezleri bozabilir. Modül 09, hepsi ya da hiçbiri arasındaki orta yol olan `sealed` kavramını tanıtır.

equals/hashCode sözleşmesi

Asıl karşılaşacağınız kısım bu. `EqualsHashCodeContract.java` dosyasından, `equals` metodunu geçersiz kılıp `hashCode` metodunu unutan bir sınıf:

```
two equal points added, set size: 2
contains an equal point:           false
but the two ARE equal:             true
```

Tekrar okuyun. Küme, eşit olduğunu kabul ettiği iki öğeyi tutuyor ve içinde zaten bulunan bir öğeye eşit olanı bulamıyor.

`HashSet` her şeyin üzerinde `equals` çağırır. Bir kova seçmek için `hashCode` hesaplar ve yalnızca o kovanın içinde karşılaştırır. `Object.hashCode` kimliğe dayalıdır, dolayısıyla iki ayrı örnek `equals` ne derse desin farklı kod alır, farklı kovalara düşer ve hiç karşılaşmaz.

`hashCode` ekleyin, düzgün davranır:

```
two equal points added, set size: 1
contains an equal point:           true
```

Sözleşmenin tamamı:

1. `a.equals(b)` ise `a.hashCode() == b.hashCode()` olmalı. İşleri bozan bu.
2. Eşit hash kodları eşitlik anlamına gelmez. Çakışmalar meşrudur.
3. Yansılmalı: `a.equals(a)`
4. Simetrik: `a.equals(b) == b.equals(a)`
5. Geçişli: `a=b` ve `b=c` ise `a=c`
6. Tutarlı: hiçbir şey değişmediyse her seferinde aynı cevap
7. `a.equals(null)` her zaman false

`Objects.hash(...)` kullanın ve `equals` metodunun karşılaştırdığı alanların **tam olarak** aynısını geçirin. Bir alanı birinde kullanıp diğerinde kullanmamak, birinci kuralı `hashCode` metodunu tamamen atlamak kadar bozar.

`toString` ailenin üçüncü üyesi. Sözleşmenin parçası değil, ama okunaklı bir log satırı ile `BadPoint@4034c28c` arasındaki fark.

Kısayol: bir record üçünü de bileşenlerinden doğru biçimde yazar. Modül 08 onları anlatıyor, ve veri taşıyan bir sınıf için record neredeyse her zaman daha iyi cevap.

Çok biçimli olmayanlar

Metotlar nesneye göre gönderilir. Alanlar ve statikler gönderilmez. `WhatIsNotPolymorphic.java` dosyasında nesne bir `Child`, ama `Parent` değişkeninde tutuluyor:

```
instance method: Child.instanceName
field access:    parent field
as a Child:      child field
static method:   Parent.staticName
```

Her iki alan da nesnede aynı anda var. Alt sınıf üst sınıfın alanını değiştirmedir, onu gizleyen ikinci bir alan ekledi. Statik metotlar da benzer şekilde *gizlenir*, geçersiz kılınmaz.

Buradan çıkan kural: **asla bir alanı gölgelemeyin, asla üst sınıfın statik metodunu yeniden bildirmeyin.** İkisi de yasaldir, ikisi de görüldüğü işi yapmaz ve ikisini de yazmak için iyi bir sebep yoktur.

Modül 04'ün `static` kelimesinin sizi sonradan engellediği uyarısının sebebi de bu. Statik bir metot geçersiz kılınmaz, dolayısıyla buradaki hiçbir şeye katılamaz.

Çalıştırın

```
java modules/06-inheritance-and-polymorphism/examples/Inheritance.java
java modules/06-inheritance-and-polymorphism/examples/AbstractAndFinal.java
java modules/06-inheritance-and-polymorphism/examples/EqualsHashCodeContract.java
java modules/06-inheritance-and-polymorphism/examples/WhatIsNotPolymorphic.java

# Bilerek başarısız olur.
java modules/06-inheritance-and-polymorphism/examples/OverrideTypo.java

./scripts/verify-examples.sh modules/06-inheritance-and-polymorphism
```

Sık Yapılan Hatalar

`equals` metodunu `hashCode` olmadan geçersiz kılmak. Nesneleriniz bir `HashMap` içine girer ve bir daha çıkmaz. Bu modüldeki en yaygın ciddi hata.

`equals` metodunun yok saydığı bir alanı `hashCode` içine koymak ya da tersi. Aynı bozukluk, fark edilmesi daha zor.

`equals` ve `hashCode` içinde değiştirilebilir bir alan kullanmak. Nesneyi bir kümeye koyun, alanı değiştirin, artık kendi koleksiyonunun içinde kaybolmuştur, çünkü yeni hash kodunun eşleşmediği bir kovada dosyalanmıştır.

`@Override` yazmamak. Bedava hata tespitini reddetmek.

Kurucudan geçersiz kılınabilir bir metot çağırmak. Alt sınıfın geçersiz kılma metodu, alt sınıf kurucusu alanlarını ilklendirmeden önce çalışır, dolayısıyla null ve sıfır görür. Böyle metotları `final` ya da `private` yapın.

Kod tekrar kullanmak için kalıtıma uzanmak. `extends` "bir çeşididir" demektir, "ondan ödünç alır" değil. Alt sınıf dürüstçe üst sınıfın kullanıldığı her yerde kullanılamıyorsa kompozisyon kullanın: diğer nesneyi bir alan olarak tutun.

Özet Çıkarımlar

- **Dinamik gönderim metodu nesneden seçer**, değişkenin tipinden değil.
- `@Override` her zaman yazın. Sessiz hataları derleme hatalarına çevirir.
- `abstract` alt sınıfları doldurmaya zorlar. `final` değişikliği yasaklar. Varsayılan olarak `final` tercih edin.
- `equals` ve `hashCode` metotlarını her zaman birlikte, aynı alanlar üzerinde geçersiz kılın.
- **Asla alanları gölgelemeyin ya da statik metotları yeniden bildirmeyin.** Nesneye göre değil bildirilen tipe göre çözümlenirler.
- **"Bir çeşididir" ilişkisi dürüst değilse kompozisyonu tercih edin.**

Ödev

[homework/README.md](#)

Bir `HashSet` kümesini bilerek kırın, teşhis edin, sonra düzeltin. Referans çözüm [solutions/06-inheritance-and-polymorphism/](#) altında.

Ödev 06: Bir Şeyleri Kaybeden Küme

Birinci bölüm, bilerek sebep olduğunuz bir hata avı. İkinci bölüm, gönderimi dürüstçe kullanan küçük bir hiyerarşi kuruyor.

Birinci Bölüm: Bir HashSet Kırın

`Library.java` dosyasını klasik biçimde oluşturun.

Tek bir `String code` tutan `BrokenIsbn` sınıfı yazın. `equals` metodunu, aynı koda sahip iki `BrokenIsbn` nesnesi eşit olacak şekilde geçersiz kılın. `hashCode` metodunu geçersiz kılmayın.

Sonra `main` içinde:

1. Aynı ISBN değerini bir `HashSet` içine iki kez ekleyin
2. Kümenin boyutunu yazdırın
3. Kümenin eşit bir ISBN `contains` edip etmediğini yazdırın
4. İki nesnenin birbirine `equals` olup olmadığını yazdırın

Çıktı kendisiyle çelişecek. Küme, eşit olduğunu kabul ettiği iki öge tutacak ve zaten içerdiği bir öğeyi bulamayacak.

Teşhis Edin

Bunun neden olduğunu **kendi kelimelerinizle** açıklayan bir yorum yazın. Doğru bir cevap kovalardan bahsetmek zorunda. "Çünkü hashCode eksik" demek belirtiyi tarif eder, mekanizmayı değil.

Sonra düzeltilmiş sürüm olan `Isbn` sınıfını yazın ve aynı dört satırın düzgün davrandığını gösterin.

Mutasyon Tuzağı

Hem `equals` hem `hashCode` tarafından kullanılan **final olmayan** bir alana sahip bir sınıf daha yazın. Sonra:

1. Bir örnek oluşturup `HashSet` içine koyun
2. `contains(theObject)` yazdırın
3. Alanı değiştirin
4. `contains(theObject)` ve kümenin boyutunu tekrar yazdırın

Nesne, onu hâlâ sayan bir kümenin içinde kaybolacak. Nedenini bir yorumda açıklayın ve bunun `equals` içinde kullanılan alanlar hakkında ne ima ettiğini söyleyin.

İkinci Bölüm: Karışık Raf

Başlığı olan soyut bir `LibraryItem` ve üç alt sınıf yazın: `Book`, `Audiobook` ve `Magazine`. Her biri boyutunu farklı ölçer. Kitabın sayfası, sesli kitabın süresi, derginin sayı numarası vardır.

Gereksinimler:

- `LibraryItem` alt sınıfların uygulamak zorunda olduğu bir `abstract` metot içersin
- `LibraryItem` miras aldıkları sıradan bir metot içersin
- `LibraryItem` neden final olduğunu söyleyen bir yorumla birlikte bir `final` metot içersin
- Bir alt sınıf, davranışı değiştirmek yerine genişletmek için `super.someMethod()` çağırınsın
- Üç alt sınıfın hepsi `final` olsun, bunun neden varsayılan olduğunu söyleyen bir yorumla

Sonra bir `LibraryItem` dizisi üzerinde döngü kurup üçünü de yazdırın. Tek döngü, üç davranış.

Kabul Kriterleri

- ☐ `java Library.java` ile çalışıyor
- ☐ `BrokenIsbn` çıktısı boyut `2`, `contains` `false` ve `equals` `true` gösteriyor
- ☐ Bir yorum kova mekanizmasını kendi kelimelerinizle açıklıyor
- ☐ `Isbn` çıktısı boyut `1` ve `contains` `true` gösteriyor
- ☐ Mutasyon tuzağı, kümenin boyutu değişmeden `contains` değerinin `true`'dan `false`'a döndüğünü gösteriyor
- ☐ `LibraryItem` bir soyut, bir sıradan ve bir `final` metot içeriyor
- ☐ Her `@Override` anotasyonlu
- ☐ Döngü `LibraryItem` tipine göre yazılmış, asla bir alt sınıfa göre değil

İleri Seviye

`Isbn` sınıfını bir record olarak yeniden yazın ve `equals`, `hashCode` ile `toString` metotlarını silin. Davranışın aynı olduğunu doğrulayın, sonra geçiş yaparak neyi bıraktığınızı anlatan bir yorum yazın. Bıraktığınız bir şey var ve Modül 08 onu isimlendirecek.

İpucu, birinci bölüm yanlış davranmıyorsa

İkinci ekleme ve `contains` çağırısı için **yeni** nesneler kurduğunuzdan emin olun. Aynı değişkeni tekrar kullanmak bir nesneyi kendisiyle karşılaştırır, bu da yalnızca kimlikle başarılı olur ve üretmeye çalıştığınız hatayı gizler.

Modül 07: Arayüzler

Bir sınıf tam olarak bir sınıfı genişletebilir ve istediği kadar arayüz uygulayabilir. Bu modüldeki her şey o asimetriden çıkar.

Modül, bu müfredattaki kalıtıma karşı en güçlü argümanla bitiyor, ve bu bir slogan değil. Standart kütüphaneden iki sınıfın, birebir aynı koda farklı cevap vermesi.

Neler Öğreneceksiniz

- Yetenek olarak arayüzler ve neden işi gören en dar olanı kabul etmeniz gerektiği
- Bir arayüzün soyut sınıfı ne zaman yendiği, ne zaman yenmediği
- Varsayılan metotlar ve icat edilme sebepleri olan o belirli sorun
- Statik ve özel arayüz metotları
- Elmas probleminin neden tahmin değil derleme hatası olduğu
- "Kompozisyonu tercih edin" sözünün sizi gerçekte neden koruduğu

Başka Bir Dilden Geliyorsanız

	Python	Java
Sözleşme	ördek tiplere ya da <code>Protocol</code>	derleme zamanında denetlenen <code>interface</code>
Çoklu ebeveyn	tam çoklu kalıtım	bir sınıf, çok arayüz
Mixin	çoklu kalıtımla	varsayılan metotlu arayüzlerle
Çakışma çözümü	MRO sessizce karar verir	derleme hatası, siz karar verirsiniz

Python belirsizliği metot çözümleme sırasıyla sizin için çözer. Java çözmeyi reddeder, çünkü iki eşit derecede makul davranış arasında sessiz bir seçim, kimsenin açıklayamayacağı hataların çıkış yoludur.

Ders

Arayüzler yetenektir

`Interfaces.java` dosyasından:

```
private static void launch(Flies thing) {  
    System.out.println("launching: " + thing.fly());  
}
```

O metot bir ördek, bir uçak ve sonradan birinin yazacağı her şey üzerinde çalışır. Nesnenin ne *olduğu* umurunda değil, yalnızca uçabildiği.

```
launching: duck flapping
launching: jet engines
```

Swims ve Speaks uygulayan ama Flies uygulamayan bir Penguin geçirilirse derlenmez. Yetenek varsayılmaz, denetlenir.

İşi gören en dar yeteneği kabul edin. Flies tipli bir parametre henüz var olmayan tipleri alabilir. Duck tipli olan alamaz.

Arayüz mü soyut sınıf mı?

	arayüz	soyut sınıf
Sınıf başına kaç tane	istediğiniz kadar	tam olarak bir
Örnek alanları	yok	var
Kurucular	yok	var
Durum	yok	var

Neyi modellediğinizi sorun. **"Yapabilir" bir arayüzdür:** Flies , Comparable , AutoCloseable . **"Bir çeşididir" bir soyut sınıftır**, ve yalnızca alt sınıflar gerçekten durum ve kurulum mantığı paylaşıyorsa.

İkisi de uyuyorsa arayüzü tercih edin. Uygulayıcının tek kalıtım yuvasını boş bırakır, ve o yuva kolayca harcanıp sonradan pişman olunan bir şeydir.

Varsayılan metotlar ve var olma sebepleri

DefaultMethods.java dosyasından:

```
interface Collection {
    List<String> tracks(); // must implement

    default int count() { return tracks().size(); } // free
    default boolean isEmpty() { return count() == 0; }
}
```

Buradaki tarih önemli. Java 8'den önce bir arayüze metot eklemek, her yerdeki mevcut her uygulamayı anında kırıyordu. Genel kütüphanelerdeki arayüzler fiilen sonsuza dek donmuştu.

Java 8'in java.util.Collection arayüzüne stream() eklemesi gerekiyordu. Varsayılan metotlar olmasaydı o tek ekleme, herhangi biri tarafından yazılmış her koleksiyon sınıfını kırardı.

Varsayılan metotlar kütüphane evrimi için vardır. Çoklu kalıtımı gizlice içeri sokmanın yolu değil, ve öyle kullanmak bir sonraki bölümü üretir.

İki küçük ekleme:

- **Özel arayüz metotları** (Java 9) birkaç varsayılanın, onu uygulayan herkese göstermeden yardımcı kod paylaşmasını sağlar.
- **Statik arayüz metotları** arayüzün kendisine aittir. Kütüphanede `List.of`, `Map.of` ve `Comparator.comparing` bulunmasının sebebi budur.

Elmas problemi

İki arayüz, aynı varsayılan, ikisini de uygulayan bir sınıf. `DiamondConflict.java`:

```
error: types Timestamped and Versioned are incompatible;
class Entry implements Timestamped, Versioned {
^
  class Entry inherits unrelated defaults for describe() from types Timestamped and
  Versioned
```

Java tam olarak bundan kaçınmak için yirmi yıl çoklu sınıf kalıtımını yasakladı. Varsayılan metotlar dar bir sürümünü geri getirdi, dolayısıyla dil bunu tek güvenli yolla ele alıyor: karar vermenizi sağlıyor.

```
@Override
public String describe() {
    return Timestamped.super.describe() + " / " + Versioned.super.describe();
}
```

`InterfaceName.super.method()` sözdizimi yalnızca bunun için var. Yalnızca bir arayüz varsayılan sağladığında ya da sınıf kendi uygulamasını verdiğinde çakışma olmaz.

Kompozisyonun sizi gerçekte neden koruduğu

Hatırlanmaya değer kısım bu. `CompositionOverInheritance.java` sayan bir sınıfı üç kez yazıyor. Gerçek çıktı:

```
--- inheritance, extending HashSet ---
added 3, counter says: 6    <- wrong

--- the SAME code, extending ArrayList ---
added 3, counter says: 3    <- right, by luck

--- composition ---
added 3, counter says: 3    <- right, by design
```

Her iki alt sınıftaki geçersiz kılma birebir aynı:

```
@Override public boolean add(String item)    { added++; return super.add(item); }
@Override public boolean addAll(Collection<? extends String> items) {
    added += items.size();
    return super.addAll(items);
}
```

`HashSet.addAll` her eleman için `this.add` çağırır, dolayısıyla geçersiz kılma tekrar çalışır ve sayı ikiye katlanır. `ArrayList.addAll` toplu kopyalar, dolayısıyla katlanmaz.

Aynı kütüphaneden iki sınıf, aynı kod, farklı cevaplar. Ve alt sınıf kendi kaynağına bakarak nedenini göremez.

İşte bu **kırılğan temel sınıf** problemi. Sınıfınızın doğruluğu, üst sınıfın hiç söz vermediği ve herhangi bir sürümde değiştirebileceği bir uygulama ayrıntısına bağlı. Kompozisyon bağımlılığı tamamen kaldırır: koleksiyonu tutun, koleksiyon olmayın.

Bedeli, kırk metodu miras almak yerine dışarı açmak istediklerinizi yazmanızdır. O bedel genellikle ödemeye değer, ve iyi bir soru sordurur: bu tipin o kırk metottan hangisini sunması gerekiyordu?

Çalıştırın

```
java modules/07-interfaces/examples/Interfaces.java
java modules/07-interfaces/examples/DefaultMethods.java
java modules/07-interfaces/examples/CompositionOverInheritance.java

# Bilerek başarısız olur.
java modules/07-interfaces/examples/DiamondConflict.java

./scripts/verify-examples.sh modules/07-interfaces
```

Sık Yapılan Hatalar

Arayüz yeterken parametreyi bir sınıfa bağlamak. Sizden sonra yazılan her tipi dışarıda bırakır.

Varsayılan metotlarla mixin kurmak. Kütüphane evrimi için yapıldılar. Arayüzler hâlâ durum tutamaz, dolayısıyla hatırlanacak bir şeyi olan "mixin" zaten çalışmaz.

Davranış eklemek için bir kütüphane sınıfını genişletmek. Sayaç örneği ne olduğunu gösteriyor. Onun yerine sarın.

Arayüz metotlarının örtük olarak public olduğunu unutmak. Birini daha dar görünürlikle uygulamak derleme hatasıdır ve mesaj ilk seferinde apaçık değildir.

"İleride paylaşılan kod olabilir" diye soyut sınıfa uzanmak. Arayüzle başlayın. Altına sonradan soyut sınıf eklemek kolaydır; uygulayıcının kalıtım yuvasını geri almak değildir.

Özet Çıkarımlar

- **Bir üst sınıf, çok arayüz.** Arayüzlerin var olma sebebi bu sınırdır.
- **İşi gören en dar yeteneği kabul edin**, böylece gelecekteki tipler onu karşılayabilsin.
- **"Yapabilir" arayüz, "bir çeşididir" sınıftır.** İkisi de uyuyorsa arayüzü alın.
- **Varsayılan metotlar kütüphanelerin uygulayıcıları kırmadan büyümesi için var.**
- **Çakışan varsayılanlar derleme hatasıdır**, `Interface.super.method()` ile çözülür.
- **Kalıtım sizi üst sınıfın API'sine değil uygulamasına bağlar.** Aynı geçersiz kılma `ArrayList` altında doğru, `HashSet` altında yanlış.

Ödev

[homework/README.md](#)

Yeteneklerle tasarlayın, sonra kırılğan temel sınıf hatasını yeniden üretip kompozisyonla düzeltin. Referans çözüm [solutions/07-interfaces/](#) altında.

Ödev 07: Yetenekler, Çakışmalar ve Kırılğan Bir Temel

Üç bölüm. Yeteneklerle tasarlayın, Java'nın sizin yerinize çözmeyi reddettiği bir çakışmayı çözün, sonra "kompozisyonu tercih edin" sözünü slogandan fazlası yapan hatayı yeniden üretin.

Birinci Bölüm: Akıllı Cihazlar

`Devices.java` dosyasını klasik biçimde oluşturun.

İki arayüz tanımlayın:

- `Switchable` — uygulayıcının sağlaması gereken bir `name()` metodu, artı **varsayılan** metotlar olarak `turnOn()` ve `turnOff()`
- `Dimmable` — varsayılan olarak `setLevel(int)` ve `full()`, ve seviyeyi 0..100 aralığına sıkıştırmak için bir **özel arayüz metodu**, böylece kural bir kez yazılsın

Sonra üç cihaz sınıfı:

Cihaz	Uyguladığı
<code>Lamp</code>	ikisi de
<code>Thermostat</code>	yalnızca <code>Switchable</code>
<code>Speaker</code>	ikisi de, ama <code>setLevel</code> metodunu geçersiz kılar

`main` içinde cihazlar üzerinde **arayüz tipiyle** döngü kurun, asla somut sınıfla değil. Bir `Thermostat` nesnesini `Dimmable` beklenen yere geçirmenin imkânsız olduğunu gösterin ve bunu nereden bildiğinizi bir yorumda söyleyin.

İkinci Bölüm: Çözmek Zorunda Olduğunuz Bir Çakışma

Her biri aynı ad ve imzaya sahip **varsayılan** metot içeren `Timestamped` ve `Versioned` tanımlayın. İkisini birden uygulayan bir sınıf yazın.

Derlenmeyecek. Hatayı okuyun, sonra açıkça çözün.

Bir yorumda aldığınız hatayı alıntılayın ve Python'un metot çözümleme sırasının sessizce seçeceği yerde Java'nın sizin yerinize seçmeyi neden reddettiğini açıklayın.

Üçüncü Bölüm: Bir Alt Sınıfı Kırın

Eklenen her şeyin kaydını tutan, `add` ve `addAll` metotlarını geçersiz kılan `LoggingHashSet extends HashSet<String>` yazın.

Üç öğeyi `addAll` kullanarak ekleyin ve kayıt boyutunu yazdırın. 3 olmayacak.

Sonra bir `HashSet` genişletmek yerine **tutan**, aynı iki metoda sahip `LoggingCollection` yazın. Aynı üç öğeyi ekleyin ve kayıt boyutunu yazdırın.

Açıklayın

Kendi kelimelerinizle şunları kapsayan bir yorum yazın:

1. Mekanizma. Miras alan sürüm neden fazla sayıyor?
2. `LoggingHashSet` sınıfının kendi kaynağında neden yanlış görünen hiçbir şey yok?
3. Kompoze edilmiş sürüm farklı olarak ne yapıyor ve sorunu nasıl ortadan kaldırıyor?

Kabul Kriterleri

- ☐ `java Devices.java` ile çalışıyor
- ☐ `Switchable` ve `Dimmable` her biri en az bir `default` metot içeriyor
- ☐ `Dimmable` varsayılanlarının kullandığı bir `private` arayüz metodu içeriyor
- ☐ `Speaker` bir varsayılanı geçersiz kılıyor, `Lamp` kılmıyor
- ☐ `main` içindeki döngüler arayüz tipiyle yazılmış, sınıfla değil
- ☐ Çakışan varsayılanlı sınıf `Interface.super.method()` ile çözüyor
- ☐ Bir yorum gördüğünüz gerçek derleyici hatasını alıntılıyor
- ☐ `LoggingHashSet` 3 olmayan bir kayıt boyutu yazdırıyor
- ☐ `LoggingCollection` 3 yazdırıyor
- ☐ Açıklamanız belirtiyi değil mekanizmayı isimlendiriyor

İleri Seviye

`LoggingHashSet` sınıfını, diğer her şey aynı kalacak şekilde `ArrayList<String>` genişletecek biçimde değiştirin. Sayı doğru çıkacak.

Bunun özgün hata hakkında size ne söylediğini ve incelediğiniz ama ebeveynini okumadığınız bir alt sınıfa ne kadar güvenebileceğinizi bir yorumda yazın.

İpucu, üçüncü bölüm hemen 3 veriyorsa

Hangi koleksiyonu genişlettiğinizi kontrol edin, ve öğeleri üç ayrı `add` çağrısı yerine `addAll` ile eklediğinizden emin olun. Her iki ayrıntı da önemli ve bunlardan biri alıştırmanın tamamı.

Modül 08: Record'lar ve Enum'lar

Modül 06 size `equals`, `hashCode` ve `toString` metotlarını elle yazdırdı ve yanlış yaptığınızda bir `HashSet` kümesinin içeriğini kaybettiğini gösterdi. Bu modül o problemin cevabı.

Record'lar veri taşır. Enum'lar örnek kümesini sabitler. İkisi birlikte, eski Java'nın meşhur olduğu elle yazılan tekrar kodunun büyük bölümünü ortadan kaldırır.

Neler Öğreneceksiniz

- Bir record'un ne ürettiği, iddia edilerek değil `javap` ile gösterilerek
- Doğrulama ve normalleştirme için kompakt kurucular
- Bir record için "değişmez" sözcüğünün kulağa geldiğinden neden daha sığ olduğu
- Durum ve davranış taşıyan, sabit örnekli sınıflar olarak enum'lar
- `ordinal()` metodunun neden bir tuzak olduğu
- Sınıf, record ve enum arasında nasıl seçim yapılacağı

Başka Bir Dilden Geliyorsanız

	Python	Java
Veri taşıyıcı	<code>@dataclass</code> ya da <code>NamedTuple</code>	<code>record</code>
Eşitlik	dekoratör üretir	record üretir
Sabit örnek kümesi	<code>enum.Enum</code>	<code>enum</code> , ama her sabit tam bir nesne
Sabit başına davranış	zahmetli	her sabitin uyguladığı soyut metot
Değişmezlik	<code>frozen=True</code> , yine de sığ	record'lar, yine de sığ

Sıgılık uyarısı her iki dilde de geçerli ve aynı sebepten. Değiştirilebilir bir liste tutan dondurulmuş bir kap, hâlâ değiştirilebilir bir liste tutar.

Ders

Record nedir

```
record Point(int x, int y) { }
```

O tek satır size bir kurucu, `equals`, `hashCode`, `toString` ve bileşen başına bir erişimci verir. `Records.java` dosyasından:

```
toString: Point[x=3, y=4]
equals:    true
hashCode: true
set size: 1
```

Son satır asıl mesele. Modül 06'nın elle yazılmış `BadPoint` sınıfı iki eşit nesneyi bir `HashSet` içine koydu ve sonra ikisini de bulamadı. Bir record o hatayı yapamaz.

Neyin üretildiğini görmek için Modül 01'deki tekniği kullanın. `public record Money(long cents, String currency) { }` için:

```
public final class Money extends java.lang.Record {
    public Money(long, java.lang.String);
    public final java.lang.String toString();
    public final int hashCode();
    public final boolean equals(java.lang.Object);
    public long cents();
    public java.lang.String currency();
}
```

Bundan üç şey çıkar:

- `final class` — record'lar asla genişletilemez.
- `extends Record` — her record'un zaten bir üst sınıfı vardır, dolayısıyla başka bir şeyi genişletemez. Arayüzleri serbestçe uygulayabilir.
- **Erişimcilerde `get` öneki yoktur.** `p.getX()` değil `p.x()`.

Bir record ek metotlar, statik fabrikalar ve arayüzler içerebilir. İçeremeyeceği şey **ek örnek durumudur**:

```
record Bad(int x) { private int cached; } // does not compile
```

Bu kısıt garantinin kendisidir. Bir record *bileşenlerinin ta kendisidir*, dolayısıyla eşit bileşenli iki record birbirinin yerine geçer.

Kompakt kurucular

Bir record'un doğrulama ve normalleştirme yaptığı yer. `CompactConstructors.java` dosyasından:

```
record Email(String address) {
    Email {
        if (address == null || !address.contains("@")) {
            throw new IllegalArgumentException("not an email address: " + address);
        }
        address = address.strip().toLowerCase();
    }
}
```

Parametre listesi yok, alan ataması yok. Derleyici parametreleri bu gövde çalıştıktan **sonra** alanlara atar, dolayısıyla parametreyi yeniden atamak neyin saklanacağını değiştirir. Burada `this.address = ...`

yazmak derleme hatasıdır.

Normalleştirme görüldüğünden daha önemli. Bir record'un `equals` metodu bileşenleri karşılaştırır, dolayısıyla mantıksal olarak eşit iki değer farklı bileşenlere sahip olabiliyorsa record yalan söylüyordur:

```
new Fraction(6, 8)    -> 3/4
new Fraction(-1, -2) -> 1/2
```

Kurucuda sadeleştirme ve işaret normalleştirilmesi olmasaydı `6/8` ile `3/4` aynı sayı oldukları hâlde eşitsiz olurdu. **Bir record ham bir demet yerine bir kavramı temsil ettiğinde, kompakt kurucuda normalleştirin.**

Sığ değişmezlik

Bir record'un alanları `final` olur. Dilin söz verdiği tek şey bu. [ShallowImmutability.java](#) dosyasından:

```
--- the leak ---
built with: Leaky[title=post, tags=[draft]]
after the caller edited their own list: Leaky[title=post, tags=[draft, smuggled-in]]
after editing the accessor's result:    Leaky[title=post, tags=[draft, smuggled-in,
smuggled-out]]

--- sealed off ---
built with: Safe[title=post, tags=[draft]]
after the caller edited their own list: Safe[title=post, tags=[draft]]
the accessor's list refused modification
```

Tek satırla kapatılan iki ayrı sızıntı:

```
record Safe(String title, List<String> tags) {
    Safe {
        tags = List.copyOf(tags);
    }
}
```

`List.copyOf` kopyalar, böylece çağıranın listesi artık sizin değildir, ve değişmez bir liste döndürür, böylece erişimci onu doğrudan vermekte serbesttir.

Bu record'lar için sıradan sınıflardan daha çok önemlidir, çünkü record'lar tam olarak `equals` metotları bedava geldiği için harita anahtarı ve küme elemanı olarak kullanılır. Modül 06'nın mutasyon tuzağı burada tümüyle geçerli: bir `Leaky` nesnesini `HashSet` içine koyun, listesini değiştirin, nesne onu hâlâ sayan kümenin içinde kaybolur.

Bir bileşen koleksiyon olduğunda kompakt kurucuya her seferinde `List.copyOf` yazın.

Enum'lar

Bir enum tam sayı sabitleri listesi değildir. Her sabit, kendi durumu ve isterseniz kendi davranışı olan tam bir nesnedir. `Enums.java` dosyasından:

```
MERCURY  gravity  3.70   a 75kg person weighs  277.7 N
EARTH    gravity  9.80   a 75kg person weighs  735.2 N
MARS     gravity  3.71   a 75kg person weighs  278.4 N
```

Sabite özgü davranış bilinmeye değer özelliktir:

```
enum Operation {
    PLUS("+") { @Override double apply(double a, double b) { return a + b; } },
    MINUS("-") { @Override double apply(double a, double b) { return a - b; } };

    abstract double apply(double a, double b);
}
```

`apply` metodu olmadan sabit eklemek derleme hatasıdır. Alternatif, başka bir yerdeki bir `switch`, elle bulunup güncellenmek zorundadır.

`ordinal()` bir tuzaktır. Bildirim konumudur, dolayısıyla araya sabit eklemek sonrasındaki her şeyi sessizce yeniden numaralandırır:

```
PENDING -> stored as P, ordinal happens to be 0
ACTIVE  -> stored as A, ordinal happens to be 1
FAILED  -> stored as X, ordinal happens to be 2
```

Bir ordinal değerini asla kalıcı olarak saklamayın ve asla onun üzerinde switch yapmayın. Sabite, `Status` örneğindeki gibi açık bir kod alanı verin.

Enum'lar, her sabit tekil olduğu için `==` kullanımının deyimsel olduğu tek yerdir.

Biçimi seçmek

Kullanın	Ne zaman
<code>record</code>	tip verisinin kendisiyse ve aynı veriye sahip iki örnek aynı şeyse
<code>enum</code>	örnek kümesi sabit ve derleme zamanında biliniyorsa
<code>class</code>	kimliği, değiştirilebilir durumu ya da verisinden ağır basan davranışı varsa

Karani veren soru: **iki örnek aynı değerleri tutuyorsa birbirinin yerine geçer mi?** İki `Point(3, 4)` geçer. Aynı bakiyeye sahip iki banka hesabı geçmez.

Çalıştırın

```
java modules/08-records-and-enums/examples/Records.java
java modules/08-records-and-enums/examples/CompactConstructors.java
java modules/08-records-and-enums/examples/ShallowImmutability.java
java modules/08-records-and-enums/examples/Enums.java

./scripts/verify-examples.sh modules/08-records-and-enums
```

Sık Yapılan Hatalar

Bir record'a değiştirilebilir koleksiyon koyup ona değişmez demek. İki sızıntı, tek satır düzeltme.

`ordinal()` değerini kalıcı saklamak. Biri araya sabit ekler ve saklanmış her satır artık başka bir şey demektir.

Kimliği olan bir şey için record kullanmak. Bir `User` record'u, aynı ad ve e-postaya sahip iki kullanıcının aynı kullanıcı olduğu anlamına gelir. Bazen doğru, genellikle değil.

Normalleştirmeyi unutmak. Ham bileşenleri karşılaştıran bir record, `6/8` ile `3/4` değerlerini farklı kesirler sayar.

Kompakt kurucuda `this.x = x` yazmak. Derlenmez. Parametreye atayın.

Bir enum üzerinde beş ayrı yerde `switch` yazmak. Sabite özgü davranış mantığı veriyle birlikte tutar ve bir eksiği derleme hatasına çevirir.

Özet Çıkarımlar

- **Bir record bileşenlerinin ta kendisidir.** Derleyici `equals`, `hashCode`, `toString` ve erişimcileri doğru biçimde yazar.
- **Record'lar final'dır ve zaten `Record` sınıfını genişletir,** dolayısıyla kalıtım yok, ama arayüzler sorun değil.
- **Kompakt kurucular doğrular ve normalleştirir,** ve parametreye atarlar.
- **Değişmezlik sığdır.** Koleksiyon bileşenini her seferinde `List.copyOf` yapın.
- **Enum sabitleri nesnedir** ve durum ile sabite özgü davranış taşıyabilir.
- `ordinal()` değerini kalıcı hiçbir şey için kullanmayın.
- **Eşit veri eşit şey demekse record seçin,** örnekler sabitse enum, aksi hâlde sınıf.

Ödev

[homework/README.md](#)

Record'lar ve enum'larla küçük bir sipariş sistemi modelleyin, sonra bilerek yarattığınız iki değişmezlik sızıntısını kapatın. Referans çözüm [solutions/08-records-and-enums/](#) altında.

Ödev 08: Bir Sipariş Sistemi

Küçük bir sipariş sistemi modelleyin, sonra bilerek yarattığınız iki değişmezlik sızıntısını kapatın.

Birinci Bölüm: Money

`Orders.java` dosyasını klasik biçimde oluşturun.

`long cents` ve `String currency` içeren bir `Money` record'u yazın. Kompakt kurucusu şunları yapmalı:

- negatif tutarı reddetsin
- null para birimini reddetsin
- para birimini boşlukları kırparak ve küçük harfe çevirerek **normalleştirsin**
- `gbp`, `eur`, `usd` destekli kümesinin dışındaki her para birimini reddetsin

Sonra `new Money(1250, "gbp")` ile `new Money(1250, "GBP ")` değerlerinin `equals` olduğunu ve ikisini bir `HashSet` içine koymanın tek eleman verdiğini gösterin.

Doğru yapılması gereken bir sıralama ayrıntısı var: desteklenen kümeye karşı denetlemeden **önce** normalleştirin, sonra değil. Nedenini bir yorumda yazın.

Ayrıca farklı para birimlerini toplamayı reddeden `plus(Money)` ve tam kuruşla yüzde hesaplayan `percent(int)` metotlarını ekleyin.

İkinci Bölüm: İki Sızıntı

`String reference` ve `List<String> notes` içeren, kompakt kurucusu **olmayan** bir `LeakyOrder` record'u yazın.

Sonra `main` içinde:

- Referansını sakladığınız bir listeden bir tane kurun
- Sonradan *kendi* listenize ekleme yapın ve record'u yazdırın
- Erişimcisinin döndürdüğü listeye ekleme yapın ve record'u yazdırın

Her iki saldırı da başarılı olacak. Sonra düzeltilmiş sürüm olan `Order` record'unu yazın ve her iki saldırının da başarısız olduğunu gösterin.

Açıklayın

Bir yorumda **her iki** sızıntıyı ayrı ayrı tarif edin. Aynı sızıntı değiller ve birine yapılan düzeltme diğerini otomatik olarak düzeltmez. Sonra Modül 06'ya atıfla, bunun neden sıradan bir sınıftan çok bir record için önemli olduğunu söyleyin.

Üçüncü Bölüm: Kargo ve Durum

Her biri bir açıklama ve **kendi ücret kuralını** sabite özgü davranış olarak taşıyan üç seçenekli bir `Shipping` enum'u yazın. Standart kargo 50.00 üzerinde bedava, altında 3.99. Ekspres her zaman 8.99. Mağazadan teslim her zaman bedava.

Her biri `"PLC"` gibi açık ve **kalıcı bir kod** taşıyan dört sabitli bir `OrderStatus` enum'u yazın. Kodun neden `ordinal()` yerine var olduğunu söyleyen bir yorum ekleyin.

Son olarak, **default** dalı olmayan bir `switch` ifadesiyle duruma göre tavsiye veren bir metot yazın, ve `default` yazmamanın size ne kazandırdığını açıklayan bir yorum ekleyin.

Kabul Kriterleri

- ☐ `java Orders.java` ile çalışıyor
- ☐ `Money` kompakt kurucusunda para birimini normalleştiriyor
- ☐ Bir yorum normalleştir-sonra-doğrula sıralamasını açıklıyor
- ☐ `Money(1250, "gbp")` ile `Money(1250, "GBP ")` eşit, çıktıda gösteriliyor
- ☐ `plus` uyuşmayan para birimlerini reddediyor
- ☐ `LeakyOrder` çıktısı her iki saldırının da başarılı olduğunu gösteriyor
- ☐ `Order` çıktısı her iki saldırının da başarısız olduğunu gösteriyor
- ☐ Bir yorum iki sızıntıyı ayrı ayrı tarif ediyor
- ☐ `Shipping` bir `switch` değil sabite özgü davranış kullanıyor
- ☐ `OrderStatus` nedenini anlatan bir yorumla birlikte açık bir kod taşıyor
- ☐ Tavsiye `switch` ifadesinde `default` yok

İleri Seviye

Tavsiye metodunuza bir `CANCELLED` dalı ekleyin, sonra `OrderStatus` enum'una beşinci bir sabit ekleyin ve `switch`'i **güncellemeyin**. Derleyin, tam hatayı kaydedin ve bir yoruma koyun.

Sonra bir `default` dalı ekleyip yeniden derleyin ve kodun güvenliği açısından neyin değiştiğini anlatın.

İpucu, ikinci bölümün ikinci saldırısı işe yaramıyorsa

Erişimcinizin ne döndürdüğüne bakın. Bir record, alanı saklandığı hâliyle geri veren bir erişimci üretir. Alan düz bir `ArrayList` ise çağırının aldığı şey de odur ve onunla istediğini yapabilir.

Modül 09: Mühürlü Tipler ve Desen Eşleme

Bu modül, daha önce tanıştığınız üç özelliğin ayrı şeyler olmaktan çıktığı yer. Mühürlü arayüzler durumların neler olduğunu, record'lar her durumun taşıdığı veriyi, kapsayıcı `switch` de bunlarla ne yapılacağını söyler. Derleyici üçünün birbirini tuttuğunu denetler.

Sonuç, diğer dillerin cebirsel veri tipleri dediği şeyin sıradan Java ile yazılmış hâli.

Neler Öğreneceksiniz

- `sealed` ve izin verilen her alt tipin seçmek zorunda olduğu üç niteleyici
- `instanceof` ile desen eşleme ve ortadan kaldırdığı dönüşüm
- Switch desenleri, `when` koşulları ve açık `case null`
- Record desenleri, iç içe parçalama dahil
- Kapsayıcılığın neden asıl kazanç olduğu ve `default` yazmanın size ne kaybettirdiği

Başka Bir Dilden Geliyorsanız

	Başka yerlerde	Java
Kapalı durum kümesi	Rust <code>enum</code> , Kotlin <code>sealed</code> , TS union	<code>sealed interface</code>
Parçalama	Python <code>match</code> , JS destructuring	record desenleri
Koşul	dal içindeki <code>if</code>	<code>when</code> ifadesi
Kapsayıcılık	Rust zorunlu tutar	mühürlü tipler için zorunlu

Rust ya da Kotlin yazdıysanız bu hemen tanıdık gelecek. Eski Java yazdıysanız, veri modelleme biçiminizi en çok değiştirecek özellik bu.

Ders

sealed

```
final      kimse bunu genişletemez
sealed     yalnızca şu isimlendirilmiş tipler genişletebilir
(hiçbiri)  herkes, her yerde, sonsuza dek
```

Modül 06 varsayılan olarak `final` kullanmayı savundu. `sealed`, alt tip *istediğiniz* ama bilinen ve sabit bir küme *istediğiniz* durumun cevabıdır:

```
sealed interface Shape permits Circle, Rectangle, Triangle { }
```

İzin verilen her alt tip kendi niyetini bildirmek zorundadır, `final`, `sealed` ya da `non-sealed` seçerek. Üçünü de yazmamak derleme hatasıdır:

```
error: sealed, non-sealed or final modifiers expected
```

Record'lar örtük olarak `final`'dır, bir record'un neden niteleyiciye ihtiyaç duymadığının sebebi budur. Bu eşleşme bilinçlidir ve modern Java veri modellemesinin çoğunun aldığı biçimdir.

Davetsiz bir tip reddedilir:

```
error: class is not allowed to extend sealed class: Result (as it is not listed in its
'permits' clause)
record Maybe(String hint) implements Result {
^
```

Durum kümesi tasarımın parçasıysa mühürleyin. Bir ödeme karttır, havaledir ya da hediye çekidir. Bir ayrıştırma ya başarılı olmuştur ya olmamıştır. Bunlarda "biri sonradan başka bir tane ekleyebilir" istediğiniz bir genişletme noktası değil, yakalanmasını istediğiniz bir risktir.

Eklenti arayüzünü mühürlemeyin. Üçüncü tarafların uygulaması gereken bir şeyi mühürlemek, kimsenin genişletemeyeceği bir kütüphane yayımlamanın yoludur.

Desen eşleme

`case String s` tipi test eder, dönüştürür ve sonucu adlandırır, hepsi bir arada. Aynı mantığı iki kez yazan `PatternMatching.java` dosyasından:

```
// modern
case String s when s.isEmpty() -> "an empty string";
case String s -> "text of length " + s.length();

// old
if (o instanceof String) {
    String s = (String) o;
    return s.isEmpty() ? "an empty string" : "text of length " + s.length();
}
```

Eski biçim tip adını iki kez yazar ve ayrı bir dönüşüm yapar. O dönüşüm hataların yaşadığı yerd: hiçbir şey sizi `if (o instanceof String) { Integer i = (Integer) o; }` yazmaktan alıkoymaz, ki bu derlenir ve fırlatır.

Bilinmeye değer üç ayrıntı:

- **Koşullar** `when` kullanır, ve koşullu durumlar aynı tip için koşulsuz olandan önce gelmelidir. İlk eşleşme kazanır, dolayısıyla başa konan koşulsuz bir `case String s` her şeyi yutar.
- **case null** açıktır. Onsuuz, null üzerinde switch yapmak uyumluluk için korunmuş önceki davranışla `NullPointerException` fırlatır.
- **instanceof** desenleri **switch dışında da çalışır**, ki onları en çok orada kullanacaksınız:

```
if (o instanceof String s && s.length() > 5) {  
    return s.substring(0, 5) + "...";  
}
```

Kapsam kuralı görüldüğünden zekidir. Erken bir dönüşten sonra değişken metodun geri kalanında kapsamdadır:

```
if (!(o instanceof String s)) {  
    return -1;  
}  
return s.length(); // s is in scope here
```

Record desenleri

Desenin kendisinde parçalama. `RecordPatterns.java` dosyasından:

```
case Circle(Point(var x, var y), var r) when x == 0 && y == 0 ->  
    "circle of radius " + r + " at the origin";  
  
case Rectangle(Point(var x1, var y1), Point(var x2, var y2))  
    when (x2 - x1) == (y2 - y1) ->  
    "a square of side " + (x2 - x1);
```

Gerçek çıktı:

```
circle of radius 5.0 at the origin  
a square of side 4  
a 6 by 3 rectangle
```

Kare tespiti için `Rectangle` üzerinde hiçbir metoda gerek olmadı. Desen tipi eşleştirdi, iç içe iki noktayı çıkardı, dört değeri bağladı ve karşılaştırdı, hepsi tek bir case etiketinde.

Kapsayıcılık asıl kazanç

Mühürlü tip üzerindeki bir switch `default` istemez, çünkü derleyici eksiksiz olduğunu kanıtlayabilir. Bir durumu dışarıda bırakın ve `NotExhaustive.java` gerçekleştir:

```
error: the switch expression does not cover all possible input values  
    return switch (payment) {  
        ^  
missing patterns:  
    Voucher _
```

Eksik tipi ismen söylüyor. `Voucher _` içindeki alt çizgi isimsiz bir desendir: derleyici size `Voucher` için bir dal gerektiğini ve değeri yok sayan bir dalın yeteceğini söylüyor.

Mühürlemenin tüm gerekçesi bu. `permits` listesine bir sabit ekleyin ve tüm kod tabanındaki her eksik switch kendini gösteren bir derleme hatasına dönüşür. Onun yerine `default -> 0` yazmak, ücretsiz bir

hediye çekini uyarısız gönderirdi.

Modül 03 bu argümanı enum'lar için yapmıştı. Mühürlü tipler onu, verisi olan kapalı herhangi bir biçim kümesine genişletiyor.

Çalıştırın

```
java modules/09-sealed-and-pattern-matching/examples/SealedTypes.java
java modules/09-sealed-and-pattern-matching/examples/PatternMatching.java
java modules/09-sealed-and-pattern-matching/examples/RecordPatterns.java

# İkisi de bilerek başarısız olur. Hangi tipi isimlendirdiklerini okuyun.
java modules/09-sealed-and-pattern-matching/examples/NotExhaustive.java
java modules/09-sealed-and-pattern-matching/examples/NotPermitted.java

./scripts/verify-examples.sh modules/09-sealed-and-pattern-matching
```

Sık Yapılan Hatalar

Mühürlü tip üzerindeki switch'e default eklemek. Gelecekteki her derleme hatasını sessiz bir yanlış cevaba çevirdiniz. Bu modüldeki en önemli alışkanlık budur.

Koşulsuz durumu başa koymak. `case String s when ...` öncesinde `case String s` yazmak koşullu dalı erişilemez kılar, ve derleyici bunu söyler.

case null unutmak. Listelediğiniz sürece null üzerinde switch fırlatır. Null mümkünse görünür biçimde ele alın.

Üçüncü tarafların uygulaması gereken bir arayüzü mühürlemek. Mühürleme sahip olduğunuz kapalı kümeler içindir, genişletme noktaları için değil.

İzin verilen bir alt tipi niteleyicisiz bildirmek. Her biri `final`, `sealed` ya da `non-sealed` demek zorunda. Record'lar zaten `final` diyor.

Mühürlü hiyerarşinin uyduğu yerde derin kalıtıma uzanmak. Alt tipler davranıştan çok veride ayrışlıyorsa, record'lardan oluşan mühürlü bir arayüz artı switch, geçersiz kılmalı soyut sınıftan genellikle daha nettir.

Özet Çıkarımlar

- sealed alt tiplerin eksiksiz kümesini isimlendirir**, ve her biri `final`, `sealed` ya da `non-sealed` bildirmek zorundadır.
- Desenler tek adımda test eder, dönüştürür ve bağlar**, eskiden yanlış giden dönüşümü ortadan kaldırarak.

- Koşullar `when` kullanır ve aynı tip için koşulsuz durumdan önce gelmelidir.
- `case null` açıktır; aksi hâlde null fırlatır.
- Record desenleri parçalar, ve iç içe girer, veri kadar derine.
- Mühürlü tip üzerinde `default` yazmayın. Eksik durum derleme hatası, satın aldığınız özelliktir.
- Mühürlü arayüz + record'lar + kapsayıcı switch, modern Java'nın kapalı bir veri biçimi kümesini modelleme yoludur.

Ödev

[homework/README.md](#)

Bir ifade değerlendiricisini mühürlü hiyerarşi olarak modelleyin, sonra derleyicinin unuttuğunuz durumu yakaladığını kanıtlayın. Referans çözüm [solutions/09-sealed-and-pattern-matching/](#) altında.

Ödev 09: Bir İfade Değerlendirici

Aritmetik ifadeleri mühürlü bir hiyerarşi olarak modelleyin, sonra derleyicinin bilerek unuttuğunuz bir durumu yakalamasına izin verin.

Bu, mühürlü tipler artı record'lar artı desen eşlemenin klasik kullanımındır ve bir kez elle yazmaya değer.

Birinci Bölüm: Model

`Calculator.java` dosyasını klasik biçimde oluşturun.

Beş record tipine izin veren bir `sealed interface Expr` tanımlayın:

Record	Bileşenler
<code>Literal</code>	bir <code>int value</code>
<code>Add</code>	iki <code>Expr</code>
<code>Multiply</code>	iki <code>Expr</code>
<code>Negate</code>	bir <code>Expr</code>
<code>Divide</code>	iki <code>Expr</code>

`Add`, `Multiply`, `Negate` ve `Divide` tiplerinin `int` değil `Expr` tuttuğuna dikkat edin. Yapıyı bir ağaç ve metotları özyinelemeli yapan şey budur.

İkinci Bölüm: Üç İşlem

Her biri `default` içermeyen tek bir `switch ifadesi` olan üç metot yazın:

- `int` sonucu döndüren `evaluate(Expr)`
- İkili işlemleri parantezleyerek okunaklı bir metin döndüren `print(Expr)`
- Ağacın kaç düğüm içerdiğini döndüren `countNodes(Expr)`

`(3 + 4) * -(2)` ifadesini kurun ve üçünü de üzerinde çalıştırın.

Bileşenleri çıkarmak için **record desenleri** kullanın. `case Add a` yazıp sonra `a.left()` çağırmak yerine `case Add(Expr left, Expr right)` yazın.

Üçüncü Bölüm: Derleyicinin İzlediğini Kanıtlayın

`permits` cümlesine altıncı bir record olan `Power` ekleyin. `evaluate` metodunu **güncellemeyin**.

Derleyin. Tam hatayı, isimlendirdiği eksik deseni dahil ederek bir yoruma kaydedin. Sonra ya `Power` durumunu ele alın ya da onu yeniden kaldırın.

`evaluate` metodunda bir `default` dalı olsaydı bunun yerine ne olacağını bir cümleyle yazın.

Dördüncü Bölüm: Sadeleştirme

Şu kurallar için **koşullar** kullanarak, daha basit ama eşdeğer bir `Expr` döndüren `simplify(Expr)` yazın:

Kural	Örnek
<code>0 + x</code> ve <code>x + 0</code>	<code>(0 + 7)</code> sonucu <code>7</code>
<code>1 * x</code> ve <code>x * 1</code>	<code>(1 * 9)</code> sonucu <code>9</code>
<code>0 * x</code> ve <code>x * 0</code>	<code>(0 * 9)</code> sonucu <code>0</code>
<code>--x</code>	<code>--5</code> sonucu <code>5</code>
sabit katlama	<code>(2 + 3)</code> sonucu <code>5</code>

Çift olumsuzlama kuralı ilginç olanı. **İç içe** bir record deseni istiyor, tek bir case etiketinde iki seviye eşleştiriyor.

Burada sıra önemli. Koşullu durumların aynı tip için koşulsuz olandan neden önce gelmesi gerektiğini bir yorumda söyleyin.

Kabul Kriterleri

- ☐ `java Calculator.java` ile çalışıyor
- ☐ `Expr` açık bir `permits` cümlesiyle `sealed`
- ☐ `evaluate`, `print` ve `countNodes` her biri `default` içermeyen tek switch
- ☐ Parçalamak için erişimci çağrılar değil record desenleri kullanılıyor
- ☐ Bir yorum ürettiğiniz gerçek "missing patterns" hatasını alınıyor
- ☐ Bir cümle `default` dalının neye mal olacağını açıklıyor
- ☐ `simplify` beş kuralın hepsini koşullarla uyguluyor
- ☐ Çift olumsuzlama kuralı iç içe bir record deseni kullanıyor
- ☐ Bir yorum koşullu-önce-koşulsuz sıralamasını açıklıyor

İleri Seviye

Sıfıra bölme, bir `int` döndürerek ele alınamaz. Sonucu, `Success(int value)` ve `Failure(String reason)` tiplerine izin veren ikinci bir mühürlü arayüz `Result` olarak modelleyin ve onu döndüren `evaluateSafely` yazın.

Sonra bunu istisna fırlatmakla karşılaştıran bir yorum yazın. Modül 10 sırada istisnaları anlatıyor, dolayısıyla her yaklaşımın çağırana neye mal olduğunu okumadan önce not edin.

İpucu, iç içe desen derlenmiyorsa

`Negate` bir `Expr` tutar, ve `Negate` kendisi de bir `Expr` tir. Dolayısıyla bileşeni başka bir `Negate` olan bir `Negate`, tipi iki kez isimlendirip ikisini de parçalayarak eşleştirilir. Dış deseni önce yazın ve içtekini parantezlerinin içine bırakın.

Modül 10: İstisnalar

Java istisnaları iki aileye ayırır ve birini derleyiciye denetletir. Bunu yapan neredeyse tek dildir, ve bunun iyi bir fikir olup olmadığı tartışması yirmi beş yıldır sürüyor.

Bu modül mekanizmayı anlatır, tartışmayı adil biçimde aktarır ve insanlara gerçek hata ayıklama süresi kaybettiren üç başarısızlık biçimini gösterir: yutulan bir istisna, kaybolan bir sebep ve asıl sorunu gizleyen bir `close()` çağırısı.

Neler Öğreneceksiniz

- Denetlenen ve denetlenmeyen ayrımı, ve hangisini fırlatacağınıza nasıl karar vereceğiniz
- `try / catch / finally`, çoklu yakalama ve yakalama sırası
- `try-with-resources`, ters kapatma sırası ve bastırılan istisnalar
- `finally` içindeki `return` neden her stil kılavuzunda yasak
- Özel istisnalar ve insanların unuttuğu kurucu argümanı

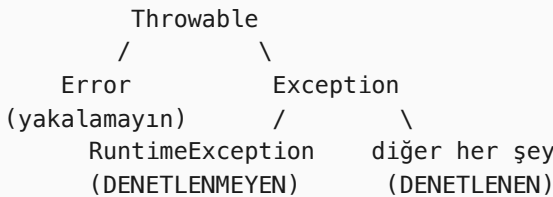
Başka Bir Dilden Geliyorsanız

	Python / JS	Java
Derleyicinin zorunlu kıldığı ele alma	yok	denetlenen istisnalar
Birkaç tipi yakalama	<code>except (A, B)</code>	<code>catch (A B e)</code>
Temizlik	<code>finally</code> , <code>with</code>	<code>finally</code> , <code>try-with-resources</code>
Kaynak protokolü	<code>__enter__ / __exit__</code>	<code>AutoCloseable</code>
İstisna zincirleme	<code>raise X from e</code>	<code>new X(msg, cause)</code>

Gerçekten yeni olan tek fikir denetlenen istisnalar. Diğer her şeyi zaten başka bir adla biliyorsunuz.

Ders

İki aile



- **Denetlenmeyen** (`RuntimeException` ve altı): derleyici bir şey söylemez. Bunlar programlama hatası demektir. Çözüm kodu değiştirmektir, onu ele almak değil.
- **Denetlenen** (`Exception` altındaki diğer her şey): derleyici sizi yakalamaya ya da bildirmeye zorlar.
- **Error**: JVM başı dertte. `OutOfMemoryError`, `StackOverflowError`. Bunları yakalamayın, içeriden düzeltemezsiniz.

Bilinmeye değer bir ayrıntı, `CheckedVsUnchecked.java` dosyasından:

```
unchecked: NumberFormatException -> For input string: "12x"  
unchecked: NumberFormatException -> Cannot parse null string
```

`Integer.parseInt(null)` çağırısı `NullPointerException` **değil**, `NumberFormatException` fırlatır. Javadoc okumak yerine tipi tahmin etmek, hiç tetiklenmeyen bir `catch` bloğu yazmanın yoludur.

Çoklu yakalama iki tipi aynı biçimde ele alır, ve değişken örtük olarak final'dır. **Yakalama sırası önemlidir**: üst tip alt tiplerinden sonra gelmeli, ve derleyici erişilemez bir catch'i orada bırakmak yerine reddeder.

Tartışma, adil biçimde

Lehine: çağırının kurtulabileceği bir hata, tip sisteminde görünür olmalıdır. Bir dosya var olmayabilir. Bu bir hata değil, hayatın olağan akışı, ve derleyicinin size bunu düşündürmesi bir özellik.

Aleyhine: pratikte insanları derleyiciyi susturmak için `catch (Exception e) {}` yazmaya itiyorlar, ki bu hiç denetim olmamasından kesinlikle daha kötü. Soyutlamalardan sızıyorlar, çünkü bir metoda `throws` eklemek yukarı doğru her çağırının imzasını değiştiriyor. Ve lambda'larla kötü uyuyorlar, stream API'sinde fırlatan bir fonksiyon kabul eden hiçbir şey olmamasının sebebi bu.

Java'dan sonra tasarlanan hiçbir dil onları kopyalamadı. Bu kanıttır, ispat değil.

Bugün ne yapmalı:

- Programlama hataları için denetlenmeyen: `IllegalArgumentException`, `IllegalStateException`.
- Denetlenen yalnızca çağırın gerçekten loglayıp pes etmekten başka bir şey yapabildiğinde.
- Asla `throws Exception` bildirmeyin. Çağırana hiçbir şey söylemez ve her şeyi yakalamaya zorlar.

try-with-resources

Mutlu yol dikkate değer değil. Tasarımın kendini gösterdiği yer hata yolu. `TryWithResources.java` dosyasından:

```
open A
open B
body running
close B (and failing)
close A (and failing)
caught: body failed
  suppressed: close B failed
  suppressed: close A failed
```

O çıktıdan iki şey çıkar:

1. **Kaynaklar ters sırada kapanır.** B en son açıldı, ilk kapandı. Bir kaynak diğerini sardığında bu önemlidir.
2. **Kapatma hataları yerine geçmez, bastırılır.** Yakaladığınız istisna gövdeninkidir; `close()` hataları ona iliştilir ve `getSuppressed()` ile okunabilir.

İkisi de elle yazılan eşdeğerinin bunları yanlış yapmasından dolayı var:

```
Resource r = null;
try {
    r = open();
    use(r);
} finally {
    if (r != null) r.close(); // if this throws, it REPLACES the real exception
}
```

Bir `close()` hatası asıl sorunu gizler ve gerçek sebep kaybolurdu.

Gövde erken döndüğünde de `close()` çalışır, dolayısıyla bunun için asla bir `finally` gerekmez.

finally ve istisnaları nasıl yediği

`finally` her zaman çalışır. Bu hem söz hem tuzaktır. `FinallySwallows.java` dosyasından:

```
static int swallowsTheException() {
    try {
        throw new RuntimeException("you will never see this");
    } finally {
        return 42;
    }
}
```

Çıktı: `42`. İstisna yok oldu. Log yok, yığın izi yok, hiçbir şey yok.

`finally` içindeki bir `return`, `try` bloğunun yaptığı her şeyi atar, uçuştaki bir istisna dahil. Aynı mekanizma iyi bir dönüş değerini de sessizce ezer, ki incelemede fark edilmesi çok daha zordur.

`finally` bloğuna asla `return`, `break` ya da `continue` koymayın. Her stil kılavuzu bunu yasaklar ve sebebi budur.

Ayrıca JVM çıkarsa `finally` çalışmaz. `try` içindeki `System.exit(0)` onu tamamen atlar. "Her zaman çalışır", normal bir metod çıkışı içinde demektir.

Özel istisnalar ve sebep

`CustomExceptions.java` dosyasından, aynı sarmalama iki kez yapılmış:

```
--- with the cause preserved ---
  caused by: java.io.IOException: users/7.json: no such file
  root cause: java.io.IOException: users/7.json: no such file

--- with the cause thrown away ---
  caused by: null
  the original IOException is gone. good luck.
```

Sebebi ikinci kurucu argümanıdır ve onu unutmak istisna işlemede en yaygın hatadır. Onsuz yığın izi yeniden fırlattığınız yerden başlar ve gerçekte neyin başarısız olduğu hakkında hiçbir şey söylemez.

Özel bir istisnaya her zaman `Throwable` sebep alan bir kurucu verin.

Mesaja gömmek yerine yapılandırılmış veri taşıyın, böylece çağırıcılar kimliği bir metinden geri ayrıştırmak yerine sorabilsin.

Denetlenen mi denetlenmeyen mi? `extends Exception` denetlenendir, `extends RuntimeException` değildir. Çağırıcının loglayıp pes etmekten başka bir şey gerçekten yapıp yapamayacağını sorun.

Çalıştırın

```
java modules/10-exceptions/examples/CheckedVsUnchecked.java
java modules/10-exceptions/examples/TryWithResources.java
java modules/10-exceptions/examples/FinallySwallows.java
java modules/10-exceptions/examples/CustomExceptions.java

# Bilerek başarısız olur.
java modules/10-exceptions/examples/UnhandledChecked.java

./scripts/verify-examples.sh modules/10-exceptions
```

Sık Yapılan Hatalar

Boş catch bloğu.

```
try { readConfig(path); } catch (IOException e) { }
```

Derlenir, derleyiciyi susturur, ve görünür bir hatayı sessizce yanlış iş yapan bir programa çevirir. Java'da yazabileceğiniz en kötü satır budur, ve denetlenen istisnaların bunu üretmesi özelliğe karşı en güçlü argümandır.

Sebebi kaybetmek. Catch bloğunun içinde `throw new MyException("failed")` yazmak, hata ayıklamak için ihtiyacınız olan her şeyi atar.

`finally` bloğunda `return`. Yukarıda anlatıldı. İstisnaları yer.

Her şeyi kapsamak için `Exception` yakalamak. Beklenen hataların yanında hataları da yakaladınız, nasıl ele alacağınızı bilmedikleriniz dahil.

`Throwable` ya da `Error` yakalamak. `OutOfMemoryError` durumundan kurtulamazsınız, ve onu yakalamak bir çökmeyi bir asılı kalmaya çevirir.

Akış kontrolü için istisna kullanmak. Döngüden çıkmak için fırlatmak yavaştır, çünkü yığın izi oluşturmak pahalıdır, ve mantığı okuyandan gizler.

`throws Exception` bildirmek. Çağıranlara her şeyin artık onların sorunu olduğu dışında hiçbir şey söylemez.

Özet Çıkarımlar

- **Denetlenmeyen hata demek, denetlenen öngörülebilir başarısızlık demektir.** Çağıranın gerçekte ne yapabileceğini sorarak seçin.
- **Asla boş catch bloğu yazmayın.** Gerçekten ele alamıyorsanız denetlenmeyen bir istisnaya sarın ve yolculuğuna devam etsin.
- **try-with-resources ters sırada kapatır ve yerine geçmek yerine bastırır**, ki elle yazılan temizlik bunu yıllarca yanlış yaptı.
- `finally` içindeki `return` istisnaları sessizce atar.
- **Sarmalarken her zaman sebebi geçirin**, ve özel istisnalara her zaman sebep kurucusu verin.
- **Asla `throws Exception` bildirmeyin.**

Ödev

[homework/README.md](#)

Küçük bir yapılandırma yükleyici kurun, sonra üç istisna yanlış kullanımını yeniden üretip her birini düzeltin. Referans çözüm [solutions/10-exceptions/](#) altında.

Ödev 10: Doğruyu Söyleyen Bir Config Yükleyici

Birkaç farklı biçimde başarısız olan bir şey kurun, sonra üç yanlış kullanımı yeniden üretin ki her birini bir kez kazara değil bilerek yazmış olun.

Birinci Bölüm: Yükleyici

`ConfigLoader.java` dosyasını klasik biçimde oluşturun.

`key=value` satırlarından ayrıştırılmış bir `Map<String, String>` döndüren `load(String name)` metodu yazın. Üç dosyayı taklit edin:

Ad	Davranış
<code>app.conf</code>	<code>host=localhost</code> ve <code>port=8080</code> döndürür
<code>missing.conf</code>	okuyucu <code>IOException</code> fırlatır
<code>broken.conf</code>	ayrıştırılır, ama <code>port</code> bir sayı değil

Şu özelliklere sahip bir `ConfigException` yazın:

- denetlenen** olsun, burada denetlenenin neden doğru seçim olduğunu söyleyen bir yorumla
- kaynak dosya adını** mesaja gömülü değil bir alan olarak taşısın
- `Throwable` sebep alan bir kurucusu olsun

`main` içinde üç dosyayı da yükleyin. Her başarısızlık için mesajı, kaynağı ve sebebi yazdırın.

`broken.conf` durumu ilginç olanı. `Integer.parseInt` *denetlenmeyen* bir `NumberFormatException` fırlatır. Onun olduğu gibi kaçmasına mı izin vereceğinize yoksa dönüştüreceklerinize mi karar verin ve seçiminizi bir yorumda gerekçelendirin.

İkinci Bölüm: Üç Yanlış Kullanım

Bunların her birini iki kez yazın, bir kez bozuk bir kez düzeltilmiş, ve her iki sonucu da yazdırın.

1. Boş catch bloğu. Bir şeyi yakalayın ve hiçbir şey yapmayın. Sonra düzeltin.

Bir yorumda, bunun programın çökmesine izin vermekten neden daha kötü olduğunu kendi kelimelerinizle açıklayın.

2. `finally` içinde `return`. `try` içinden fırlatın, `finally` içinden dönün, ve istisnanın yok olduğunu gösterin. Sonra düzeltin.

Bunun boş catch bloğunun kılık değiştirmiş hâli olduğunu açıklayın.

3. Çok geniş Exception yakalamak. Bir ayrıştırma hatasını ele almayı *amaçlayan* ama asıl hatası bir `NullPointerException` olan bir metot yazın ve işleyicinin onu yanlış raporladığını gösterin. Sonra düzeltin.

Geniş yakalamanın, hata ayıklamak zorunda kalan kişiye neye mal olduğunu açıklayın.

Üçüncü Bölüm: Kaynaklar

Tek bir try-with-resources içinde iki `AutoCloseable` kaynak açın. **Her iki** `close()` metodunu fırlatacak şekilde yazın, ve **gövdeyi** de fırlatacak şekilde.

Sonucu yakalayın ve yakaladığınız istisnayı artı `getSuppressed()` içeriğini yazdırın.

Sonra şunları kapsayan bir yorum yazın:

1. Hangi kaynağın önce kapandığı ve bu sıranın neden doğru olduğu
2. Hangi istisnayı yakaladığınız ve diğer ikisine ne olduğu
3. Bir `finally` bloğundaki elle yazılmış temizliğin bunun yerine ne yapacağı

Kabul Kriterleri

- ☐ `java ConfigLoader.java` ile çalışıyor
- ☐ `ConfigException` denetlenen, kaynağı alan olarak taşıyor ve sebep kurucusu var
- ☐ Üç config durumu da çalıştırılıyor ve sebebini yazdırıyor
- ☐ Bir yorum `ConfigException` için denetleneni gerekçelendiriyor
- ☐ Bir yorum `NumberFormatException` kararını gerekçelendiriyor
- ☐ Üç yanlış kullanım da bozuk ve düzeltilmiş hâlde görünüyor, çıktı farkı gösteriyor
- ☐ Her yanlış kullanımın kendi kelimelerinizle bir açıklaması var
- ☐ try-with-resources çıktısı ters kapatma sırasını ve iki bastırılan istisnayı gösteriyor
- ☐ Son sürümde, bilerek yanlış kullanım olarak etiketlenen hariç hiçbir boş catch bloğu kalmamış

İleri Seviye

`load` metodunu hiç fırlatmayacak, bunun yerine Modül 09'un ileri seviye bölümündeki gibi mühürlü bir `Result` tipi döndürecek şekilde yeniden yazın.

Sonra kısa bir karşılaştırma yazın. Çağıranın hangisini yanlış yapması daha kolay? Hangisi çağrı yerinde daha iyi okunuyor? Hangisi bir stream içindeki lambda'dan çağrılmaya dayanıyor, ve Modül 10'un denetlenen istisnalar ile stream API'si hakkında söyledikleri göz önüne alındığında bu neden önemli?

İpucu, bastırılan liste boş geliyorsa

Bastırılan istisnalar yalnızca **gövde** de fırlattığında ortaya çıkar. Gövde normal biterse ve bir `close()` fırlatırsa, o kapatma istisnası asıl istisna olur ve onu bastıracak bir şey kalmaz.

Modül 11: Jenerikler

Jenerikler bütün bir başarısızlık sınıfını çalışma zamanından derleme zamanına taşıdı. Java 5'ten önce her koleksiyon `Object` tutuyordu ve her okuma, doğru yapmak zorunda olduğunuz bir dönüşüm gerektiriyordu.

Özelliğin kullanımı kolay, yanlış okunması da kolaydır, çünkü 2004'te verilen tek bir tasarım kararı karşılaşılabileceğiniz neredeyse her kısıtı açıklar. O karar tip silmedir ve bu modül onunla başlıyor.

Neler Öğreneceksiniz

- Jenerik sınıflar ve metotlar, ve sınırların onları neden işe yarar kıldığı
- Tip silme, neyi yasakladığı ve yine de neden seçildiği
- Ham tipler, yığın kirlenmesi ve ortaya çıkan çökmenin gerçekte nereye düştüğü
- PECS üzerinden joker karakterler
- Jeneriklerin neden değişmez, dizilerin neden değişken olduğu ve hangi seçimin hata olduğu

Başka Bir Dilden Geliyorsanız

	Python	Java
Tip parametreleri	ipucu, zorlanmaz	derleme zamanında zorlanır
Çalışma zamanında	<code>__orig_class__</code> ile erişilebilir	silinmiş, hiçbir şey kalmaz
Değişkenlik	TypeVar üzerinde işaretlenir	kullanım yerinde joker karakterle
İlkel tipler	ayrım yok	<code>List<int></code> yasadışı

Sizi şaşırtacak olan satır tip silme. Java'da tip argümanı çalışma zamanında gerçekten var olmaz, ve şaşırtıcı sayıda kısıt bu tek olgudan çıkar.

Ders

Temeller

```
class Box<T> {  
    private final T value;  
    T get() { return value; }  
}  
  
static <T> T first(List<T> items) { return items.get(0); }
```

Jenerik bir metotta açılı parantezler dönüş tipinden **önce** gelir ve tip argümanlardan çıkarılır.

Sınırlı tip parametreleri, jenerik bir metodun değerleri oradan oraya taşımaktan fazlasını yapmasını sağlayan şeydir:

```
static <T extends Comparable<T>> T max(List<T> items)
```

Sınır olmadan `T` bir `Object` gibi ele alınır ve `compareTo` diye bir şey yoktur. Birden fazla sınır `&` kullanır, sınıf önce gelir: `<T extends Number & Comparable<T>>`.

Geleneksel adlar `T` tip, `E` eleman, `K` anahtar, `V` değer, `R` sonuç. Bunlar yalnızca gelenektir.

Tip silme

`Eraseure.java` dosyasından:

```
List<String> class: java.util.ArrayList
List<Integer> class: java.util.ArrayList
same class: true
```

Tip argümanı hiçbir iz bırakmadı. **Neden böyle çalışıyor:** jenerikler JVM'den on beş yıl sonra geldi. Tip silme, jenerik ve jenerik öncesi kodun birbirini çağırabilmesi ve değişmemiş bir JVM üzerinde çalışabilmesi demektir, böylece mevcut kütüphanelerin yeniden yazılması gerekmedi. Bilinçli bir takastı: o zaman uyumluluk, karşılığında sonsuza dek bu kısıtlar.

Tip silmenin yasakladıkları:

Yasak	Sebebi
<code>x instanceof List<String></code>	çalışma zamanında öyle bir tip yok
<code>new T[]</code> , <code>new T()</code>	tahsis edilecek bir şey yok
<code>show(List<String>)</code> ve <code>show(List<Integer>)</code>	aynı silme, çakışmalar
<code>List<int></code>	ilkel tipler nesne değil
<code>Box<T></code> içinde <code>static T shared;</code>	sınıf başına tek statik alan, tip argümanı başına değil

Çalışma zamanında tipe gerçekten ihtiyacınız olduğunda standart çözüm bir `Class<T>` geçirmektir, kütüphanedeki bu kadar çok metodun birini almasının sebebi budur.

Ham tipler ve yığın kirlenmesi

`HeapPollution.java` bir uyarıyla derlenir ve sonra başarısız olur:

```
List<String> words = new ArrayList<>();
List raw = words; // raw type, legal for backward compatibility
raw.add(42); // unchecked warning, allowed
```

```
the list now contains: [legitimate, 42]
reading element 0 is fine: legitimate
reading element 1 will not be:
Exception in thread "main" java.lang.ClassCastException:
    class java.lang.Integer cannot be cast to class java.lang.String
```

Çökmenin nereye düştüğüne bakın. Kuralı bozan satır `raw.add(42)` idi. Başarısızlık `get` çağrısında, çünkü derleyici bildirilen tipin söz verdiği dönüşümü oraya koydu. Gerçek kodda bu iki satır genellikle farklı kişilerce yazılmış farklı sınıflardadır, ve o mesafe yığın kirlenmesini hata ayıklaması zor kılan şeydir.

Yeni kodda asla ham tip kullanmayın. Unchecked uyarıları, derleyicinin artık hiçbir şeyi garanti edemez hâle geldiği anlamına gelir. Birini bastırmak zorunda kaldığınızda `@SuppressWarnings` anotasyonunu mümkün olan en dar kapsama koyun ve neden güvenli olduğunu söyleyin.

Joker karakterler: PECS

Producer Extends, Consumer Super.

Jenerikler **değişmezdir**: `Integer` bir `Number` olmasına rağmen `List<Integer>` bir `List<Number>` değildir. Joker karakterler olmadan eleman tipi başına bir aşırı yükleme gerekirdi.

```
// PRODUCER: reads from the list, so any subtype of Number will do
static double sum(List<? extends Number> numbers)

// CONSUMER: writes into the list, so any supertype of Integer will do
static void addAll(List<? super Integer> destination, List<Integer> source)
```

Her yön size değerini kaybettirir:

- `? extends` **salt okunurdur**. Liste bir `List<Double>` olabilir, dolayısıyla bir `Integer` eklemek onu bozardı. Yalnızca `null` eklenebilir.
- `? super` **Object olarak okunur**. Liste bir `List<Object>` olabilir, dolayısıyla derleyicinin söz verebileceği tek şey budur.

`List<?>` sınırsız biçimdir: `Object` okuyun, boyuta bakın, hiçbir şey eklemeyin. Ham `List` tipinden önemli olan biçimde ayrılır, çünkü `List<?>` tip güvenlidir ve zorlanır, ham tip ise denetimi tamamen kapatır.

Değişmezlik doğru karardı

Diziler öteki cevabı seçti. `Wildcards.java` dosyasından:

```
Object[] array = new String[2]; // compiles: arrays are covariant
array[0] = 42; // throws at runtime
```

```
ArrayStoreException: java.lang.Integer
```

O karar yüzünden her dizi yazma işlemi bir çalışma zamanı tip denetimi taşır. Jenerikler eşdeğerini derleme zamanında reddeder, dolayısıyla hiçbir denetim gerekmez ve hata üretime ulaşamaz.

Çalıştırın

```
java modules/11-generics/examples/Generics.java
java modules/11-generics/examples/Erasure.java
java modules/11-generics/examples/Wildcards.java

# Bu ikisi bilerek başarısız olur, biri çalışma zamanında biri derleme zamanında.
java modules/11-generics/examples/HeapPollution.java
java modules/11-generics/examples/CannotCreateGenericArray.java

./scripts/verify-examples.sh modules/11-generics
```

Sık Yapılan Hatalar

Bir uyarıyı susturmak için ham tip kullanmak. Özelliğin var olma sebebi olan denetimi kapattınız.

Tüm sınıfa ya da metoda `@SuppressWarnings` koymak. O kapsamdaki gelecekteki her hatayı da gizler. Yalnızca ihtiyaç duyan tek bildirime koyun.

`new T[size]` denemek. Bir `List` kullanın, ya da `Object[]` tahsis edip dönüştürün ve diziyi sınıfa özel tutun.

Eklemeniz gereken bir parametre için `List<? extends Number>` yazmak. Ekleyemezsiniz. Hem okuyup hem yazıyorsanız düz bir `List<T>` kullanın.

Dönüş tipinde joker karakter. Her çağıranı joker karakterle uğraşmaya zorlar. Somut tipi döndürün.

Tip parametresi yeterken `Object` kullanmak. O zaman her çağıranın bir dönüşüme ihtiyacı olur, ki bu tam olarak 2004 öncesi durumdur.

Özet Çıkarımlar

- **Jenerikler derleme zamanında denetlenir ve çalışma zamanında silinir.** Neredeyse her kısıt bundan çıkar.
- **Sınırlar, jenerik kodun değer taşımanın ötesinde bir şey yapmasını sağlar.**
- **Ham tipler özelliğin tamamını bozar,** ve ortaya çıkan çökme sebep olan satırdan çok uzakta görünür.
- **PECS:** okumak için üretici `extends`, yazmak için tüketici `super`.
- `? extends` yazılamaz; `? super` `Object` olarak okunur.
- **Jenerikler değişmez, diziler değişkendir.** Dizi seçimi hataydı ve `ArrayStoreException` onun faturası.

Ödev

[homework/README.md](#)

Jenerik bir kap kurun, üç tip silme kısıtına çarpın ve bilerek yığın kirlenmesine sebep olun. Referans çözüm [solutions/11-generics/](#) altında.

Ödev 11: Tipli Bir Önbellek

Jenerik bir kap kurun, bilerek üç tip silme kısıtına çarpın ve yığın kirlenmesine sebep olup suçun tam olarak nereye düştüğünü görün.

Birinci Bölüm: Önbellek

`TypedCache.java` dosyasını klasik biçimde oluşturun.

Sabit kapasiteli bir `Cache<K, V>` yazın:

- `put(K key, V value)` — doluyken eklemeden önce **en eski** girdiyi çıkarsın
- `get(K key)` — değeri ya da `null` döndürsün
- `size()` ve `keys()`

`Cache<String, Integer>` ile çalıştığını gösterin, çıkarmanın gerçekleştiğini kanıtlayın, sonra tek bir uygulamanın ilgisiz tiplere hizmet ettiğini göstermek için aynı sınıfı `Cache<Integer, List<String>>` olarak kullanın.

Kısıtlara Çarpın

Yazarken şunların her birini deneyin, derlenmediğini doğrulayın, sonra başka biçimde çözün. Üçünü de sınıfa yazacağınız bir yorumda belgeleyin:

- Değerleri bir `V[]` dizisinde saklayın
- `if (value instanceof V)` ile doğrulayın
- `static V defaultValue;` ile bir yedek paylaşın

Her biri için hatanın ne olduğunu ve onun yerine ne yaptığınızı söyleyin.

İkinci Bölüm: Sınırlar ve PECS

Üç metot yazın:

Metot	İmza biçimi
<code>largest(List<T>)</code>	<code>compareTo</code> çağırabilmesi için sınır gerekiyor
<code>average(...)</code>	sayı okur, asla yazmaz
<code>drainInto(...)</code>	tam sayı yazar, asla okumaz

`average` ve `drainInto` metotlarına doğru joker karakterleri verin. `average` metodunun hem `List<Integer>` hem `List<Double>` üzerinde, `drainInto` metodunun da hem `List<Number>` hem `List<Object>` içine çalıştığını kanıtlayın.

Açıklayın

Kendi kelimelerinizle şunları kapsayan bir yorum yazın:

- `largest` neden sınırı olmadan çalışamaz
- `average` neden bir joker karakteri, `drainInto` neden diğerini kullanıyor
- Her joker karakterin neyi **feda ettiği**, ve o fedanın onu neden güvenli kıldığı

Yalnızca "producer extends, consumer super" ifadesini tekrarlayan cevaplar sayılmaz. Her durumda derleyicinin size neyi yaptırmayacağını söyleyin.

Üçüncü Bölüm: Yığını Kirletin

Bir `List<String>` kurun, **ham** bir `List` değişkenine atayın ve ham referans üzerinden bir `Integer` ekleyin.

Sonra sırayla şunları yazdırın:

1. Listenin içeriği
2. Eleman 0, `String` olarak okunmuş
3. Eleman 1, `String` olarak okunmuş, ne olduğunu yakalayarak

Asıl Önemli Soru

Şunu cevaplayan bir yorum yazın: **hangi satır kuralı bozdu ve hangi satır fırlattı?**

Sonra neden farklı satırlar olduklarını, derleyicinin nereye ne eklediğini ve bunun neden normal bir çökmeden daha zor hata ayıklandığını açıklayın.

Kabul Kriterleri

- ☐ `java TypedCache.java` ile çalışıyor
- ☐ `Cache<K, V>` her iki parametrede jenerik ve en eski girdiyi çıkarıyor
- ☐ Aynı `Cache` sınıfı iki ilgisiz tip argümanı çiftiyle kullanılıyor
- ☐ Bir yorum üç tip silme kısıtını ve kullanılan çözümü belgeliyor
- ☐ `largest` bir sınır bildiriyor, neden gerekli olduğunu söyleyen yorumla
- ☐ `average` ve `drainInto` doğru joker karakterleri kullanıyor
- ☐ İkisi de iki farklı eleman tipine karşı kanıtlanıyor
- ☐ Açıklama her joker karakterin neyi yasakladığını isimlendiriyor
- ☐ Yığın kirlenmesi yeniden üretiliyor ve `ClassCastException` yakalanıyor
- ☐ Bir yorum kuralı bozan satır ile fırlatan satırı farklı olarak tanımlıyor

İleri Seviye

`Cache` sınıfını, en son eklenen yerine **en az kullanılan** girdiyi çıkaracak şekilde değiştirin, böylece bir `get` çağrısı kullanım sayılsın.

`LinkedHashMap` bunu sizin için tek bir kurucu argümanı ve tek bir geçersiz kılınan metotla yapabilir. Bulun, kullanın ve aksi hâlde elle yazmanız gerekecekleri bir yorumda anlatın.

İpucu, üçüncü bölüm fırlatmıyorsa

Eleman 1'i `var` ya da `Object` içine değil bir `String` değişkene okuduğunuzdan emin olun. Dönüşüm, **okuma yerindeki bildirilen tip** yüzünden eklenir. Onu `Object` olarak okursanız dönüşüm olmaz, dolayısıyla başarısızlık da olmaz ve kirlenmiş liste biri onu düzgün okuyana kadar sessizce orada oturur.

Modül 12: Koleksiyonlar

Dört biçim, her birinin birkaç uygulaması. Doğru seçmek becerinin çoğu, kalanı da hangi üç dört tuzağın var olduğunu bilmek.

O tuzaklardan ikisi, zaten iki kez karşılaştığınız aynı kural. Modül 06 bir `HashSet` içinde nesne kaybetti; Modül 08 record'daki değiştirilebilir bileşen konusunda uyardı. İkisi de bu modülün kuralıydı, farklı açılardan görülmüş.

Neler Öğreneceksiniz

- `List`, `Set`, `Map`, `Deque` ve hangi uygulamaya uzanacağınız
- Sıralı koleksiyonlar ve kapattıkları otuz yıllık boşluk
- `List.of` ve arkadaşları, insanları şaşırtan dört davranışla
- Yineleme sırasında silmenin neden fırlattığı ve dört doğru alternatif
- Anahtarlar hakkındaki kural, nihayet açıkça

Başka Bir Dilden Geliyorsanız

	Python	Java
Sıralı dizi	<code>list</code>	<code>ArrayList</code>
Benzersiz	<code>set</code>	<code>HashSet</code> , sıra için <code>LinkedHashSet</code>
Anahtardan değere	<code>dict</code> , ekleme sıralı	<code>HashMap</code> sırasız, <code>LinkedHashMap</code> sıralı
Sıralı	istendiğinde <code>sorted()</code>	<code>TreeMap</code> , <code>TreeSet</code> sıralı tutar
Değişmez sabit	<code>tuple</code> , <code>frozenset</code>	<code>List.of</code> , <code>Set.of</code> , <code>Map.of</code>
Yığın / kuyruk	<code>list</code> , <code>deque</code>	<code>ArrayDeque</code>

Python geliştiricilerini yakalayan satır: `HashMap` ekleme sırasını korumaz. Python sözlükleri 3.7'den beri koruyor. Sıra ihtiyacınız varsa `LinkedHashMap` kullanarak bunu söyleyin.

Ders

Seçmek

`ChoosingACollection.java` dosyasından:

İhtiyaç	Kullanın
indeks ya da tekrar	ArrayList
benzersizlik, sıra önemsiz	HashSet
benzersizlik, kararlı çıktı	LinkedHashSet
sıralı	TreeSet / TreeMap
anahtardan değere	HashMap ya da LinkedHashMap
yiğın ya da kuyruk	ArrayDeque

ArrayList ile LinkedList karşılaştırması: ArrayList kullanın. Bağlı liste *bilinen bir konumda* $O(1)$ ekleme verir, ama o konumu bulmak $O(n)$ 'dir ve her eleman fazladan bir nesne artı iki işaretçiye mal olur. Ardışık bellek hızlıdır, işaretçi takibi değil, dolayısıyla büyük O tablosu kaybetmesi gerektiğini söylediği yerlerde bile ArrayList pratikte kazanır.

java.util.Stack asla kullanmayın. Vector sınıfını genişletir, 1998'de terk edilmiş bir iş parçacığı modeli için her metodu senkronize eder ve alttan üste yineler, ki bu bir yiğın için terstir. ArrayDeque hem yiğın hem kuyruk için cevaptır.

Bir HashSet yineleme sırası bir söz değildir. Testlerinizde kararlı görünmesi hiçbir şey ifade etmez; JDK sürümleri arasında değişebilir.

Hemen öğrenmeye değer üç Map metodu:

```
map.getOrDefault(key, 0)
map.computeIfAbsent(key, k -> new ArrayList<>()).add(value) // map of lists
map.merge(word, 1, Integer::sum) // counting
```

Sıralı koleksiyonlar

Java 21'de, 1998'den beri açık duran bir boşluğu kapatmak için eklendi. İlk elemanı almak tipe göre üç farklı ifadeydi ve bir LinkedHashSet kümesinin **son** elemanını almak tamamını yinelemek demekti.

[SequencedCollections.java](#) dosyasından:

```
getFirst: a
getLast: c
reversed: [c, b, a]
after add, the view shows: [d, c, b, a]
```

`reversed()` bir **görünümdür**, kopya değil. Oluşturması bedavadır ve özgün üzerindeki sonraki değişiklikleri yansıtır.

SequencedMap ayrıca `firstEntry`, `lastEntry`, `putFirst` ve `putLast` ekler:

```
firstEntry: x=1
putFirst: {w=0, x=1, y=2, z=3}
```

Arayüz	Uygulayanlar
SequencedCollection	List , Deque , LinkedHashSet , SortedSet
SequencedSet	LinkedHashSet , TreeSet
SequencedMap	LinkedHashMap , TreeMap

HashSet ve HashMap bilerek sıralı **değildir**. Tanımlı bir sıraları yoktur, dolayısıyla derleyici artık soruyu, yinelemenize ve umut etmenize izin vermek yerine reddeder.

Değişmez fabrikalar

`ImmutableFactories.java` dosyasından dört sürpriz, hepsi bilinçli:

1. **Gerçekten değişmez.** Değiştiren her metot fırlatır. "Gelenek olarak değiştirilemez" değil.
2. **null reddedilir.** `List.of("a", null)` bir `NullPointerException` fırlatır. Eski kodu `List.of` kullanacak şekilde dönüştürmek, var olduğunu bilmediğiniz null değerleri ortaya çıkarabilir.
3. **Tekrarlar hatadır.** `Set.of(1, 1)` çağrısı `duplicate element: 1` fırlatır, `new HashSet<> (List.of(1, 1))` ise sessizce tek eleman verirdi. Bir sabit listede tekrar neredeyse her zaman yazım hatasıdır.
4. **Set.of yineleme sırası belirsizdir** ve bilerek değişir. Sıraya ihtiyacınız varsa `List.of` ya da `LinkedHashSet` kullanın.

Birbirine benzeyen ama olmayan üç şey:

	add	set
<code>List.of</code>	fırlatır	fırlatır
<code>Arrays.asList</code>	fırlatır	çalışır
<code>Collections.unmodifiableList</code>	fırlatır	fırlatır

`Arrays.asList` **sabit boyutludur, değişmez değil**, ve arkasındaki diziye yazar.

`Collections.unmodifiableList` değiştirilebilir bir listenin **görünümüdür**:

```
after mutating the source:
unmodifiable VIEW sees it: [x, y]
List.copyOf does not:      [x]
```

Genellikle istediğiniz `List.copyOf` tur, Modül 05 ve 08'in ona uzanmasının sebebi de bu.

Yineleme sırasında değiştirme

`MutationDuringIteration.java` fırlatır:

```
Exception in thread "main" java.util.ConcurrentModificationException
at java.base/java.util.ArrayList$Itr.checkForComodification
```

"Concurrent" yanıltıcı. Tek bir iş parçacığı var. Koleksiyonun başka bir iş parçacığından değil, devam eden bir yineleme ile eş zamanlı olarak değiştiği anlamına gelir.

`ArrayList` her yapısal değişiklikte artan bir `modCount` tutar. Yineleyici onu kaydeder ve her adımdan önce denetler, eleman atlamak ya da sonunu aşmak yerine hızlı başarısız olur. Belgelerde **en iyi çaba** olarak geçer, dolayısıyla asla istisnanın fırlatılmasına dayanan kod yazmayın.

Dört doğru alternatif:

```
numbers.removeIf(n -> n % 2 == 0);           // 1. usually this

var it = numbers.iterator();                  // 2. when a predicate is not enough
while (it.hasNext()) {
    if (it.next() % 2 == 0) it.remove();
}

// 3. collect, then remove after
// 4. build a new collection (Module 14)
```

Anahtarlar hakkındaki kural

`KeysMustNotChange.java` değiştirilebilir bir nesneyi haritaya koyup yeniden adlandırıyor:

```
after renaming the key object:
get(key):      null
containsKey:   false
size:          1
but iterating still finds it: [MutableKey(invoices)]
after remove(key), size is still: 1
```

Girdi, **eski** hash kodunun kovalarında duruyor. Arama yenisini hesaplıyor, farklı bir kovaya bakıyor ve hiçbir şey bulamıyor. Harita onu hâlâ sayıyor ve yineleme hâlâ veriyor, dolayısıyla veri kaybolmadı. Yalnızca aramayla ulaşılamaz, ki bir haritanın var olma sebebi tek şey budur. Ayrıca artık anahtarla silinemez.

Harita anahtarları ve küme elemanları, `equals` ve `hashCode` metotlarının kullandığı her alanda değişmez olmalıdır.

Bileşenleri de değişmez olan bir record bunu bedavaya karşılar. Record'lar ile haritaların bu kadar iyi anlaşmasının sebebi çoğunlukla budur, ve bu Modül 08'in uyarısının diğer taraftan tekrarıdır.

Çalıştırın

```
java modules/12-collections/examples/ChoosingACollection.java
java modules/12-collections/examples/SequencedCollections.java
java modules/12-collections/examples/ImmutableFactories.java
java modules/12-collections/examples/KeysMustNotChange.java

# Bilerek başarısız olur.
java modules/12-collections/examples/MutationDuringIteration.java

./scripts/verify-examples.sh modules/12-collections
```

Sık Yapılan Hatalar

`HashMap` ya da `HashSet` yinleme sırasına güvenmek. Tanımlı değildir ve değişir.

`for-each` döngüsü içinde silmek. `removeIf` kullanın.

Değiştirilebilir bir nesneyi anahtar olarak kullanmak. İlgili bir alan değiştiği anda girdi ulaşamaz olur.

`Arrays.asList` sonucunun değişmez olmasını beklemek. Sabit boyutludur ve `set` arkasındaki diziye yazar.

`Collections.unmodifiableList(internalList)` döndürüp güvenli sanmak. Bu bir görünümdür. Özgünü elinde tutan hâlâ çağırının gördüğünü değiştirebilir.

Ekleme $O(1)$ diye `LinkedList` seçmek. Ekleme noktasını bulmak öyle değil ve sabit çarpanlar kötü.

`java.util.Stack` kullanmak. Sebepsiz senkronize, ve ters yönde yineliyor.

Özet Çıkarımlar

- `ArrayList`, `HashMap`, `ArrayDeque` çoğu ihtiyacı karşılar. Başka yere belirtilmiş bir sebeple uzanın.
- Sıranın görülebildiği her yerde `LinkedHashSet` ya da `LinkedHashMap` kullanın, örneğin çıktıda ve testlerde.
- Sıralı koleksiyonlar `getFirst`, `getLast` ve `reversed()` verir, ve `reversed()` bedava bir görünümdür.
- `List.of` null'ı reddeder, küme tekrarlarını reddeder ve gerçekten değişmezdir. `Arrays.asList` ve `unmodifiableList` ikisi de değil.
- Döngü içinde silme değil `removeIf`.
- Anahtarlar, `equals` ya da `hashCode` metotlarının okuduğu hiçbir alanda değişmemelidir.

Ödev

[homework/README.md](#)

Bir kelime dizini kurun, her adımda doğru koleksiyonu seçin ve bilerek bir harita girdisini ulaşılamaz kılın.
Referans çözüm [solutions/12-collections/](#) altında.

Ödev 12: Bir Kelime Dizini

Küçük bir metin dizini kurun. Her adım bir koleksiyon seçip gerekçelendirmenizi istiyor, çünkü bu modülün asıl öğrettiği beceri doğru seçmek.

Metin

```
private static final String TEXT = ""  
    the quick brown fox jumps over the lazy dog  
    the dog barks and the fox runs  
    a quick fox is a happy fox"";
```

Birinci Bölüm: Sayma

`WordIndex.java` dosyasını klasik biçimde oluşturun.

Her kelimenin kaç kez geçtiğini sayın, sonra en sık geçen üçünü yazdırın.

Gereksinimler:

- Sayma için `merge` kullanın. `get`, null kontrolü, `put` dizisi **yazmayın**.
- Sayımlar **öngörülebilir** bir sırada yazdırılmalı, ve bir yorum hangi harita tipini seçtiğinizi ve `HashMap` seçiminin neden yanlış olacağını söylemeli.
- `the` ve `fox` dörder kez geçiyor. İlk üçünüz çalıştırmalar arasında **belirlenimci** olmalı, dolayısıyla eşitliği nasıl bozacağınıza karar verin ve bir yorumda söyleyin.

İkinci Bölüm: Dizinleme

Her baş harften, o harfle başlayan kelimelerin kümesine bir dizin kurun.

Gereksinimler:

- Baş harfler ayrı bir sıralama adımı olmadan **alfabetik** çıkmalı
- Her kelime baş harf başına **bir kez** görünmeli, ve kelimeler **sıralı** olmalı
- Null kontrolü değil `computeIfAbsent` kullanın

Sonra sıralı koleksiyon metotlarıyla ilk girdiyi, son girdiyi ve anahtarları ters sırada yazdırın.

Son olarak benzersiz kelimeler kümesini `HashSet`, `LinkedHashSet` ve `TreeSet` ile **üç ayrı biçimde** kurun, üçünü de yazdırın ve her birinin ne söz verdiğini bir yorumda yazın. Bunlardan biri hiçbir söz vermiyor.

Üçüncü Bölüm: Bir Girdiyi Kaybedin

Yeniden adlandırılabilen tek bir alanı, doğru `equals` ve `hashCode` metotları ve bir `toString` metodu olan bir `MutableTag` sınıfı yazın.

Birini bir `HashMap` içine **anahtar** olarak koyun, sonra yeniden adlandırın. Şu sırayla yazdırın:

1. Yeniden adlandırmadan önce `get`, `containsKey` ve `size`
2. Sonrasında aynı üçü
3. Yineleyerek anahtar kümesi
4. `remove(tag)` çağrısından sonra `size`

Açıklayın

Kendi kelimelerinizle şunları kapsayan bir yorum yazın:

- Girdi açıkça hâlâ oradayken `get` neden null döndürüyor
- Yineleme neden buluyor ama arama neden bulmuyor
- `remove` neden onu da silemiyor
- Ondak kurtulmanın geriye kalan tek yolu ne

Sonra aynısını anahtar olarak bir record ile yapın ve yeni kurulmuş eşit bir anahtarın değeri hâlâ bulunduğunu gösterin.

Kabul Kriterleri

- ☐ `java WordIndex.java` ile çalışıyor
- ☐ Sayma `merge` kullanıyor
- ☐ Bir yorum harita tipini `HashMap` yerine gerekçelendiriyor
- ☐ İlk üç çıktısı belirlenimci, eşitlik bozma açıklanmış
- ☐ Baş harf dizini alfabetik, değerleri sıralı ve tekrarsız
- ☐ `computeIfAbsent` kullanılıyor
- ☐ `firstEntry`, `lastEntry` ve `reversed` hepsi görünüyor
- ☐ Üç küme tipi de, her birinin ne söz verdiğine dair bir yorumla yazdırılıyor
- ☐ Değiştirilebilir anahtar gösterimi `get` null, `containsKey` false, `size` 1 gösteriyor
- ☐ Yineleme kaybolan girdiyi hâlâ veriyor, ve `remove` onu silemiyor
- ☐ Açıklama yukarıdaki dört sorunun hepsini kapsıyor

İleri Seviye

Kaybolan girdiden kurtulun. Haritayı yeniden kurmadan, ulaşılamaz anahtarı bir `Iterator` kullanarak silin.

Sonra `Iterator.remove` metodunun `Map.remove` metodunun yapamadığını neden yapabildiğini bir yorumda yazın ve bunu Modül 12'nin `ConcurrentModificationException` hakkında söylediğine, yani aynı mekanizmanın öteki taraftan görünümü olduğuna bağlayın.

İpucu, ilk üçünüz değişip duruyorsa

Yalnızca sayıya göre sıralamak eşit sayıları, o an hangi sıradaysalar o sırada bırakır. O sıra bir hash tablosundan gelebilir, yani kararlı değildir. Eşitliklerin her seferinde aynı çözülmesi için ikinci bir karşılaştırma ekleyin.

Modül 13: Lambda'lar

Bir lambda, tam olarak bir soyut metodu olan bir arayüzün, sınıf töreni olmadan yazılmış uygulamasıdır.

Bu tanımlı aklınızda tutmaya değer. Lambda, Java'da yeni bir değer türü değildir. Hâlâ bir arayüz uygulayan bir nesnedir. Modül 07, tek soyut metotlu bir arayüzün sınıf yazmadan uygulanabileceğini söylemişti; bu modül o cümlelerin uygulamaya dökülmüş hâli.

Neler Öğreneceksiniz

- Lambda sözdizimi ve derleyicinin sizin için neyi çıkardığı
- Dört metot referansı biçimi, birebir aynı görünen ikisi dahil
- Yakalama ve yakalanan yerelin neden etkin final olması gerektiği
- Standart fonksiyonel arayüzler ve nasıl birleştikleri
- `java.util.function` paketinde neden bu kadar çok neredeyse aynı tip olduğu

Başka Bir Dilden Geliyorsanız

	Python / JS	Java
Anonim fonksiyon	<code>lambda</code> , <code>=></code>	<code>-></code>
Çok ifadeli gövde	JS evet, Python hayır	evet, süslü parantezle
Yakalanan değişken değişebilir	evet	hayır , etkin final olmalı
Lambda'nın tipi	fonksiyon nesnesi	arayüz uygulaması
Fonksiyon tipi	birinci sınıf	yok; bir arayüz kullanın

Önemli olan iki satır: Java'nın **fonksiyon tipi yoktur**, dolayısıyla her lambda bir hedef arayüze ihtiyaç duyar, ve yakalanan bir yerel **yeniden atanamaz**.

Ders

Aynı şey, üç biçimde

`Lambdas.java` dosyasından:

```
Greeter named = new PoliteGreeter();           // a named class
Greeter anonymous = new Greeter() {             // an anonymous class
    @Override public String greet(String n) { return "Good day, " + n; }
};
Greeter lambda = name -> "Good day, " + name;    // a lambda
```

Üçü de `Greeter` uygulayan bir nesne üretir. Lambda; arayüz adını, metod adını, niteleyicileri ve parametre tipini atlar, çünkü derleyici hepsini hedef tipten çıkarabilir.

Sözdizimi biçimleri:

```
() -> doSomething()           // no parameters, parentheses required
name -> "hello " + name        // one parameter, parentheses optional
(String name) -> ...           // explicit type when inference needs help
name -> { ...; return x; }      // block body needs braces and return
(a, b) -> a + b                // two parameters always need parentheses
```

Fonksiyonel arayüzün tam olarak bir soyut metodu vardır. Varsayılan ve statik metodlar sayılmaz, `Comparator` sınıfının onlarca metodu olup yine de lambda hedefi olabilmesinin sebebi bu. Kendi arayüzlerinize `@FunctionalInterface` yazın: derleyicinin kuralı zorlamasını sağlar, böylece sonradan ikinci bir soyut metod eklemek, onu kullanan her lambda'da değil arayüzün kendisinde başarısız olur.

Metot referansları

Dört biçim, `MethodReferences.java` dosyasından:

Biçim	Örnek	Eşdeğer lambda
statik	<code>Integer::parseInt</code>	<code>s -> Integer.parseInt(s)</code>
kurucu	<code>ArrayList::new</code>	<code>() -> new ArrayList<>()</code>
bağlı örnek	<code>prefix::concat</code>	<code>s -> prefix.concat(s)</code>
bağımsız örnek	<code>String::toUpperCase</code>	<code>s -> s.toUpperCase()</code>

Bağımsız biçim insanların yanlış okuduğu biçimdir. Lambda'nın ilk argümanı **alıcı** olur, argüman değil.

`toUpperCase` hiç parametre almamasına rağmen `Function<String, String> upper = String::toUpperCase` yazımının çalışmasının sebebi budur.

Üç ve dördüncü biçimler aynı görünür ama değildir:

```
bound, arg "fix":           pre-fix
bound, arg "amble":        pre-amble
unbound, ("pre-", "fix"):   pre-fix
unbound, ("post-", "fix"):  post-fix
```

Bağlı olan her zaman `prefix` üzerine ekler. Bağımsız olan alıcısını ilk argümandan alır, dolayısıyla üzerine eklediği şey çağrıdan çağrıya değişir. `::` işaretinin solunda ne olduğuna bakarak ayırt edin: değişken bağlı, tip adı bağımsız ya da statik demektir.

Ne zaman kullanılmaz. `words.forEach(System.out::println)` daha nettir. Ama başka bir şey olduğu anda lambda daha iyi okunur. Bir metod referansı sığsın diye kodu eğip bükmek genellikle onu kötüleştirir.

Yakalama

Yakalanan bir yerel **final** ya da **etkin final** olmalıdır.

Etkin final, ilklendirmeden sonra ona hiç atama yapmadığınız anlamına gelir. Anahtar kelimeyi yazmazsınız; derleyici davranışı denetler. `NotEffectivelyFinal.java`:

```
error: local variables referenced from a lambda expression must be final or effectively final
```

Kuralın var olma sebebi: bir lambda, onu oluşturan metottan daha uzun yaşayabilir. Çalıştığında o yığın çerçevesi çoktan gitmiş olabilir, dolayısıyla lambda değişkene referans değil **değerin bir kopyasını** tutar. Yeniden atamaya izin verilseydi iki sorunun iyi cevabı olmazdı: zaten oluşturulmuş bir lambda yeni değeri görür mü, ve iki iş parçacığı aynı anda atarsa ne olur?

Kural değişkenle ilgilidir, nesneyle değil. Bu sorun değil:

```
List<String> items = new ArrayList<>();
Runnable r = () -> items.add("x");           // the variable never changes
```

İnsanların kuralı aşmak için kullandığı boşluk da bu:

```
int[] counter = {0};
list.forEach(x -> counter[0]++);           // compiles, and is a smell
```

Derlenir çünkü değişken hiç değişmez. Kötü kokar çünkü lambda mutasyon için kullanılıyor, ki Modül 14'ün var olma sebebi tam olarak onu değiştirmek.

Alanlarda böyle bir kural yoktur. Bir lambda `this` yakalar ve alanı onun üzerinden okur, dolayısıyla değiştirilebilir bir alan çalışır. Bu ayrıca saklanan bir lambda'nın tüm kuşatan nesneyi hayatta tuttuğu anlamına gelir, ki bu uzun ömürlü dinleyici listelerinde önemlidir.

Lambda ile anonim sınıf arasında `this` farklıdır:

```
inside a lambda, this is:           Capture
inside an anonymous class, this is: Capture$1
```

`$1` gerçek, ayrı bir sınıftır. Lambda böyle bir şey oluşturmaz, dolayısıyla içindeki `this` kuşatan nesneyi ifade eder. Bu bir davranış farkıdır, stil tercihi değil.

Standart arayüzler

Arayüz	Biçim	Metot
<code>Function<T,R></code>	T girer, R çıkar	<code>apply</code>
<code>Predicate<T></code>	T girer, boolean çıkar	<code>test</code>
<code>Supplier<T></code>	hiçbir şey girmez, T çıkar	<code>get</code>
<code>Consumer<T></code>	T girer, hiçbir şey çıkmaz	<code>accept</code>

Artı `BiFunction`, `BiPredicate`, `BiConsumer`, `UnaryOperator<T>` (bir `Function<T,T>`) ve `BinaryOperator<T>`.

Birleştirme, `StandardInterfaces.java` dosyasından:

```
twice.andThen(plusOne) on 5: 11
twice.compose(plusOne) on 5: 12
```

`andThen` "önce beni, sonra argümanı yap" demektir. `compose` "önce argümanı, sonra beni yap" demektir. İşlemler değişmeli olmadığında bunları ters çevirmek klasik bir hatadır.

`Predicate`'ler `and`, `or` ve `negate` ile birleşir.

İlkel tip özelleştirmeleri var çünkü `Function<Integer,Integer>` her değeri kutular. `IntPredicate`, `ToIntFunction`, `IntUnaryOperator` ve `long` ile `double` eşdeğerleri bundan kaçınır. Bu, Modül 02'nin kutulama bedelinin API tasarımında görünmesidir, ve `java.util.function` paketinin bu kadar büyük olmasının sebebi.

Çalıştırın

```
java modules/13-lambdas/examples/Lambdas.java
java modules/13-lambdas/examples/MethodReferences.java
java modules/13-lambdas/examples/Capture.java
java modules/13-lambdas/examples/StandardInterfaces.java

# Bilerek başarısız olur.
java modules/13-lambdas/examples/NotEffectivelyFinal.java

./scripts/verify-examples.sh modules/13-lambdas
```

Sık Yapılan Hatalar

Yakalanan bir yereli yeniden atamaya çalışmak. Önce nihai değeri hesaplayın ya da ikinci bir değişken kullanın.

Biriktirmek için `int[] box = {0}` **numarası**. Derlenir. Ayrıca lambda'nın bir indirgeme olmak istediğini söyler, ki Modül 14 onu anlatıyor.

andThen ile **compose** **kariřtırmak**. Sesli okuyun: "and then" ikinci, "compose" etrafını sarar.

Karřılařtırıcı olarak `a.length() - b.length()` **yazmak**. Büyük deęerlerde taşar.

`Comparator.comparingInt` kullanın.

Lambda içindeki `this` **deęerinin lambda olmasını beklemek**. Atıfta bulunacak bir lambda nesnesi yoktur.

Standart biri uyarken kendi fonksiyonel arayüzünüzü bildirmek. `Function<String,Integer>` bilen bir okuyucu, gidip sizin `StringScorer` arayüzünüze bakmak zorunda kalır.

Lambda'ları `this` etkisini düşünmeden uzun ömürlü koleksiyonlarda saklamak. Lambda kuřatan nesneyi tutar, dolayısıyla o nesne toplanamaz.

Özet Çıkarımlar

- **Lambda, tek soyut metotlu bir arayüzü uygular**. Java'nın fonksiyon tipi yoktur.
- `@FunctionalInterface` **derleyicinin kuralı zorlamasını sağlar**, sahip olduęunuz arayüzlerde.
- **Dört metot referansı biçimi**, ve baęsız olan ilk argümanı alıcıya çevirir.
- **Yakalanan yereller etkin final olmalıdır**, çünkü lambda deęeri kopyalar ve çerçeveden uzun yaşayabilir.
- **Kural deęiřkenle ilgilidir, nesneyle deęil**, dolayısıyla yakalanan bir koleksiyonu deęiřtirmek yasaldır.
- `andThen` ve `compose` **ters sırada çalışır**.
- **İlkel tip özelleřtirmeleri kutulamadan kaçınmak için vardır**.

Ödev

[homework/README.md](#)

Birleřtirilmiř predicate ve fonksiyonlardan küçük bir kural motoru kurun, sonra bilerek yakalama kuralına çarpın. Referans çözüm [solutions/13-lambdas/](#) altında.

Ödev 13: Bir Kural Motoru

Birleştirilmiş predicate'lerden uygunluk kuralları kurun, dört metot referansı biçimini de kullanın, sonra bilerek yakalama kuralına çarpın.

Veri

```
record Applicant(String name, int age, int creditScore, boolean employed) { }

List.of(
    new Applicant("ada", 36, 780, true),
    new Applicant("grace", 45, 610, true),
    new Applicant("alan", 17, 800, false),
    new Applicant("edsger", 52, 550, true),
    new Applicant("barbara", 29, 720, false));
```

Birinci Bölüm: Birleştirilmiş Predicate'ler

`RulesEngine.java` dosyasını klasik biçimde oluşturun.

Ayrı ayrı adlandırılmış üç predicate yazın:

Ad	Kural
<code>isAdult</code>	yaş 18 ve üzeri
<code>hasGoodCredit</code>	kredi puanı 700 ve üzeri
<code>isEmployed</code>	employed true

Sonra bunları **birleştirerek**, yeni bir lambda yazmadan iki bileşik kural kurun:

- `eligible` — yetişkin, iyi kredi ve çalışıyor
- `needsReview` — yetişkin, ama kredi 700 altında

Her başvuran için üç sonucu ve ayrıca reşit olup olmadığını gösteren bir satır yazdırın; sonuncusu için `negate()` kullanın.

Üç adlandırılmış predicate'in neden tek bir büyük lambda'yı yendiğini bir yorumda yazın. Cevabınız tekrar kullanımdan bahsetmeli: iki bileşik kuraldan biri diğeriyle bir bileşen paylaşıyor.

İkinci Bölüm: Dört Metot Referansı Biçimi

Her biçimi en az bir kez kullanın, her birinde hangisi olduğunu isimlendiren bir yorumla:

1. Bir **statik** metot referansı

2. Bir **kurucu** referansı
3. Bir **bağlı örnek** referansı
4. Bir **bağısız örnek** referansı

Sonra Zor Kısım Açıklayın

Üçüncü ve dördüncü biçimlerin ikisi de `Something::method` olarak yazılır. Kendi kelimelerinizle şunları açıklayan bir yorum yazın:

- Her durumda `::` işaretinin solunda ne durduğu ve bunun neden ayırt edici işaret olduğu
- Hangisinin alıcısını bir kez seçtiği, hangisinin her çağrıda seçtiği
- Argüman sayısı açısından gözlemlenebilir sonuç.** Tek argümanlı bir metoda bağlı referans, kaç parametrelili bir fonksiyonel arayüz ister? Aynı metoda bağısız referans için ne dersiniz? İkisini de gösterin.

Son olarak `andThen` ile bir `Applicant` nesnesini A, B veya C kredi bandına çeviren bir boru hattı kurun ve herkes için yazdırın.

Üçüncü Bölüm: Yakalama Kuralı

Önce **derlenmeyen** sürümü yazın: bir `forEach` içinde artırılan yerel bir `int approved = 0;`. Hatayı doğrulayın, sonra kuralın neden var olduğunun açıklamasıyla birlikte bir yoruma koyun.

Sonra aynı sayımı çalışan **üç** biçimde yazın:

1. `int[] box = {0}` numarası
2. Sıradan bir `for` döngüsü
3. `filter` ve `count` kullanan bir stream

Soru

Düz `int` derlenmezken dizi numarasının neden derlendiğini ve yine de neden kötü bir cevap olduğunu açıklayan bir yorum yazın. Açıklamanız **değişken** ile **gösterdiği nesne** arasında ayırım yapmalı, çünkü kuralın tamamı o ayırım.

Bir lambda içinde bir `List` yakalayıp değiştirerek bunun yasal olduğunu gösterip bitirin, sonra yine de `.toList()` kullanmanın neden daha iyi olduğunu söyleyin.

Kabul Kriterleri

- ☐ `java RulesEngine.java` ile çalışıyor
- ☐ Ayrı ayrı adlandırılmış üç predicate var
- ☐ `eligible` ve `needsReview` yeniden yazılarak değil birleştirilerek kuruldu

- ☐ `negate()` kullanılıyor
- ☐ Bir yorum birleştirmeyi, paylaşılan bileşenden bahsederek gerekçelendiriyor
- ☐ Dört metod referansı biçimi de görünüyor, her biri etiketli
- ☐ Bağlı ile bağısız açıklaması argüman sayısı sonucunu kapsıyor ve her iki imzayı gösteriyor
- ☐ Bir `andThen` boru hattı kredi bantları üretiyor
- ☐ Derlenmeyen yakalama denemesi gerçek hatasıyla alıntılanmış
- ☐ Sayım üç çalışan biçimde üretiliyor, hepsi aynı sonucu veriyor
- ☐ Açıklama değişkeni nesneden ayırıyor

İleri Seviye

Bir `Predicate<Applicant>` fabrikası yazın: minimum bir puan alan ve ona karşı test eden bir predicate döndüren bir metod.

Sonra o metodun neyi yakaladığını, yakalanan parametrenin etkin final olup olmadığını, ve bu modülün yığın çerçeveleri hakkında söyledikleri göz önüne alındığında bir metod parametresinin neden yakalanmasına izin verildiğini bir yorumda yazın.

İpucu, `compose` tip denetiminden geçmiyorsa

`a.andThen(b)` önce `a` çalıştırır. `a.compose(b)` önce `b` çalıştırır. `a` bir `Applicant` alıp `b` bir `int` alırken yalnızca bir sıralama tutar. Derleyici reddediyorsa ters yazmışsınız, ve o ret size iyilik yapıyor.

Modül 14: Stream'ler

Bir stream (akış) bir boru hattıdır, koleksiyon değil. Hiç veri tutmaz ve bir kez tüketilir. Stream'lerle ilgili şaşırtıcı olan neredeyse her şey bu iki olgudan çıkar.

Modül 13 parçaları kurdu. Bir stream boru hattındaki her lambda o modüldeki arayüzlerden biri, dolayısıyla karşılığını almaya burada başlıyorlar.

Neler Öğreneceksiniz

- Boru hattı anatomisi ve iddia edilerek değil gösterilerek tembellik
- Elemanların neden aşama aşama değil teker teker aktığı
- Collector'lar, API'yi haklı çıkaran `groupingBy` ile birlikte
- Özel ara işlemlerin ait olduğu boşluğu dolduran gatherer'lar
- Paralel stream'ler, ne zaman yardım ettikleri ve onları tehlikeli kılan tuzak

Başka Bir Dilden Geliyorsanız

	Python	Java
Dönüştürme	comprehension, <code>map</code>	<code>.map(...)</code>
Filtreleme	comprehension <code>if</code> , <code>filter</code>	<code>.filter(...)</code>
Tembel	generator'lar	stream'ler, her zaman
Tekrar kullanım	liste iki kez yinelenebilir	bir stream yinelenemez
Gruplama	<code>itertools.groupby</code> sıralama ister	<code>Collectors.groupingBy</code> istemez
Paralel	<code>multiprocessing</code>	<code>.parallel()</code> , ve önce düşünün

Isıran satır tekrar kullanım. Python liste comprehension'ı size bir liste verir. Java stream'i, bir kez kullanıldığında buharlaşan bir boru hattı verir.

Ders

Anatomi

```
words.stream() // SOURCE
    .filter(w -> w.length() > 4) // INTERMEDIATE, lazy
    .map(String::toUpperCase) // INTERMEDIATE, lazy
    .toList(); // TERMINAL, runs everything
```

Ara işlemler bir stream döndürür ve hiçbir şey yapmaz. Boru hattını gerçekte çalıştıran terminal işlemdir.

Tembellik, gösterilerek

`StreamBasics.java` dosyasından, içinde `peek` olan bir boru hattı kurmak hiçbir şey yazdırmaz:

```
pipeline built. notice nothing was peeked.  
now adding a terminal operation:  
peeked at banana  
...
```

Tembellik **kısa devreyi** mümkün kılan şeydir:

```
examined 1  
examined 2  
examined 3  
found Optional[3]
```

Eleman 4 ve 5'e hiç dokunulmadı. Sonsuz bir stream'in çalışmasının sebebi de bu:

```
Stream.iterate(1, n -> n + 1).map(n -> n * n).limit(5).toList()
```

Aşama aşama değil eleman eleman

Yaygın bir yanlış okuma, `filter` metodunun her şey üzerinde çalışıp sonra `map` metodunun her şey üzerinde çalıştığıdır. Öyle değil. Her eleman, bir sonraki başlamadan **boru hattının tamamından** geçer:

```
filter sees a  
map sees a  
forEach sees a  
filter sees b  
...
```

Collector'lar

`CollectingResults.java` dosyasından. Tek başına `groupingBy` API'yi öğrenmeyi haklı çıkarır:

```
STAFF.stream().collect(Collectors.groupingBy(Employee::department))
```

Aşağı akış collector'ı ikinci argümandır, ve API'nin laf kalabalığı olmaktan çıkıp değerli olmaya başladığı yer orasıdır:

```
groupingBy(Employee::department, Collectors.counting())  
groupingBy(Employee::department, Collectors.averagingInt(Employee::salary))  
groupingBy(Employee::department, TreeMap::new, Collectors.counting()) // ordered
```

Harita fabrikası olmadan sırası tanımsız olan bir `HashMap` alırsınız. Modül 12 burada da geçerli.

`partitioningBy` tam olarak iki gruba böler ve bir taraf boş olsa bile her zaman her iki anahtarı da taşır.

`toMap` tekrarlayan anahtarda ezmek yerine fırlatır:

```
duplicate key threw: Duplicate key engineering ...
```

Bu genellikle istediğiniz şeydir. Aksini söylemek için bir birleştirme fonksiyonu verin.

Stream üzerindeki `.toList()` değiştirilemez bir liste döndürür ve modern biçimdir.

`Collectors.toList()` tipi belirtilmemiş, değiştirilebilir bir liste döndürür. Birincisini tercih edin.

Gatherer'lar

JDK 24'te kesinleşti, dolayısıyla burada bayrak gerekmiyor. Gerçek bir boşluğu dolduruyorlar: bir `Collector` olarak özel bir **terminal** işlem her zaman yazabiliyordunuz, ama özel bir **ara** işlem asla. Elemanlar arasında durum tutan her şey stream'den çıkmak demekti.

`UsingGatherers.java` dosyasından:

Gatherer	1,2,3,4,5 üzerinde sonuç
<code>windowFixed(2)</code>	<code>[[1, 2], [3, 4], [5]]</code>
<code>windowSliding(2)</code>	<code>[[1, 2], [2, 3], [3, 4]]</code>
<code>scan(() -> 0, Integer::sum)</code>	<code>[1, 3, 6, 10]</code>

`windowFixed` son kısa pencereyi atmak yerine korur, ki toplu iş için istediğiniz budur: hiçbir kayıt kaybolmaz. `scan`, her ara değeri yayan `reduce` işlemidir, ki yürüyen bir bakiyenin ihtiyacı olan şey budur.

`mapConcurrent(n, fn)` sınırlı eşzamanlılıkla eşler **ve sırayı korur**, her birini bir sanal iş parçacığında çalıştırarak. Bu, IO biçimli işler için `.parallel()` seçeneğinden daha iyi bir varsayılandır, ve sonraki bölüm nedenini açıklıyor.

Ne zaman birine uzanmalı: elemanlar arasında durum, ya da tükettiğinizden farklı sayıda eleman yaymak. Bir girer bir çıkar, hafızasız ise o sadece `map`.

Paralel stream'ler

`ParallelStreams.java` dosyasından. `.parallel()` eklemek tek bir kelime, ve neredeyse hiçbir zaman doğru karar değil.

Aynı makinede üç kez çalıştırılan tuzak:

```
the list now holds: 23839
the list now holds: 22694
the list now holds: 84097
```

100.000 eleman paralel olarak bir `ArrayList` içine eklendi. Her çalıştırma farklı bir sayı kaybetti, ve dördüncü bir çalıştırma bunun yerine fırlatabilir, bir iş parçacığı diziyi yeniden boyutlandırırken bir diğeri

yazmanın ortasındaysa.

Tutarlı biçimde yanlış değil, ki bu tutarlı biçimde yanlış olmaktan daha kötü. Küçük girdide ya da yüksüz bir makinede genellikle çalışır. Bunun üretime ulaşma biçimi budur.

Çözüm kilit değil, `collect` :

```
IntStream.range(0, 100_000).parallel().boxed().toList(); // 100000, always
```

Her iş parçacığı kendi kabına biriktirir ve sonuçlar sonunda birleştirilir, dolayısıyla yazılırken hiçbir şey paylaşılmaz. `add` metodunu senkronize etmek de doğru olurdu ve ardışık sürümden *daha yavaş* olurdu, çünkü her iş parçacığı tek bir kilitte sıraya girerdi.

Paralelde `forEach` sırayı korumaz; `forEachOrdered` korur.

<code>.parallel()</code> düşünün	Kaçının
büyük N, eleman başına gerçekten pahalı iş	küçük N: fork/join maliyeti baskın gelir
kaynak ucuza bölünüyor: dizi, <code>ArrayList</code> , <code>IntStream.range</code>	<code>LinkedList</code> ya da yineleyici kaynaklar
paylaşılan durum yok, sıra gereksinimi yok	IO yapan her şey

IO durumu vurgulanmayı hak ediyor. Ortak havuz CPU çekirdeklerine göre boyutlandırılmıştır ve onu bloke etmek JVM'deki diğer her paralel stream'i aç bırakır. Onun yerine `Gatherers.mapConcurrent` ya da Modül 19'un sanal iş parçacıklarını kullanın.

Dürüst varsayılan: bir profilleyici bu boru hattını darboğaz olarak isimlendirene kadar `.parallel()` yazmayın.

Tek kullanım

`StreamsAreSingleUse.java` :

```
IllegalStateException: stream has already been operated upon or closed
```

Bir stream tekrar oynatacak kendi elemanlarını tutmaz. Stream'i değil **kaynağı** saklayın, ve bir koleksiyon size istediğiniz kadar stream verir.

Çalıştırın

```
java modules/14-streams/examples/StreamBasics.java
java modules/14-streams/examples/CollectingResults.java
java modules/14-streams/examples/UsingGatherers.java
java modules/14-streams/examples/ParallelStreams.java

# Bilerek başarısız olur.
java modules/14-streams/examples/StreamsAreSingleUse.java

./scripts/verify-examples.sh modules/14-streams
```

Sık Yapılan Hatalar

Bir stream'i alanda ya da değişkende saklayıp iki kez kullanmak. Kaynağı saklayın.

forEach içinden paylaşılan bir koleksiyonu değiştirmek. **collect** kullanın. Paralelde bu bir veri yansıdır; ardışıkta bile toplanmış sürümden daha zor okunur.

Bir şeyi hızlandırmak için .parallel() eklemek. Önce ölçün. Sıklıkla daha yavaş ve ara sıra yanlış.

Paralel stream içinde bloke olmak. Tüm JVM'in paylaştığı bir havuzdan iş parçacığı aldınız.

Gerçek yan etkiler için peek kullanmak. O bir hata ayıklama penceresidir. Uygulamaların, sonuç gerekmediğinde onu atlamasına izin verilmiştir.

toMap metodunun tekrarlarla fırlattığını unutmak. Anahtarlar tekrarlanabildiğinde bir birleştirme fonksiyonu verin.

Yerine geçtiği döngüden daha uzun bir boru hattı. Stream'ler *ne* olduğunu ifade etmek içindir, döngülerin yasak olduğu bir kural değil. Döngü daha netse onu yazın.

Özet Çıkarımlar

- **Bir stream tembel bir boru hattıdır, bir kez tüketilir.** Stream'i değil kaynağı saklayın.
- **Terminal işleme kadar hiçbir şey çalışmaz,** ki kısa devreyi ve sonsuz kaynakları mümkün kılan budur.
- **Elemanlar boru hattının tamamından teker teker akar,** aşama aşama değil.
- **Aşağı akış collector'ı ile groupingBy**, API'deki en kullanışlı şeydir.
- **Gatherer'lar size özel ara işlemler verir,** pencereleme ve yürüyen toplamalar için.
- **Paylaşılan değiştirilebilir durumla .parallel() veriyi öngörülemez biçimde kaybeder.** **collect** kullanın, ve paralelleştirmeden önce ölçün.

Ödev

[homework/README.md](#)

Bir log'u stream boru hattıyla analiz edin, bir gatherer kullanın ve paralel veri yarışını yeniden üretin.
Referans çözüm [solutions/14-streams/](#) altında.

Ödev 14: Log Analizi

Bir log'u stream boru hatlarıyla analiz edin, `map` metodunun gidemeyeceği yere bir gatherer ile uzanın, ve paralel veri yarışını olurken izlemek için yeniden üretin.

Veri

```
List.of(
    "INFO auth 120",
    "ERROR auth 450",
    "INFO billing 80",
    "WARN billing 900",
    "ERROR auth 380",
    "INFO search 60",
    "ERROR billing 1200",
    "INFO auth 95",
    "WARN search 700",
    "INFO billing 110");
```

Her satırı `record Entry(String level, String service, int millis)` içine ayırıştırın.

Birinci Bölüm: Gruplama

`LogAnalysis.java` dosyasını klasik biçimde oluşturun. Şunların hepsini **tek** stream boru hattıyla üretin:

1. Servis başına girdi sayısı
2. Servis başına ortalama yanıt süresi
3. Servis başına görülen farklı seviyeler, sıralı
4. Girdilerin yavaş (500ms üstü) ve hızlı olarak iki gruba ayrılması
5. Servis başına en yavaş süre

Gereksinimler:

- Gruplanmış her sonuç **öngörülebilir** bir anahtar sırasında yazdırılmalı. Hangi harita tipini kullandığınızı ve varsayılanın neden kabul edilemez olduğunu bir yorumda söyleyin.
- En az üçü için ikinci bir geçiş değil, aşağı akış collector'ı kullanın.
- İki gruba ayırma için `partitioningBy` kullanın ve bir boolean üzerindeki `groupingBy` seçeneğinin vermediği neyi garanti ettiğini bir yorumda yazın.
- En yavaş süre için önce düz bir `toMap` deneyin ve ne olduğunu kaydedin, sonra düzeltin.

İkinci Bölüm: Gatherer'lar

Yalnızca süreleri çıkarın, sonra şunları üretin:

1. Üçerli gruplar
2. Ardışık her okuma çifti arasındaki fark
3. Yürüyen toplam

Soru

Bu üçünün **hiçbirinin** neden `map` ile yazılamayacağını kendi kelimelerinizle açıklayan bir yorum yazın. Cevabınız üçünü de ele almalı, ve farklı sebeplerle aynı testte kalıyorlar. `map` metodunun uyduğu kuralı isimlendirin ve her birinin hangi kısmını çiğnediğini söyleyin.

Üçüncü Bölüm: Yarıştırın

Paralel bir stream'den düz bir `ArrayList` içine 50.000 tam sayı ekleyin. Bunu **üç kez** yapın ve her çalıştırmada ortaya çıkan boyutu yazdırın.

Bir try/catch ile koruyun, çünkü eleman kaybetmek tek başarısızlık biçimi değil.

Sonra doğru sayımı iki biçimde üretin: `.toList()` ve `.count()` ile.

Açıklayın

Şunları kapsayan bir yorum yazın:

1. **Mekanizma.** `ArrayList.add` birden fazla adımdır. Adımları isimlendirin ve araya giren iki iş parçacığının birbirine ne yapabileceğini söyleyin.
2. **collect neden bağışık.** Kaplarla farklı olarak ne yapıyor?
3. **Neden synchronized eklemek yanlış çözüm,** doğru olmasına rağmen.

Dördüncü Bölüm: Tembellik ve Tek Kullanım

İçinde `peek` olan ama terminal işlemi olmayan bir boru hattı kurun ve hiçbir şeyin çalışmadığını gösterin.

`findFirst` ekleyin ve peek çıktısından on girdinin hepsini incelemek yerine **erken durduğunu** gösterin.

Sonra aynı stream değişkeninde ikinci bir terminal işlem çağırın, ne olduğunu yakalayın ve mesajı yazdırın. Aynı cevabı kaynaktan doğru biçimde alarak bitirin.

Kabul Kriterleri

- ☐ `java LogAnalysis.java` ile çalışıyor
- ☐ Birinci bölümün beş sonucu da tek boru hattından geliyor
- ☐ Anahtar sırası öngörülebilir, harita tipini gerekçelendiren bir yorumla
- ☐ En az üç aşağı akış collector'ı kullanılıyor

- ☐ Bir yorum `partitioningBy` metodunun neyi garanti ettiğini belirtiyor
- ☐ `toMap` tekrarlayan anahtar hatası tetikleniyor ve sonra düzeltiliyor
- ☐ Üç gatherer işleminin hepsi doğru çıktı üretiyor
- ☐ Yorum `map` metodunun üçünü de neden yapamayacağını açıklıyor
- ☐ Üç paralel çalıştırma boyutlarını yazdırıyor, fırlatmaya karşı korunmuş
- ☐ Doğru sayım `.toList()` ve `.count()` ile gösteriliyor
- ☐ Yarış açıklaması mekanizmayı, `collect` neden güvenli ve kilit neden yanlış çözüm olduğunu kapsıyor
- ☐ peek çıktısı hem tembelliği hem kısa devreyi kanıtlıyor
- ☐ Stream tekrar kullanımı yakalanıyor ve mesajı yazdırılıyor

İleri Seviye

Üçüncü bölümdeki paralel stream'i `Gatherers.mapConcurrent(4, ...)` ile değiştirin.

Sonra ikisini karşılaştıran bir yorum yazın. Hangisi eşzamanlılığını sınırlıyor? Hangisi sırayı koruyor? Bir ağ servisi çağıran iş için hangisini kullanırdınız ve Modül 14 diğerinin orada neden kötü fikir olduğunu söylüyor?

İpucu, farklarınız bir eleman fazla çıkıyorsa

On eleman üzerinde `windowSliding(2)` on değil dokuz pencere verir. On okuma arasında dokuz boşluk vardır. On aldıysanız pencereleme mi yoksa eşleme mi yaptığınızı kontrol edin.

Modül 15: Optional

`Optional` tek bir iş için var: **meşru olarak hiçbir şey bulamayabilecek bir metodun dönüş tipi olmak**. Bunun için kullanıldığında mükemmeldir. Başka bir yerde kullanıldığında genellikle yerine geçtiği null değerinden daha kötüdür, çünkü güvenlik eklemekten tören ekler.

Bu modül çoğunlukla bu farkla ilgili.

Neler Öğreneceksiniz

- `Optional` size gerçekte ne kazandırıyor ve bunu kimin için kazandırıyor
- Asla "orada mı" diye sormayan işlemler
- `map` ile `flatMap`, stream'lerdekiyle aynı kural
- `orElse` ile `orElseGet` neden birbirinin yerine geçmez
- Çökmeyi geri getiren dahil beş yanlış kullanım

Başka Bir Dilden Geliyorsanız

	Başka yerlerde	Java
Yok olan değer	<code>None</code> , <code>null</code> , <code>undefined</code>	<code>Optional.empty()</code>
Güvenli erişim	<code>?.</code>	<code>.map(...)</code>
Varsayılan	<code>x ?? y</code> , <code>or</code>	<code>.orElse(y)</code>
Derleyici zorlaması	Rust <code>Option</code> , Kotlin <code>T?</code>	yok

Son satır önemli. Rust ya da Kotlin'de derleyici yokluğu göz ardı etmenizi engeller. Java'nın `Optional` sınıfı sıradan bir sınıftır ve hiçbir şey `.get()` çağırmanızı engellemez. Bu, **bir tipe desteklenen bir gelenektir**, garanti değil.

Ders

Ne işe yarar

İki imzayı karşılaştırın:

```
User findUser(int id) // might return null. Read the source to find out.
Optional<User> findUser(int id) // says so, in the type.
```

Tüm değer önerisi budur. **Çağırının yükümlülüğünün görünür olmasıyla** ilgilidir, içeride null'dan kaçınmakla değil.

`Optional.of` bilerek null'ı reddeder. Onu yazmak değerin orada olduğunu iddia etmek demektir, dolayısıyla bir NPE iddianızın yanlış olduğunu söyler. Gerçekten bilmiyorsanız `ofNullable` kullanın.

Kutudan çıkarmadan kullanmak

Deyimsel biçim asla bir değerin var olup olmadığını sormaz. Onunla ne yapılacağını ve neye geri düşüleceğini tarif eder. `OptionalBasics.java` dosyasından:

```
findUser(1).map(User::email).orElse("no email")
findUser(1).filter(u -> u.name().startsWith("a")).map(User::name).orElse("no match")
findUser(9).or(() -> findUser(1)) // an alternative Optional, so it chains
```

`ifPresentOrElse` iki dallı durumu ele alır. `Optional.stream()` sıfır ya da bir eleman verir, dolayısıyla `flatMap(Optional::stream)` bir boru hattı içinde ayrı bir filtre olmadan ıskalamaları düşürür.

map ile flatMap

Stream'lerdekiyle aynı kural. Fonksiyonunuz düz bir değer döndürüyorsa `map` kullanın. Zaten bir `Optional` döndürüyorsa `map` size iç içe bir tane verir:

```
map with an Optional-returning fn: Optional[Optional[ada@example.com]]
flatMap:                           Optional[ada@example.com]
```

Bir zincirde aniden iç içe `Optional` tipleri belirdiğinde `flatMap` istiyordunuz.

orElse ile orElseGet

En yaygın `Optional` hatası. `OrElseVsOrElseGet.java` dosyasından, iki çağrı da **dolu** bir `Optional` üzerinde yapılmış:

```
calling orElse:
>>> expensiveDefault ran, called from orElse
result: the real value
calling orElseGet:
result: the real value
```

İkisi de gerçek değeri döndürdü. Yalnızca biri, sonra attığı varsayılanı hesapladı.

	Aldığı	Değerlendirilme
<code>orElse(T other)</code>	bir değer	her zaman, çünkü Java argümanları çağrıdan önce değerlendirir
<code>orElseGet(Supplier<T>)</code>	bir lambda	yalnızca boşken

Varsayılanın yan etkisi olduğu anda performans notu olmaktan çıkar:

```
after orElse on a PRESENT optional, counter = 1
after orElseGet on the same,             counter = 0
```

Artık bir doğruluk hatası. Satır ekleyen, istek gönderen ya da loglayan bir varsayılan, yapmaması gerektiğinde yapacaktır.

Kural: `orElse("")` ve `orElse(0)` sorun değil. `orElse(buildDefault())` ve `orElse(repo.findDefault())` hatadır.

`orElseThrow(supplier)`, "burada yokluk bir hatadır" demenin dürüst yoludur, ve neyin beklendiğini açıklayan bir mesaj yazmanızı sağlar.

Beş yanlış kullanım

`AntiPatterns.java` dosyasından:

1. `isPresent()` sonra `get()`. Bu, daha fazla yazımla yapılmış bir null kontrolüdür. Böyle çiftlerin neredeyse hepsi `map`, `filter`, `ifPresent`, `ifPresentOrElse`, `orElse` ya da `orElseGet` olur.
2. Alan olarak `Optional`. `Serializable` değildir, örnek başına bir nesneye mal olur ve bir alan zaten null olabilir. Alanı düz tutun ve erişimciden bir `Optional` döndürün.
3. Parametre olarak `Optional`. Artık her çağırın `Optional.of(x)` ya da `Optional.empty()` yazmak zorunda, ve yine de null geçirebilirler, dolayısıyla hiçbir şey kazanmadınız. Bir aşırı yükleme kullanın.
4. `Optional<List<T>>`. Çağırının artık üç durumu var: yok, var-ve-boş, var-ve-dolu. Boş bir liste zaten burada hiçbir şey yok demektir.
5. Sıcak döngüde `Optional`. Her biri bir tahsistir. Bu, o argümanın gerçekten tuttuğu ender yerlerden biri.

Önlemesi gereken çökme

`UnguardedGet.java`:

```
Exception in thread "main" java.util.NoSuchElementException: No value present
```

Bu bir `NullPointerException`'dan daha kötüdür, daha iyi değil. Java 14'ten beri bir NPE null olan tam ifadeyi isimlendirir. "No value present" hiçbir şey isimlendirmez. Ve kod *güvenli görünür*, çünkü tipte `Optional` geçer, dolayısıyla inceleyen kişi üstünden atlar.

`get()` metodunun `orElseThrow()` metodunun daha iyi karşılamadığı hiçbir kullanımı yoktur. JDK 10 argümansız `orElseThrow()` metodunu tam olarak bunun için ekledi: aynı davranış, ama bir gözden kaçma değil bilinçli bir seçim olarak okunuyor.

Çalıştırın

```
java modules/15-optional/examples/OptionalBasics.java
java modules/15-optional/examples/OrElseVsOrElseGet.java
java modules/15-optional/examples/AntiPatterns.java

# Bilerek başarısız olur.
java modules/15-optional/examples/UnguardedGet.java

./scripts/verify-examples.sh modules/15-optional
```

Sık Yapılan Hatalar

Hesaplanan varsayımlarla `orElse`. Dolu ya da boş, her seferinde çalışır.

`isPresent()` ardından `get()`. Kostüm giymiş bir null kontrolü yazdınız.

`Optional` alanlar ve parametreler. Hiçbiri amaç değildi, ikisi de gürültü ekler.

`Optional<List<T>>` döndürmek. Boş liste döndürün.

`get()` çağırmak. Mesajlı `orElseThrow` kullanın.

`Optional` döndüren bir metottan `null` döndürmek. Oluyor, ve her iki dünyanın en kötüsü: çağıran tipe güveniyor ve yine de bir NPE alıyor.

Özet Çıkarımlar

- `Optional` aramalar için bir dönüş tipidir, genel amaçlı bir null sarmalayıcı değil.
- Bir gelenektir, garanti değil. Java'nın derleyicisi onu zorlamaz.
- Ne yapılacağını tarif edin, asla orada olup olmadığını sormayın.
- Fonksiyon zaten bir `Optional` döndürüyorsa `flatMap`.
- `orElse` argümanını her zaman değerlendirir; `orElseGet` değerlendirmez. Yan etkiyle bu bir doğruluk hatasıdır.
- Asla `get()`. Mesajlı `orElseThrow` ne beklediğinizi söyler.

Ödev

[homework/README.md](#)

Bir arama zinciri kurun, `orElse` tuzağını bir sayaçla kanıtlayın ve dört yanlış kullanımı düzeltin. Referans çözüm [solutions/15-optional/](#) altında.

Ödev 15: Bir Rehber

Bir arama API'si kurun, `orElse` tuzağını izleyebileceğiniz bir sayaçla kanıtlayın ve dört yanlış kullanımı düzeltin.

Veri

```
record Contact(String name, String email, String phoneOrNull) { }

Map.of(
    "ada", new Contact("ada", "ada@example.com", "555-0100"),
    "grace", new Contact("grace", "grace@example.com", null),
    "alan", new Contact("alan", "alan@example.com", "555-0199"));
```

`grace` kaydının telefonu olmadığına ve alanın `phoneOrNull` olarak adlandırıldığına dikkat edin; sebebini üçüncü bölümde uygulayacaksınız.

Birinci Bölüm: Kutudan Çıkarmadan Arama

`ContactBook.java` dosyasını klasik biçimde oluşturun.

`Optional<Contact>` döndüren `find(String name)` metodu yazın.

Sonra `ada`, `grace` ve `nobody` için her kişinin telefonunu ve e-posta alan adını yazdırın. Eksik değerler okunabilir bir yedek alsın.

Kurallar:

- Bu bölümde hiçbir yerde `isPresent` yok
- Dosyanın tamamında hiçbir yerde `get` yok
- Telefon araması `map` değil `flatMap` gerektirmeli. İkisini de gösterin ve `map` sürümünün neden bir `orElse(String)` içine bile derlenmediğini söyleyin

Sonra `or()` ile üç yedeği zincirleyin ve `orElse` seçeneğinin o zinciri kurmak için neden kullanılamayacağını bir yorumda açıklayın.

Bir ad listesini `find` üzerinden eşleyip yalnızca isabetleri toplayarak bitirin; filtre değil `Optional::stream` kullanın.

İkinci Bölüm: Tuzağı Kanıtlayın

Bir `static int defaultLookups` sayacı ve onu artıran bir `lookupDefaultEmail()` metodu ekleyin.

Sonra üç durumu çalıştırın ve her birinden sonra sayacı yazdırın:

Durum	Beklenen
Dolu bir Optional üzerinde <code>orElse(lookupDefaultEmail())</code>	?
Aynısında <code>orElseGet(ContactBook::lookupDefaultEmail)</code>	?
Boş bir Optional üzerinde <code>orElseGet(...)</code>	?

Çalıştırmadan önce ne tahmin ettiğinizi, sonra gerçekte ne olduğunu yazın.

Açıklayın

Kendi kelimelerinizle bir yorumda:

- Değer mevcutken `orElse` argümanı neden yine de çalışıyor
- `orElseGet` neden çalışmıyor
- Bunun neden yalnızca bir performans notu değil bir doğruluk hatası olduğu.** Cevabınız sayaca atıfta bulunmalı ve gerçek kodda ne olacağını söylemeli.

Üçüncü Bölüm: Dört Yanlış Kullanımı Düzeltin

Her birinin düzeltilmiş biçimini, neyin yanlış olduğunu isimlendiren bir yorumla gösterin:

- `isPresent` sonra `get` — tek bir zincirlenmiş çağrıyla değiştirin
- Alan olarak** `Optional` — `phoneOrNull` alanını düz tutun, erişimciden `Optional` döndürün, ve `Optional` bir alanın neye mal olacağını söyleyin
- Parametre olarak** `Optional` — isteğe bağlı bir etiket alan bir metodu bunun yerine bir **aşırı yükleme** ile yazın
- `Optional<List<T>>` — boş liste döndüren bir arama yazın, ve sarmalanmış sürümün çağırana kaç durum dayatacağını söyleyin

Sonra bir `get()` çağrısını mesajlı `orElseThrow` ile değiştirin. Hem o mesajı hem `get()` metodunun söyleyeceğini yazdırın ve farkı yorumlayın.

Dördüncü Bölüm: Sizi Neye Karşı Korumadığı

`Optional<String>` döndürdüğü bildirilen ama `null` döndüren bir metot yazın.

Çağırın, ne geldiğini yazdırın, sonra sonuç üzerinde herhangi bir metot çağırın ve ne olduğunu yakalayın.

Bir yorumda bunun neden düz null döndüren bir metottan daha kötü olduğunu ve `Optional` sınıfının neden bir garanti değil bir gelenek olduğunu kanıtladığını açıklayın. Derleyicinin işin içinde olduğu Kotlin ya da Rust ile karşılaştırın.

Kabul Kriterleri

- ☐ `java ContactBook.java` ile çalışıyor
- ☐ Dosyada hiçbir yerde `get()` geçmiyor
- ☐ Yanlış kullanımı gösterdiğiniz yer hariç hiçbir yerde `isPresent` geçmiyor
- ☐ Telefon araması `flatMap` kullanıyor, her iki biçim gösteriliyor
- ☐ `or()` üç yedeği zincirliyor, `orElse` neden yapamaz yorumuyla
- ☐ Iskalamaları düşürmek için `Optional::stream` kullanılıyor
- ☐ Sayaç üç farklı sonuçla üç kez yazdırılıyor
- ☐ Açıklama argüman değerlendirmesini ve neden doğruluk hatası olduğunu kapsıyor
- ☐ Dört yanlış kullanımın hepsi düzeltilmiş gösteriliyor, her biri bir yorumla
- ☐ `orElseThrow` mesajı `get()` mesajıyla karşılaştırılıyor
- ☐ Null döndüren `Optional` metodu gösteriliyor ve açıklanıyor

İleri Seviye

Var olan ilk adı döndüren, gereğinden fazla arama değerlendirmeyen `Optional<Contact>` `findAny(String... names)` yazın.

Sonra sürümünüzün tembel olup olmadığını, ve bir stream kullandıysanız `findFirst` metodunun `or()` üzerindeki bir döngünün yapmayacağı neyi yaptığını bir yorumda yazın.

İpucu, birinci bölümün `flatMap` noktası oturmuyorsa

`find("ada").map(Contact::phone).orElse("none")` yazmayı deneyin ve derleyici hatasını okuyun.

Tip `Optional<Optional<String>>` ve onun üzerindeki `orElse` bir `String` değil bir `Optional<String>` istiyor. Hatanın kendisi ders.

Modül 16: Dosyalar ve Giriş/Çıkış

İşin çoğunu iki tip yapar. `Path` bir **addır** ve üzerindeki her metot, diske hiç dokunmayan saf metin işlemesidir. `Files` ise gerçekten okuyan, yazan ve dosya sistemine soru soran şeydir.

Bu ikisini kafanızda ayrı tutmak şaşırtıcı sayıda hatayı önler, güvenlik olanından başlayarak.

Neler Öğreneceksiniz

- Saf ad işlemesi olarak `Path` ve önlediği yol geçişi hatası
- Okuma ve yazma için tek satırlık `Files` metotları
- `Files.lines` neden kapatılmalı, `readString` neden kapatılmamalı
- `java.io.File` neyi yanlış yapıyor, hata yan yana gösterilerek
- Dosyayı ve sebebi isimlendiren istisna ailesi

Başka Bir Dilden Geliyorsanız

	Python	Java
Yol nesnesi	<code>pathlib.Path</code>	<code>java.nio.file.Path</code>
Birleştirme	<code>/</code> operatörü	<code>.resolve(...)</code>
Hepsini oku	<code>path.read_text()</code>	<code>Files.readString(path)</code>
Satırları tembel oku	<code>with open(...) as f</code>	<code>Files.lines(path)</code> , kapatılmalı
Eksik dosya	<code>FileNotFoundException</code>	<code>NoSuchFileException</code>
Varsayılan kodlama	UTF-8	Java 18'den beri UTF-8

`pathlib` ile `java.nio.file` yakın akraba, dolayısıyla buradakilerin çoğu tanıdık gelecek. Dikkat edilecek olan `Files.lines` metodunun kapatma zorunluluğu.

Ders

Path bir addır, dosya değil

`PathBasics.java` dosyasından. `resolve` ekler ve her platformda ayrıca doğru koyar:

```
Path.of("/var/data").resolve("reports/q3.csv") // /var/data/reports/q3.csv
```

Mutlak bir argüman tabanı tamamen değiştirir:

```
Path.of("/var/data").resolve("/etc/passwd") // /etc/passwd
```

Doğrulanmamış kullanıcı girdisinin yol geçişi hatasına dönüşme biçimi budur. Savunma, normalleştirip nereye indiğinizi denetlemektir:

```
Path attempted = uploads.resolve(userSupplied).normalize();
attempted.startsWith(uploads) // false for ../../etc/passwd
```

`String.startsWith` değil `Path.startsWith` kullanın. Karakterler yerine ad öğelerini karşılaştırır, dolayısıyla kandırılmaz:

```
String.startsWith would have said: true // "/srv/uploads-evil/x"
Path.startsWith says: false
```

`normalize()` saftır ve sembolik bağları **takip etmez**, dolayısıyla `b` bir bağ ise `b/..` kaldırmak ulaştığınız dosyayı değiştirebilir. Diske dokunan, bağları çözen ve dosya yoksa fırlatan metot `toRealPath()`.

`Path.equals` sözcükseldir. `Files.isSameFile` dosya sistemine sorar.

Okuma ve yazma

`ReadingAndWriting.java` dosyasından:

```
Files.writeString(path, "content"); // create or
truncate
Files.writeString(path, "more\n", StandardOpenOption.APPEND); // append
String whole = Files.readString(path);
List<String> lines = Files.readAllLines(path);
```

Java 18'den beri varsayılan karakter kümesi her yerde UTF-8. Öncesinde aynı kod farklı makinelerde farklı baytlar okuyordu, ki bu uzun bir kodlama hatası kuyruğu üretti.

Kullanışlı sorgular: `exists`, `isRegularFile`, `isDirectory`, `size`, `isReadable`.

`exists` hakkında bir not. Geçmiş hakkında bir soruyu cevaplar. Cevaba göre hareket etmeden önce dosya kaybolabilir. Önce denetlemek yerine işlemi deneyip istisnayı ele almayı tercih edin. O yarışın adı TOCTOU ve gerçek bir hata kaynağı.

`Files.createDirectories` eksik her üst dizini oluşturur. `copy`, `move` ve `deleteIfExists` adlarının söylediğini yapar, kopyalama seçenekleri de üzerine yazmayı denetler.

Akış halinde okuma ve kapatma kuralı

`readString` ve `readAllLines` **dosyanın tamamını** yükler. Bir yapılandırma dosyası için uygun, bir log için ölümcül.

`StreamingLargeFiles.java` dosyasından:

```
try (var lines = Files.lines(log)) {  
    long errors = lines.filter(l -> l.startsWith("ERROR")).count();  
}
```

`Files.lines`, `Files.list`, `Files.walk` ve `Files.find` hepsi açık bir dosya tanıtıcısı tutar. Onu kapatabilecek tek şey stream'dir, dolayısıyla try-with-resources isteğe bağlı değildir. Java dosya kodunda en sık sızdırılan kaynak budur, ve yoğun bir sunucuda tanıtıcıların tükenmesi demektir.

Bellek kullanımı dosya boyutu ne olursa olsun tek satırda kalır, ve boru hattı kısa devre yapabilir, dolayısıyla ilk eşleşmeyi bulmak yalnızca gerektiği kadar okur.

Bir stream içinden yazarken bir pürüz:

```
lines.forEach(line -> {  
    try { out.write(line); } catch (IOException e) { throw new UncheckedIOException(e); }  
});
```

Bir lambda denetlenen istisna fırlatamaz. Bu, Modül 10'un denetlenen istisnalar ve lambda'lar hakkındaki şikâyetinin doğada karşılaşılan hâli, ve sarmalama alışılmış çözüm.

java.io.File neden değil

`WhyNotLegacyFile.java` dosyasından, aynı başarısızlık iki kez:

```
File.delete() returned: false  
why did it fail? no permission? not there? locked?  
the API does not say, and cannot be made to.  
  
Files.delete() threw NoSuchFileException  
naming the file: /var/.../does-not-exist.txt
```

Merkezi şikâyet bu. Başarısız olabilen her `File` metodu `false` döndürür, dolayısıyla hata işleme tahmin yürütmeye dönüşür.

Üç tane daha: `File` sembolik bağları ele alamaz, dosya niteliklerini tek çağrıda okuyamaz, ve `listFiles()` bir **dizi** döndürür, dolayısıyla bir milyon girdili bir dizin bir milyon elemanlı diziye dönüşür.

`path.toFile()` ve `file.toPath()` serbestçe dönüştürür. **Eski API'lerle sınırda dönüştürün ve kendi kodunuzun içinde her yerde `Path` kullanın.**

İstisna ailesi

`MissingFile.java` bir `NoSuchFileException` fırlatır, ki yolu **yapılandırılmış veri** olarak taşır:

```
catch (NoSuchFileException e) {  
    log.warn("missing config at {}", e.getFile());  
}
```

İstisna	Anlamı
<code>NoSuchFileException</code>	yol yok
<code>FileAlreadyExistsException</code>	var, ve siz oluşturmak istediniz
<code>AccessDeniedException</code>	izinler
<code>DirectoryNotEmptyException</code>	boş olmayan dizinde silme
<code>NotDirectoryException</code>	bir dosyayı dizin gibi ele aldınız

Hepsi `IOException` alt tipidir. Farklı tepki verebildiğinizde belirli olanı, veremediğinizde `IOException` yakalayın.

Çalıştırın

```
java modules/16-files-and-io/examples/PathBasics.java  
java modules/16-files-and-io/examples/ReadingAndWriting.java  
java modules/16-files-and-io/examples/StreamingLargeFiles.java  
java modules/16-files-and-io/examples/WhyNotLegacyFile.java  
  
# Bilerek başarısız olur.  
java modules/16-files-and-io/examples/MissingFile.java  
  
./scripts/verify-examples.sh modules/16-files-and-io
```

Her örnek geçici bir dizinde çalışır ve çıkmadan önce onu siler, dolayısıyla çalıştırmak geride hiçbir şey bırakmaz.

Sık Yapılan Hatalar

`Files.lines` ya da `Files.walk` kapatmamak. Çağrı başına sızdırılmış bir dosya tanıtıcısı.

Büyük bir dosyada `readAllLines`. `Files.lines` kullanın.

Yolları metin birleştirmeye kurmak. `resolve` kullanın, ve kullanıcı girdisinden türeyen her şeyi normalleştirin.

Yollarda kapsama denetimi için `String.startsWith`. `/srv/uploads-evil` geçer. `Path.startsWith` geçirmez.

Her işlemten önce `exists` denetlemek. Cevap elinize geçtiği anda bayatlar.

Dosya işlerinin etrafında `Exception` yakalamak. Belirli alt tipler size yeniden denemeniz mi, dosyayı oluşturmanız mı yoksa pes etmeniz mi gerektiğini söyler.

Yeni kodda `java.io.File` kullanmak. Sebep yerine sessiz boolean.

Özet Çıkarımlar

- `Path` bir addır; `Files` diske dokunur. Path metotları saftır.
- Mutlak argümanla `resolve` tabanı değiştirir, ki bu patlamayı bekleyen bir geçiş hatasıdır. Normalleştirin ve `Path.startsWith` ile denetleyin.
- `readString` her şeyi yükler; `Files.lines` yüklemes ve kapatılmalıdır.
- Her dizin gezme metodu kapatılabilir bir stream döndürür.
- Varsayılan karakter kümesi Java 18'den beri UTF-8, dolayısıyla kodlama artık makineye göre değişmiyor.
- `Files` dosyayı ve sebebi isimlendiren istisnalar fırlatır. `File` ise `false` döndürür.

Ödev

[homework/README.md](#)

Akış halinde okuyan bir log analizcisi kurun, sonra yol geçişine direnen bir yol doğrulayıcı yazın. Referans çözüm [solutions/16-files-and-io/](#) altında.

Ödev 16: Bir Log Aracı

Bir dosyayı düzgün okuyan, bir tane güvenle yazan ve yol geçişi denemesini reddeden bir şey kurun.

Oluşturduğunuz her dosya program çıkmadan önce silinmelidir. `Files.createTempDirectory(...)` içinde çalışın ve bir `finally` bloğunda temizleyin. Geride dosya bırakmak bu ödevi geçersiz kılar.

Birinci Bölüm: Akışla Okuyun, Yutmayın

`LogTool.java` dosyasını klasik biçimde oluşturun.

Şu biçimde 1.000 satırlık bir log üretin:

```
ERROR request=0 ms=10
INFO request=1 ms=47
```

Sonra dört soruyu, **her biri kendi** `Files.lines` çağrısıyla cevaplayın:

- Her seviyeden kaç satır var, seviye adına göre alfabetik
- İlk `ERROR` satırı
- En yüksek `ms=` değerine sahip satır
- Tüm `ms=` değerleri üzerinde özet istatistikler

O stream'lerin hepsi kapatılmalı. try-with-resources olmasaydı neyin sızacağını ve bu metotta kaç kez sızacağını bir yorumda yazın.

İkinci soru için, dosya boyutu ne olursa olsun `readAllLines` metodunun asla yapamayacağı neyi `findFirst` metodunun yaptığını bir yoruma ekleyin.

İkinci Bölüm: Filtrelenmiş Bir Kopya Yazın

Yalnızca `ERROR` satırlarını ikinci bir dosyaya yazın, **iki kaynaklı tek bir try-with-resources** içinde okuyup yazarak.

Bir sorunla karşılaşacaksınız: `forEach` bir `Consumer` alır, ve `write` denetlenen bir `IOException` fırlatır. Çözün, ve genel problemi isimlendirip standart çözümün özgün başarısızlığı neden koruduğunu bir yorumda yazın.

Sonra:

- Sona bir satır ekleyin, ve eklemenin neden açık bir seçenek gerektirdiğini yorumlayın
- İç içe bir dizin oluşturup dosyayı `move` ile içine taşıyın
- Özgününün gittiğini gösterin

Üçüncü Bölüm: Yol Doğrulayıcı

Bir `uploads` dizini oluşturun. Kullanıcıdan gelen bir dosya adını alıp izin verilip verilmediğine karar veren bir metot yazın.

Bu beşinin hepsiyle test edin:

Girdi	Olması gereken
<code>photo.jpg</code>	izin verilir
<code>nested/../photo.jpg</code>	izin verilir
<code>../../etc/passwd</code>	reddedilir
<code>/etc/passwd</code>	reddedilir
<code>..</code>	reddedilir

Dördüncüsü ilginç olanı. `resolve` metodunun mutlak bir argümanla ne yaptığını bulun ve denetiminizin onu yine de yakaladığından emin olun.

Sonra String Tuzağını Kanıtlayın

`uploads--evil` adında kardeş bir dizin ve içinde bir dosya oluşturun.

O dosyanın `String.startsWith` ve `Path.startsWith` metotlarına göre "uploads içinde" olup olmadığını yazdırın. Anlaşamayacaklar.

Nedenini, `Path` metodunun `String` metodunun karşılaştırmadığı neyi karşılaştırdığını ve bunun neden bir merak değil gerçek bir atlatma olduğunu bir yorumda açıklayın.

Dördüncü Bölüm: Bir Şey Söyleyen İstisnalar

Şunların her birini tetikleyip yakalayın ve dosya adını zaten elinizde olan bir değişkenden değil istisnanın kendisinden yazdırın:

1. `NoSuchFileException`
2. `FileAlreadyExistsException`
3. `DirectoryNotEmptyException`

Sonra **eski** `java.io.File` API'si üzerinden eksik bir dosyada `delete()` çağırın ve ne döndürdüğünü yazdırın.

Şunları kapsayan bir yorum yazın:

- Eski dönüş değerinin size ne söylediği ve neyi söyleyemediği
- **Önce** `exists()` denetlemenin neden çözüm olmadığı. Yarışı isimlendirin ve istisnanın neden korunmuş görünen bir satırdan yine de gelebileceğini söyleyin

Kabul Kriterleri

- ☐ `java LogTool.java` ile çalışıyor
- ☐ Geçici dizin siliniyor ve program bunu kanıtlıyor
- ☐ Dört ayrı `Files.lines` çağrısı, her biri try-with-resources içinde
- ☐ Bir yorum onlarsız neyin ve kaç kez sızacağını belirtiyor
- ☐ Bir yorum `findFirst` metodunun `readAllLines` üzerine ne kattığını açıklıyor
- ☐ Filtrelenmiş kopya tek try-with-resources içinde iki kaynak kullanıyor
- ☐ Lambda içindeki denetlenen istisna problemi çözülmüş ve açıklanmış
- ☐ Ekleme açık bir dosya açma seçeneği kullanıyor, nedenini söyleyen yorumla
- ☐ Beş doğrulayıcı durumunun hepsi doğru kararı veriyor
- ☐ `String.startsWith` ve `Path.startsWith` anlaşmazlık içinde gösteriliyor
- ☐ Açıklama `Path` metodunun bunun yerine neyi karşılaştırdığını söylüyor
- ☐ Üç dosya istisnası da yakalanıyor ve dosya adları istisnadan yazdırılıyor
- ☐ Bir yorum TOCTOU yarışını ve `exists()` metodunun onu neden çözmediğini isimlendiriyor

İleri Seviye

Filtrelenmiş kopyayı **atomik** yapın: aynı dizinde geçici bir dosyaya yazın, sonra `StandardCopyOption.ATOMIC_MOVE` ile yerine taşıyın.

Sonra atomik olmayan bir yazma sırasında hedef dosyayı okuyan birinin ne gözlemleyebileceğini ve taşımanın atomik olması için "aynı dizin" koşulunun neden önemli olduğunu bir yorumda yazın.

İpucu, `/etc/passwd` elinizden kaçıyor

Normalleştirmeden önce `base.resolve("/etc/passwd")` çağrısının ne döndürdüğüne bakın. Mutlak bir argüman tabana eklenmez, onu tamamen değiştirir, dolayısıyla sonucun uploads dizininizle hiçbir ilgisi kalmaz. Bunu yakalaması gereken sizin kapsama denetiminiz, ki denetimin `resolve` metoduna güvenmek yerine var olma sebebi tam olarak budur.

Modül 17: Standart Kütüphane Turu

Sürekli kullanacağınız dört köşe, her birinin insanlara gerçek zaman kaybettiren bir tuzağı olduğu için seçildi.

Buradaki her örnek **tekrarlanabilir**: sabit saatler, sabit tarihler, tohumlanmış rastgele üreticiler. Bunun bir sebebi dersin kararlı olması, bir sebebi de zamanı ve rastgeleliği tekrarlanabilir kılmanın test yazmak için ihtiyacınız olan tekniğin kendisi olması.

Neler Öğreneceksiniz

- Ezberlemeye değer String metotları ve `formatted`
- `java.time` : hangi tipi seçmeli, ve aritmetiği neden sınırlıyor
- Tarih işleminin gerçekte bozulduğu yer olan yaz saati uygulaması
- Gerçek hatalara yol açmış beş `Math` davranışı
- Tekrarlanabilir rastgele sayılar, ve onları asla ne zaman kullanmamalı

Başka Bir Dilden Geliyorsanız

	Python	Java
Biçimlendirme	f-string'ler	<code>"%s".formatted(x)</code>
Tarih	<code>datetime</code>	<code>LocalDate</code> , <code>LocalDateTime</code> , <code>ZonedDateTime</code>
Saat dilimi farkındalığı	<code>tzinfo</code> ile isteğe bağlı	seçtiğiniz tiple isteğe bağlı
Süre	<code>timedelta</code>	<code>Duration</code> ve <code>Period</code> , ve farklılar
Rastgele	<code>random.seed(42)</code>	<code>RandomGeneratorFactory.of(...).create(42)</code>

Önemli olan satır: Java size dilim taşıyan ya da taşımayan bir tip seçtirir. `LocalDateTime` bir zaman anı **değildir**, ve onu öyle saymak en yaygın `java.time` hatasıdır.

Ders

String'ler

`Strings.java` dosyasından:

```
" x ".strip() // Unicode-aware; trim() is not
"a,b,c".split(",") // takes a REGEX, not a literal
String.join("-", "2026", "09")
"ab".repeat(3)
"%-10s|%8.2f|%,d".formatted("name", 3.14159, 1_234_567)
```

iki tuzak:

- split** **düzenli ifade alır.** Kaçış olmadan `.` ya da `|` üzerinde bölmek yanlış cevap verir, ve bu klasik bir bir saatlik hatadır.
- compareTo** **negatif bir sayı, sıfır ya da pozitif bir sayı döndürür.** `-1`, `0`, `1` değil. Aksini varsayan kod yanlıştır.

Ezberlemeye değer biçimlendirme dönüşümleri: `%s` her şey, `%d` tam sayı, `%.2f` iki ondalık basamak, `%n` platform satır sonu, `%,d` binlik ayracı, `%%` gerçek yüzde işareti. `%10s` sağa, `%-10s` sola yaslar.

java.time: tipi seçmek

Tip	Tuttuğu	Kullanım
<code>LocalDate</code>	yalnızca tarih	doğum günü
<code>LocalTime</code>	yalnızca saat	açılış saatleri
<code>LocalDateTime</code>	ikisi, dilim yok	duvar saati okuması
<code>ZonedDateTime</code>	bir yerdeki gerçek an	toplantı
<code>Instant</code>	UTC zaman çizgisinde bir nokta	zaman damgası
<code>Duration</code>	saat, saniye	geçen süre
<code>Period</code>	yıl, ay, gün	takvim miktarları

`LocalDateTime` dilim taşımaz. `2026-09-23T10:15` belirtilmemiş bir yerdeki bir andır. Birini saklayıp UTC varsaymak, bu modülün en çok kaçınmanızı istediği hatadır.

Her `now()` isteğe bağlı bir `Clock` alır. Argümansız `now()` çağıran kod, "29 Şubat'ta ne olur" sorusu için test edilemez. Bir `Clock` bağımlılık olarak alın ve sorun kaybolur.

Sınırlayan aritmetik

`DateAndTime.java` dosyasından:

```
31 Jan + 1 month: 2026-02-28
29 Feb + 1 year: 2025-02-28
and back again: 2026-01-28
so +1 month then -1 month is NOT the identity.
```

İkisi de ileri sarmak yerine son geçerli güne sınırlar. Bu bilinçlidir ve **geri alınabilir değildir**. Faturalama kodunuz bir ay ekleyip bir ay çıkarıyorsa, başladığı yere inmez.

`Duration` zaman tabanlı ve tamdır. `Period` takvim farkındalıktır, dolayısıyla bir ay uygulandığı yere göre 28, 29, 30 ya da 31 gündür. İkisinin de var olma sebebi budur.

`ofPattern` metoduna her zaman bir `Locale` verin. Onsuz makinenin varsayılanını kullanır, dolayısıyla aynı kod yurt dışındaki bir sunucuda farklı biçimlendirir. Java 18 öncesindeki varsayılan karakter kümesiyle aynı hata sınıfı.

Yaz saati

`TimeZonesAndDst.java` dosyasından:

```
--- the spring forward gap ---
01:30 EST:      2026-03-08T01:30-05:00[America/New_York]
plus one hour:  2026-03-08T03:30-04:00[America/New_York]
02:30 does not exist; atZone gives: 2026-03-08T03:30-04:00[America/New_York]

--- the autumn fall-back overlap ---
first  (EDT): 2026-11-01T01:30-04:00[America/New_York]
second (EST): 2026-11-01T01:30-05:00[America/New_York]
```

Orada üç şey oluyor:

- 01:30 üzerine bir saat eklemek **03:30** veriyor, çünkü 02:00 hiç var olmadı. Aritmetik doğru; bir gerçek saat geçti.
- Boşluktaki yerel bir saat hiç yoktur. Java fırlatmaz, ileri taşır.
- Sonbaharda 01:30 **iki kez** yaşanır. `atZone` erken ofseti seçer. Geç olanı istiyorsanız söylemeniz gerekir, ve sessizce birini varsaymak rezervasyonların iki kez sayılma biçimidir.

Ve en çok önem taşıyan ayrım:

`plusDays(1)` duvar saatini korur. `plusHours(24)` 24 gerçek saat ekler. Yılda iki gün farklı cevaplar verirler. "Yarın aynı saatte" ile "şu andan yirmi dört saat sonra" farklı işlemlerdir.

Sizi güvende tutan kurallar: bir `Instant` ya da UTC saklayın ve yalnızca gösterim için dönüştürün, sabit ofset yerine `Europe/London` gibi bölge `ZoneId` kullanın, ve gerçek bir an için asla `LocalDateTime` saklamayın.

Math

`MathSurprises.java` dosyasından:

```
Math.abs(MIN_VALUE):    -2147483648
is it negative?         true
  abs(MIN_VALUE) % 7     = -2    <- a negative array index
  floorMod(MIN_VALUE, 7) = 5     <- always in range

-7 / 2                  = -3
Math.floorDiv           = -4

Math.round(2.5)         = 3
Math.round(-2.5)        = -2

MAX_VALUE + 1           = -2147483648
Math.addExact throws: integer overflow
```

abs negatif bir sayı döndürebilir. `int` aralığı asimetriktir, dolayısıyla en küçüğü negatiflemek kendisine sarar. Bu önemlidir çünkü `Math.abs(hash) % buckets` yaygın bir deyimdir ve yukarıdaki çıktının gösterdiği gibi **negatif indeks** üretir. Onun yerine her zaman aralıkta olan `Math.floorMod(hash, buckets)` ya da fırlatan `Math.absExact` kullanın.

Örneğin 8 değil 7 kova kullandığına dikkat edin. `MIN_VALUE` sayısı 8'e tam bölünür, dolayısıyla 8 ile her iki biçim de `0` döndürür ve hata gizli kalır. Başarısızlığın gerçekten görüldüğü bir örnek seçmek için bir parçası.

Tam sayı bölmesi sıfıra doğru keser; `floorDiv` negatif sonsuza doğru yuvarlar. Negatiflerde farklılaşırlar.

round yarımı pozitif sonsuza doğru yuvarlar, dolayısıyla `-2.5` sonucu `-3` değil `-2` olur.

Taşma sessizdir. `addExact`, `subtractExact` ve `multiplyExact` sarmayı reddeder. Sarmalanmış bir değer in çökmeden daha kötü olduğu her yerde kullanın, ki para ya da bellek sayan her şey için bu her zaman geçerlidir.

Rastgele sayılar

`RandomNumbers.java` dosyasından:

```
RandomGenerator rng = RandomGeneratorFactory.of("L64X128MixRandom").create(42);
```

Biçime dikkat edin: üretici **fabrika** oluşturur. `RandomGenerator.of(name)` doğrudan bir üreteç döndürür ve `create` metodu yoktur.

Aynı tohum, aynı dizi. Karıştıran ya da örnekleyen bir testin sabit bir tohuma ihtiyacı vardır, aksi hâlde ayda bir kimsenin tekrar üretmediği sebeplerle başarısız olur. Üretici, bir `Clock` alır gibi parametre olarak alın.

`nextInt(bound)` sınırı dışarıda bırakır; `nextInt(low, high)` alt sınırı içerir, üst sınırı dışarıda bırakır.

Bunları güvenlikle ilgili hiçbir şey için kullanmayın. Sözde rastgeledirler ve yeterli çıktı görüldüğünde tahmin edilebilirler. Token, oturum kimliği ve anahtar için `java.security.SecureRandom` kullanın.

Bir üretici iş parçacıkları arasında paylaşmayın. `java.util.Random` senkronizedir ve darboğaz olur; `ThreadLocalRandom.current()` her iş parçacığına kendininkini verir.

Çalıştırın

```
java modules/17-standard-library-tour/examples/Strings.java
java modules/17-standard-library-tour/examples/DateAndTime.java
java modules/17-standard-library-tour/examples/TimeZonesAndDst.java
java modules/17-standard-library-tour/examples/MathSurprises.java
java modules/17-standard-library-tour/examples/RandomNumbers.java

./scripts/verify-examples.sh modules/17-standard-library-tour
```

Herhangi birini iki kez çalıştırın, çıktı birebir aynıdır, çünkü hiçbirisi geçerli zamana ya da tohumuz bir üretece bağlı değildir.

Sık Yapılan Hatalar

Gerçek bir an için `LocalDateTime` saklamak. Dilimi yoktur. Bir `Instant` saklayın.

Bölge kimliği yerine sabit ofset kullanmak. `+01:00` Londra'da yılın yarısı yanlışır.

`Clock` olmadan `now()` çağırmak. Kod tarihe bağlı her dal için test edilemez hâle gelir.

`Locale` olmadan `ofPattern`. Yurt dışındaki bir sunucuda farklı çıktı.

Kova indeksi için `Math.abs(hash) % n`. Dört milyarda bir negatif. `floorMod` kullanın.

`double` değerlerini `==` ile karşılaştırmak. Bir tolerans içinde karşılaştırın, ya da para için kayan noktadan kaçın.

Testte tohumuz bir üretici. Ayda bir, kimsenin tekrar üretmediği bir kararsızlık.

`split(".")`. O, herhangi bir karakteri eşleyen bir düzenli ifadedir.

Özet Çıkarımlar

- `java.time` tipini dilim gerekip gerekmediğine göre seçin. `LocalDateTime` dilim taşımaz.
- `Clock` ve `RandomGenerator` değerlerini bağımlılık olarak alın, böylece zaman ve rastgelelik test edilebilir olur.

- Tarih aritmetiđi sınırlar ve geri alınabilir deđildir.
- `plusDays(1)` ile `plusHours(24)` yaz saati g nlerinde farklıdır.
- `abs` negatif olabilir, b lme keser, `round` asimetriktir, tařma sessizdir. `Exact` metotları bunun yerine fırlatır.
- Testler i in tohumlayın;  nemli olan her řey i in `SecureRandom` kullanın.

 dev

[homework/README.md](#)

Yaz saatinden sađ  ıkan bir planlama aracı kurun, sonra d rt `Math` tuzađını yeniden  retin. Referans   z m [solutions/17-standard-library-tour/](#) altında.

Ödev 17: Mart'tan Sağ Çıkan Bir Planlayıcı

Tarihleri doğru işleyen bir şey kurun, sonra dört `Math` tuzağını yeniden üretin.

Programınızın yazdirdığı her satır her çalıştırmada birebir aynı olmalı, ve makinenin hangi saat dilimine ayarlı olduğundan bağımsız olmalı. Bu, sabit bir `Clock`, sabit tarihler ve tohumlanmış bir üreteç demek. Değişen çıktı bu ödevi geçersiz kılar.

Birinci Bölüm: Aylık Faturalama

`Scheduler.java` dosyasını klasik biçimde oluşturun.

Bir müşteri **31 Ocak 2026** tarihinde kaydoluyor. O tarihi sabit bir `Clock` üzerinden okuyun, argümansız `LocalDate.now()` ile değil.

Sonraki dört fatura tarihini **iki biçimde** yazdırın:

1. **Naif biçimde**, bir önceki sonuca tekrar tekrar bir ay ekleyerek
2. **Sabitlenmiş biçimde**, her zaman özgün kayıt tarihinden hesaplayarak

İkisi ikinci aydan itibaren ayrışıyor.

Açıklayın

Şunları kapsayan bir yorum yazın:

- `plusMonths` metodunun 31 Ocak'a ne yaptığı ve o kısmın neden doğru olduğu
- Naif sürümün hatasının neden tek seferlik değil **kalıcı** olduğu
- Sabitlenmiş sürümün farklı olarak ne yaptığı, tek cümlede

İkinci Bölüm: Yaz Saatini Aşan Bir Toplantı

6 Mart 2026 Cuma günü New York saatiyle 09:00'a bir toplantı koyun, sonra onu iki biçimde üç gün ileri taşıyın:

- `plusDays(3)`
- `plusHours(72)`

İkisini de yazdırın, artı özgün ile `plusDays` sonucu arasında geçen gerçek saat sayısını. 72 olmayacak.

Sonra Londra'nın her iki an için ne gördüğünü gösterin.

Üç Durum Daha

1. **Belirsiz saat.** 1 Kasım 2026'da New York saatiyle 01:30 iki kez yaşanır. İkisini de ve aralarındaki farkı yazdırın.
2. **Var olmayan saat.** 8 Mart 2026'da 02:30 hiç yaşanmaz. `atZone` metodunun onunla ne yaptığını yazdırın.
3. **Kayan fark.** Londra ve New York farklı tarihlerde değişir. Bunun beş saatlik farkı sabit yazan biri için ne anlama geldiğini söyleyin.

Açıklayın

`plusDays` ile `plusHours` metotlarının neden farklılaştığını, her birinin hangi soruyu cevapladığını, ve tekrarlayan bir toplantı ile bir zaman aşımı için hangisini kullanacağınızı kapsayan bir yorum.

Üçüncü Bölüm: Dört Math Tuzağı

Her birini yeniden üretin ve açıklayın:

1. `Math.abs(Integer.MIN_VALUE)` ve yol açtığı **negatif kova indeksi**. `abs(hash) % buckets` ile `floorMod(hash, buckets)` sonuçlarını yan yana gösterin.
2. `-7 / 2` ile `Math.floorDiv(-7, 2)` karşılaştırması
3. `Math.round(2.5)` ile `Math.round(-2.5)` karşılaştırması
4. Sessiz taşma, ve `Math.addExact` metodunun onun yerine ne yaptığı

Birinci tuzakta dikkatli olun. Kova sayınızı hatanın gerçekten görüneceği şekilde seçin. Bazı değerler her iki biçimi de aynı sonuca götürüp hatayı tamamen gizler, ve hangisinin öyle olduğunu bulmak alıştırmamanın bir parçası.

Dördüncü Bölüm: Tekrarlanabilir Örnekleme ve Biçimlendirme

Bir listeden rastgele N ad seçen, **tohumu** parametre olarak alan bir metot yazın.

Aynı tohumla iki kez ve farklı bir tohumla bir kez çağırın. İlk ikisinin uyuştuğunu gösterin.

Tohumsuz bir üretecin bir teste ne yapacağını ve bu üreticileri asla ne için kullanmamanız gerektiğini bir yorumda yazın.

`formatted` kullanan küçük bir biçimlendirilmiş rapor ile bitirin:

- Açık bir `Locale` üzerinden biçimlendirilmiş bir tarih
- Sola yaslı bir etiket sütunu
- Binlik ayrıçlı bir sayı

`Locale` değerinin neden isteğe bağlı olmadığını bir yorumda söyleyin.

Kabul Kriterleri

- ☐ `java Scheduler.java` ile çalışıyor
- ☐ İki kez çalıştırmak birebir aynı çıktıyı üretiyor
- ☐ Sabit bir `Clock` kullanılıyor; hiçbir yerde çıplak `now()` yok
- ☐ Her iki faturalama takvimi de yazdırılıyor ve ikinci aydan itibaren ayrışıyor
- ☐ Bir yorum naif hatanın neden kalıcı olduğunu açıklıyor
- ☐ `plusDays(3)` ile `plusHours(72)` farklı sonuç verirken gösteriliyor
- ☐ Geçen gerçek saat yazdırılıyor ve 72 değil
- ☐ Belirsiz saat, bir saat arrayla iki kez gösteriliyor
- ☐ Var olmayan saatin ileri taşındığı gösteriliyor
- ☐ Dört Math tuzağı da görünür çıktıyla yeniden üretiliyor
- ☐ Birinci tuzağın kova sayısı negatif indeksi gerçekten açığa çıkarıyor
- ☐ Örneklem tohumlanmış, ve aynı tohumlu iki çalıştırma uyuyor
- ☐ `ofPattern` metoduna açık bir `Locale` veriliyor, nedenini söyleyen yorumla

İleri Seviye

Yaz saati geçişinde aynı **yerel** saatte kalan haftalık bir tekrar döndüren `List<ZonedDateTime> weeklyMeetings(ZonedDateTime first, int count)` yazın.

Sonra her tekrarı yedi günlük saniye ekleyerek hesaplanan bir `Instant` olarak saklasaydınız neyin ters gideceğini, ve bir yıl sonraki, dilim kuralları o zamana kadar değişebilecek bir toplantı için bunun yerine ne saklayacağınızı bir yorumda yazın.

İpucu, çıktınız çalıştırmalar arasında değişiyorsa

Argümansız `now()` çağrılarını ve tohumsuz oluşturulmuş üreteçleri arayın. Buradaki tek değişkenlik kaynağı bunlar, ve ikisinin de bağımlı oldukları şeyi parametre olarak alan bir biçimi var. Bu ödev için bir numara değil; zamana bağlı kodun genel olarak test edilebilir kılınma biçimi.

Modül 18: Eşzamanlılık Temelleri

İş parçacıkları veri paylaştığında iki şey ters gider, ve bunlar aynı şey değildir.

Yarışlar araya girmeyele ilgilidir: iki iş parçacığı bir işlemin ortasında birbirine karışır. **Görünürlük**, bir iş parçacığının diğerinin yazdığını hiç görmemesiyle ilgilidir. Bu modül ikisini de gösterir, sonra her birini çözen araçları anlatır.

Neler Öğreneceksiniz

- İş parçacıkları, kesme, ve neden kendiniz iş parçacığı oluşturmamalısınız
- `count++` neden artışları kaybediyor, yüz binlerce kayıpla gösterilerek
- `synchronized`, atomikler ve `LongAdder`, ve her birinin bedeli
- Bir döngünün hiç bitmediği görünürlük problemi
- `ExecutorService`, ve sessiz kalan başarısızlık biçimi

Başka Bir Dilden Geliyorsanız

	Python	Java
Gerçek paralellik	GIL engelliyor	evet, gerçekten
<code>x += 1</code> üzerinde yarış	mümkün	ölçekte kesin
Kilit	<code>threading.Lock</code>	<code>synchronized</code> , <code>ReentrantLock</code>
Atomik sayaç	yerleşik yok	<code>AtomicInteger</code>
İş parçacığı havuzu	<code>ThreadPoolExecutor</code>	<code>ExecutorService</code>
Görünürlük anahtar kelimesi	gerekmiyor	<code>volatile</code>

Python'un global yorumlayıcı kilidi bunların çoğunu gizler. Java iş parçacıklarını gerçek çekirdeklerde paralel çalıştırır, dolayısıyla buradaki her tehlike gerçekten karşılaşacağınız bir tehlikedir.

Ders

Doğrudan iş parçacıkları

```
Thread worker = new Thread(() -> doWork(), "worker-1");
worker.start();
worker.join();
```

`start()` ile `run()` aynı şey değildir. `start()` bir iş parçacığı oluşturur; `run()` yalnızca mevcut olanda bir metod çağırır. Araya girecek bir şey olmadığında çıktı aynı olduğu için hata çalışıyormuş gibi görünür.

Kesme bir istektir, bir öldürme değil. `InterruptedException` yakalamak bayrağı temizler, dolayısıyla geri koyun:

```
catch (InterruptedException e) {  
    Thread.currentThread().interrupt();  
}
```

`Thread.stop()` 2000'de kullanımdan kaldırıldı ve artık tamamen kaldırıldı, çünkü bir iş parçacığını işlemin ortasında öldürmek, değiştirdiği her şeyi bilinmeyen bir durumda bırakır.

Neden kendiniz iş parçacığı oluşturmamalısınız: bir platform iş parçacığı yaklaşık 1MB yığın alanına mal olur, kaçının var olduğuna dair bir sınır yoktur, ve hiçbir şey sonuçları toplamaz ya da hataları yukarı iletmez.

Yarış

`TheRaceCondition.java` dosyasından, her biri 200.000 kez artıran dört iş parçacığı:

```
expected total: 800000  
actual total:   242657  
-> 557343 increments were LOST
```

`count++` üç işlemdir: oku, ekle, yaz. İki iş parçacığı da `101` yazmadan önce `100` okuyabilir, ve bir artış kaybolur.

Aynı biçim her yerde görünür: `x += 1`, `list.add(item)`, `balance -= amount`, ve her denetle-sonra-davran çifti:

```
if (!map.containsKey(k)) { map.put(k, v); }
```

İki iş parçacığı da biri koymadan önce denetimi geçebilir. `Map` sınıfının tek atomik işlem olarak `putIfAbsent` metoduna sahip olmasının sebebi budur.

Üç çözüm

`FixingTheRace.java` dosyasından:

unsafe ++	796131	WRONG
synchronized	800000	correct
AtomicInteger	800000	correct
LongAdder	800000	correct

synchronized karşılıklı dışlama ve bir önce-olur kenarı verir: bir iş parçacığının kilidi bırakmadan önce yaptığı her şey, kilidi sonra alan iş parçacığına görünür. **İnsanların unuttuğu ikinci garanti budur**, ve **synchronized** metodunun görünürlüğü de çözmesinin sebebi. Bedeli, bloke olan iş parçacıklarının sıraya girmesidir.

Atomikler bir karşılaştırmak-ve-değiştir komutu kullanır: yalnızca hiçbir şey değişmediyse geri yaz, aksi hâlde yeniden dene. Bloke olma yok, ama çekişme altında yeniden denemeler bedel olur.

LongAdder birkaç iç hücre tutar, dolayısıyla iş parçacıkları çoğunlukla aynı belleğe dokunmaz. Yoğun yazma çekişmesi altında **AtomicLong** sınıfından hızlı, okuması yavaştır. Metrikler için onu; her güncellemeden sonra değere ihtiyacınız olduğunda **AtomicLong** kullanın.

Ve genellikle en iyisi olan dördüncü seçenek: durumu hiç paylaşmayın. Modül 14 paralel stream'ler için aynı noktayı yapmıştı, orada çözüm her iş parçacığına kendi kabını veren **collect** idi. Yaşayamayacağınız bir yarış, senkronize ettiğiniz bir yarıştan ucuzdur.

Neye kilitlememeli:

Yapmayın	Çünkü
<code>synchronized (this)</code>	referansı olan herkes sizi kilitleyebilir
<code>synchronized (SomeClass.class)</code>	aynısı, ama küresel olarak
bir <code>String</code> sabiti	sabitler havuzlanır ve ilgisiz kodla paylaşılır
kutulanmış bir <code>Integer</code>	128 altındaki değerler önbelleklenir

Bir `private final Object` kullanın.

Görünürlük

İnsanların beklemediği kısım bu. `MemoryVisibility.java` dosyasında bir iş parçacığı bir bayrak üzerinde dönüyor ve bir diğeri onu ayarlıyor:

```
main set plainFlag = true
after waiting 1.5s, reader still running: true

main set volatileFlag = true
after waiting 1.5s, reader still running: false
```

Düz okuyucu yazmayı **hiç görmedi**. Burada hiç araya girme yok.

O döngüdeki hiçbir şey paylaşılan duruma dokunmuyor, dolayısıyla JIT `plainFlag` değerinin değişmeyeceğini varsayıp okumayı tamamen döngü dışına çıkarabilir ve onu `while (true)` hâline getirebilir. Bu **yasal** bir optimizasyondur. Java Bellek Modeli, bir iş parçacığının diğerinin yazdığını görmesini yalnızca ikisini bir önce-olur ilişkisi bağladığında söz verir, ve düz bir alan böyle bir ilişki kurmaz.

volatile ne yapar: her okuma ana belleğe gider, ve önce-olur kurar, dolayısıyla bir volatile yazmadan önce yazılan her şey eşleşen okumadan sonra görünür.

Ne yapmaz: `count++` işlemini atomik kılmaz. Üç işlem üç işlem olarak kalır.

volatile bayraklar ve bir referans yayımlamak içindir. **synchronized** , atomikler ve kilitler bileşik eylemler içindir.

ExecutorService

```
try (ExecutorService pool = Executors.newFixedThreadPool(3)) {  
    Future<Integer> result = pool.submit(() -> compute());  
    System.out.println(result.get());  
}
```

ExecutorService Java 19'dan beri `AutoCloseable` , dolayısıyla try-with-resources insanların düzenli olarak yanlış yaptığı `shutdown` ve `awaitTermination` çiftinin yerini alıyor.

Gerçek bir tuzak, `UsingExecutors.java` dosyasından:

```
submitted, and nothing has been reported yet  
get() threw ExecutionException  
caused by: java.lang.IllegalStateException: the task failed
```

`submit()` ile gönderilen, fırlatan ve `Future` nesnesi hiç incelenmeyen bir görev **sessizce başarısız olur**. Hiçbir şey loglanmaz, hiçbir şey çıkmaz. Varsayılan işleyicinin raporlamasını istiyorsanız `execute()` kullanın, ya da `Future` nesnesini her zaman inceleyin.

Havuz	Kullanım
<code>newFixedThreadPool(n)</code>	sınırlı; CPU işi için güvenli varsayılan
<code>newCachedThreadPool()</code>	sınırsız; görev patlaması iş parçacığı patlaması yapar
<code>newSingleThreadExecutor()</code>	teker teker, sırayla
<code>newScheduledThreadPool(n)</code>	gecikmeli ve tekrarlayan

IO ağırlıklı iş için bunların hepsi çok az iş parçacığı ile çok fazlası arasında bir seçim dayatır. Modül 19 bu seçimi ortadan kaldırıyor.

Çalıştırın

```
java modules/18-concurrency-basics/examples/ThreadsAndRunnables.java
java modules/18-concurrency-basics/examples/TheRaceCondition.java
java modules/18-concurrency-basics/examples/FixingTheRace.java
java modules/18-concurrency-basics/examples/MemoryVisibility.java
java modules/18-concurrency-basics/examples/UsingExecutors.java

./scripts/verify-examples.sh modules/18-concurrency-basics
```

Yarış örneğini birkaç kez çalıştırın. Kaybedilen sayı her seferinde değişir, ve çok çekirdekli bir makinede asla sıfır olmaz. Ders o tutarsızlıktır.

Sık Yapılan Hatalar

`start()` yerine `run()` çağırmak. Hiçbir iş parçacığı oluşturulmaz.

`volatile` kelimesinin `++` işlemini güvenli kıldığını varsaymak. Kılmaz.

`InterruptedException` yutmak. Bayrağı geri koyun, yoksa kapanma sinyali kaybolur.

`this`, bir sınıf, bir `String` ya da kutulanmış bir `Integer` üzerine kilitlemek. Hepsi düşündüğünüzden çok daha geniş paylaşılır.

Bir `Future` nesnesini denetlememek. İstisnalar tamamen kaybolur.

Bir sunucuda `newCachedThreadPool`. Yük altında sınırsız iş parçacığı oluşturma.

Eşzamanlı bir haritada denetle-sonra-davran. `containsKey` sonra `put` iki işlemdir. `putIfAbsent` ya da `compute` kullanın.

Güvende olmak için her şeyi senkronize etmek. Çekişme, paralel sürümü ardışık olandan yavaş yapabilir, ve Modül 14 tam olarak bunu göstermişti.

Özet Çıkarımlar

- `count++` üç işlemdir, dolayısıyla çekişme altında artış kaybeder.
- `synchronized` karşılıklı dışlama ve görünürlük verir. Atomikler kilitsiz güncelleme verir. `LongAdder` yoğun yazma çekişmesinde kazanır.
- Görünürlük ayrı bir tehlikedir. Düz bir alan önbelleklenebilir ya da döngü dışına çıkarılabilir, ve okuyucu yazmayı hiç görmeyebilir.
- `volatile` görünürlüğü çözer, atomikliği değil.
- Duruma erişimi senkronize etmek yerine paylaşmamayı tercih edin.
- Bir `ExecutorService` kullanın, try-with-resources ile kapatın, ve `Future` nesnesini her zaman inceleyin.

Ödev

[homework/README.md](#)

Bir sayacı dört biçimde kurun, kırın, sonra bir denetle-sonra-davran yarışını düzeltin. Referans çözüm [solutions/18-concurrency-basics/](#) altında.

Ödev 18: Sayamayan Bir Sayaç

Aynı sayacı dört biçimde kurun, birinin başarısız olmasını izleyin, sonra iş parçacığı güvenli bir koleksiyonun sizi **korumadığı** bir yarışı düzeltin.

Programınız her seferinde temiz çıkmalı. Bitmeyebilecek her iş parçacığı daemon olmalı, ve her `join` bir zaman aşımına sahip olmalı.

Birinci Bölüm: Dört Sayaç

`BankAccount.java` dosyasını klasik biçimde oluşturun.

`increment()`, `value()` ve `name()` metotlarına sahip bir `Counter` arayüzü ve dört uygulama tanımlayın:

Ad	Mekanizma
<code>UnsafeCounter</code>	düz bir <code>long count</code> ve <code>count++</code>
<code>SynchronizedCounter</code>	bir <code>synchronized</code> bloğu
<code>AtomicCounter</code>	<code>AtomicLong</code>
<code>AdderCounter</code>	<code>LongAdder</code>

Her birini aynı iş yükünden geçirin: 4 iş parçacığı, her biri 100.000 artış. Sonucu, doğru olup olmadığını ve ne kadar sürdüğünü yazdırın.

Senkronize sürüm için tek gereksinim: `this` üzerine değil, bir `private final Object` üzerine kilitlenin. `this` ile neyin ters gidebileceğini söyleyen bir yorum yazın, ve kilitlenmesi kötü olan iki şey daha isimlendirip nedenini söyleyin.

Açıklayın

Kendi kelimelerinizle bir yorumda:

- `count++` gerçekte neye derleniyor
- Tam olarak bir artış kaybeden, iki iş parçacığına ait somut bir araya girme
- Kaybedilen sayının neden **her çalıştırmada değiştiği**, ve bunun sizin için neden tutarlı biçimde yanlış bir cevaptan daha kötü olduğu

İkinci Bölüm: Hiç Bitmeyen Bir Döngü

İki alan tutan bir sınıf yazın: düz bir `boolean` ve bir `volatile boolean`.

Her biri için:

1. Bayrak true olana kadar dönen bir okuyucu iş parçacığı başlatın
2. Kısa süre uyuyup `main` içinden bayrağı ayarlayın
3. Zaman aşımıyla `join` yapın, sonra okuyucunun hâlâ canlı olup olmadığını yazdırın

Düz olan hâlâ canlı olacak. Volatile olan olmayacak.

Açıklayın

Doğru anlaşılmaya değer kısım bu. Şunları kapsayan bir yorum yazın:

- **Bunun neden bir yarış olmadığı.** Kaç iş parçacığı yazıyor, ve kaç kez?
- Derleyicinin o döngüye ne yapmasına izin verildiği, ve neden izin verildiği
- `volatile` neyi değiştiriyor
- `volatile` **birinci bölümün sayacını neden düzeltmezdi.** İki ayrı problemi isimlendirin ve `volatile` hangisini çözüyor söyleyin

Üçüncü Bölüm: Denetle Sonra Davran

Bir `ConcurrentHashMap` kullanın. 8 iş parçacığının hepsi aynı anahtarı almaya çalışsın:

```
if (!map.containsKey("winner")) {
    map.put("winner", id);
    winners.add(id);
}
```

Kaç iş parçacığının kazandığına inandığını yazdırın. Birden fazla olacak.

Sonra tam olarak birinin kazanabileceği şekilde düzeltin ve onu da yazdırın.

Sayacınızın tepe değeri kendisi de yarışsız olmalı. Aynı anda tepeyi yükselten iki iş parçacığı bir güncellemeyi kaybetmemeli, ve bunu nasıl yapacağınızı çözmek alıştırmamızın bir parçası.

Asıl Önemli Soru

Harita bir `ConcurrentHashMap`, yani iş parçacığı güvenli. Bunun yarışı neden **önlemediğini**, iş parçacığı güvenli bir koleksiyonun neyi garanti edip neyi etmediğini, ve harita zaten güvenliyse `putIfAbsent`, `compute` ve `merge` metotlarının neden var olduğunu açıklayan bir yorum yazın.

Yarışın gerçekten tetiklenmesi için ipucu: her iş parçacığı bir sonraki başlamadan önce başlayıp bitiyorsa hiçbir şey çıkmaz. İki `CountDownLatch` nesnesi, her iş parçacığını kapıda tutup hepsini birlikte salmanızı sağlar.

Dördüncü Bölüm: Kimsenin Duymadığı Başarısızlık

Bir `ExecutorService` içine fırlatan bir görev gönderin ve `Future` nesnesini asla incelemeyin. Hiçbir şeyin raporlanmadığını gösterin.

Sonra aynı görevi gönderip `get()` çağırın ve geleni yakalayın. Sebebi yazdırın.

Bir başarısızlığın asla kaybolmamasını sağlamanın üç yolunu sıralayın.

Kabul Kriterleri

- ☐ `java BankAccount.java` ile çalışıyor ve her seferinde temiz çıkıyor
- ☐ Dört sayacın hepsi aynı iş yükünü zamanlamalarıyla çalıştırıyor
- ☐ Güvensiz olan görünür biçimde yanlış; diğer üçü tam olarak doğru
- ☐ `synchronized` bir private final nesne üzerine kilitleniyor, nedenini söyleyen yorumla
- ☐ Kilitlenmesi kötü iki hedef daha, gerekçeleriyle isimlendirilmiş
- ☐ Açıklama kaybettiren somut bir araya girme veriyor
- ☐ Düz bayrak okuyucusu join sonrası hâlâ canlı; volatile olan değil
- ☐ Okuyucu iş parçacıkları daemon ve join'ler zaman aşımli
- ☐ Açıklama bunun neden yarış olmadığını ve `volatile` metodunun birinci bölüm için neden yetmediğini söylüyor
- ☐ Bozuk denetle-sonra-davran birden fazla kazanan bildiriyor
- ☐ Düzeltilmiş sürüm tam olarak bir tane bildiriyor
- ☐ Bir yorum `ConcurrentHashMap` neyi garanti edip etmediğini açıklıyor
- ☐ Sessiz başarısızlık gösteriliyor ve sonra açığa çıkarılıyor

İleri Seviye

Düzeltilmiş denetle-sonra-davran kodunu `compute` ile değiştirin, böylece kazanan iş parçacığı kimliğinin yanına atomik olarak bir zaman damgası da kaydedebilsin.

Sonra lambda'nız çalışırken `compute` metodunun neyi tuttuğunu, ve o lambda'nın içinde başka bir haritanın metodunu çağırmanın, IO yapmanın ya da ikinci bir kilit almanın neden kötü fikir olacağını bir yorumda yazın.

İpucu, güvensiz sayaç doğru çıkıyorsa

İş parçacıklarının gerçekten çakıştığından emin olun. İş yükünüz yeterince küçükse ya da havuz tek elemanlıysa, her görev bir sonraki başlamadan biter ve araya girecek hiçbir şey olmaz. Daha fazla yineleme ve çekirdek sayısı kadar iş parçacığı bunu sağlar.

Modül 19: Sanal İş Parçacıkları

Sanal iş parçacığı, işletim sisteminin değil JVM'in zamanladığı bir iş parçacığıdır. Aynı `Thread` API'si, aynı kod, aynı hata ayıklayıcı. Değişen şey fiyatı.

Bir platform iş parçacığı yaklaşık bir megabayt yığın alanı ve bir işletim sistemi zamanlama yuvası ayırır. Sanal iş parçacığı ise gerektiğinde büyüyen bir yığın nesnesidir. Bu tek değişiklik bloke olmayı yeniden ucuzlatır, ve bu modül çoğunlukla bundan ne çıktığıyla ilgili.

Diğer kaynaklar hakkında bir uyarı: sanal iş parçacığı sabitlenmesi hakkında yazılmış materyalin çoğu güncelliğini yitirmiştir. Bu modül davranışı tekrarlamak yerine ölçüyor.

Neler Öğreneceksiniz

- Binme, inme, ve taşıyıcı iş parçacığının ne olduğu
- Savı kuran kıyaslama, kendi makinenizde ölçülmüş
- Bugün neyin sabitlediği, ki eskiye göre çok daha azı
- `ScopedValue`, ve `ThreadLocal` neden bir milyon iş parçacığına ölçeklenmiyor
- İnsanların onları yanlış kullandığı dört yol, üçü eski alışkanlıklardan

Başka Bir Dilden Geliyorsanız

	Go	Python	Java
Hafif eşzamanlılık	goroutine	<code>asyncio</code> görevi	sanal iş parçacığı
Bloke eden çağrı	sorun değil, zamanlayıcı halleder	döngüyü bloke eder	sorun değil, iş parçacığı iner
Renk problemi	yok	<code>async</code> / <code>await</code> API'nizi ikiye böler	yok
Birim maliyet	~2KB	küçük	küçük, gerektiğinde büyür

Sanal iş parçacıkları Java'yı Go'nun başından beri olduğu yere getiriyor: sıradan, bloke eden, ardışık kod yazıyorsunuz ve çalışma zamanı onu ölçeklendiriyor. Kütüphaneyi ikiye bölen bir `async` anahtar kelimesi yok.

Ders

Binme ve taşıyıcılar

```
Thread.ofVirtual().name("worker").start(() -> ...);
```

İş parçacığını yazdırmak `VirtualThread[#33,worker]/runnable@ForkJoinPool-1-worker-1` gibi bir şey gösterir. O `ForkJoinPool-1-worker-1` **taşıyıcıdır**: sanal iş parçacığının şu an üzerine bindiği gerçek bir platform iş parçacığı.

Sanal iş parçacığı bloke olduğunda iner ve taşıyıcı bir başkasını alır. Tüm mekanizma bu. Bloke olmak artık bir iş parçacığına mal olmuyor.

Gerçekten kullanacağınız executor bir havuz değil, **görev başına bir iş parçacığı** oluşturur:

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) { ... }
```

Boyutlandırılacak bir şey yok, çünkü kıt kaynak iş parçacıkları değil. `VirtualThreads.java` dosyasından:

```
100000 virtual threads, each sleeping 10ms: 216ms
```

Kıyaslama

`Benchmark.java` dosyasından, 8 çekirdekli bir makinede her biri 100ms bloke olan 5.000 görev:

virtual thread per task	127ms
platform pool of 200	2716ms
platform pool of 50	10750ms

Aritmetik basit. 200'lük bir havuzda aynı anda en fazla 200 görev bloke olabilir, dolayısıyla 5.000 görev 25 grup halinde 100ms sürer. 50'lik havuz 100 grup alır. Sanal iş parçacıklarının böyle bir sınırı yoktur, dolayısıyla 5.000'i birden bloke olur.

Bunun ortadan kaldırdığı seçim. IO için bir platform havuzunu boyutlandırmak her zaman kötü bir takastı: çok az iş parçacığı ve istekler bloke olanların arkasında sıraya girer, çok fazla ve her biri için bir megabayt yığın öderken işletim sistemi zamanlayıcısı çırpınmaya başlar.

Alışılmış kaçış eşzamansız koddu, ki çalışır ve size okunabilir yığın izlerini, adım adım hata ayıklamayı ve sıradan `try / catch` kullanımını kaybettirir. Sanal iş parçacıkları eşzamansız ölçeklenebilirliği düz bloke eden koda geri veriyor.

Yardımcı olmadıkları yer: CPU ağırlıklı iş. Hiç bloke olmayan bir görev hiç inmez, ve çekirdek sayınızdan fazla işi paralel çalıştıramazsınız. Orada sabit bir platform havuzu hâlâ doğrudur.

Sabitlenme, bugün

Sabitlenme, sanal iş parçacığının inememesi ve dolayısıyla taşıyıcısının bloke kalıp bir platform iş parçacığının boşa harcanmasıdır.

Java 21 ile 23 arasında, `synchronized` bloğu içinde bloke olmak taşıyıcıyı sabitliyordu, ve o dönemden kalma her yazı `synchronized` yerine `ReentrantLock` kullanmanızı söyler.

JDK 24'te gelen JEP 491 bunu kaldırdı. `PinningToday.java` dosyasından, her biri kendi kilidini tutarken 100ms uyuyan 2.000 görev:

```
cores, and therefore carriers: 8
elapsed: 131ms

If synchronized still pinned, only 8 tasks could sleep at
once, so this would take about 25000ms.
```

Hâlâ sabitleyenler:

1. **Yerel bir metot ya da yabancı fonksiyon çağırısı.** JVM, denetlemediği bir yığını indiremez.
2. **Bloke olan bir sınıf ilklendiricisi.** Nadir, ve genellikle zaten bir tasarım problemi.

İkisi de eski kuraldan çok daha dar.

Sabitlenme ile çekişme farklı şeylerdir. Kilit hâlâ kilittir: bin sanal iş parçacığı tek bir monitör için yarışırsa 999'u bekler. Sabitlenme bir *taşıyıcıyı* harcar; çekişme *zaman* harcar. JEP 491 birincisini çözdü ve ikincisine dokunamazdı, çünkü sıraya sokmak kilidin işidir.

Kendi kodunuzu, bu yazı dahil hiçbir makaleye güvenmek yerine `-Djdk.tracePinnedThreads=full` ile denetleyin.

ScopedValue

JDK 25'te kesinleşti. Yaygın durumda `ThreadLocal` yerine geçer.

Problem: bir `ThreadLocal`, birinin `remove()` çağırmaı hatırlamasına kadar yaşayan, değiştirilebilir, sınırsız, iş parçacığı başına bir girdidir. Birkaç yüz havuzlanmış iş parçacığıyla bu katlanılabildi. Bir milyon sanal iş parçacığıyla değil.

```
private static final ScopedValue<String> CURRENT_USER = ScopedValue.newInstance();

ScopedValue.where(CURRENT_USER, "ada").run(() -> {
    handleRequest();           // CURRENT_USER.get() works at any depth
});
// binding is gone here, including if the body threw
```

Unutulacak bir `remove()` yok. Bağlama tam olarak `run()` süresince yaşar.

Değişmezdir, ki asıl nokta bu. Bir `ThreadLocal`, ona ulaşabilen her şey tarafından, her derinlikte ayarlanabilir ve değişiklik iş parçacığının ömrü boyunca kalır. Bir `ScopedValue` yalnızca iç içe bir kapsamla gölgelenebilir, ki o da kodda görünür.

Bilinmeye değer bir kısıt:

```
child thread sees it bound? false
```

Düz bir çocuk iş parçacığı bağlamayı **miras almaz**, bilerek: ebeveynin kapsamı çocuk hâlâ çalışırken bitebilir. Miras alma yapılandırılmış eşzamanlılık gerektirir, ki o da JEP 533 ve bu sürümde hâlâ preview. Modül 22 onu anlatıyor.

Kullanım	Ne için
<code>ScopedValue</code>	bir çağrı süresince salt okunur değer: mevcut kullanıcı, istek kimliği
<code>ThreadLocal</code>	gerçekten değiştirilebilir iş parçacığı başına durum: tekrar kullanılan bir tampon

Dört yanlış kullanım

`CommonMisuses.java` dosyasından:

- Onları havuzlamak.** `newFixedThreadPool(200, Thread.ofVirtual().factory())` sizi 200 ile sınırlar ve hiçbir şey kazandırmaz. Havuzlar pahalı bir kaynağı tekrar kullanır; bunlar pahalı değil.
- Havuzun aynı zamanda hız sınırınız olduğunu unutmak.** Bir platform havuzu iki iş yapıyordu: iş parçacığı sağlamak ve eşzamanlılığı sınırlamak. Sanal iş parçacıkları yalnızca birincisinin yerine geçer. 10'luk bir havuz veritabanınızı koruyorsa, onu kaldırmak korumayı kaldırır ve bunu sabah üçte öğrenirsiniz. Kısıtı ima etmek yerine belirten bir `Semaphore` kullanın.
- CPU ağırlıklı iş.** Aritmetik yapan on bin sanal iş parçacığı çekirdek sayınızı yenmez; yalnızca zamanlama maliyeti ekler.
- Ölçekte `ThreadLocal`.** İş parçacığı başına bir girdi, ve artık bir milyon tane var.

Sanal iş parçacıkları bloke olmayı ucuzlatır. Paylaşmayı güvenli kılmazlar, ve bir CPU çarpanı değildirler. Modül 18'deki yarışlar, görünürlük ve kilitler hakkındaki her şey kelimesi kelimesine geçerlidir.

Çalıştırın

```
java modules/19-virtual-threads/examples/VirtualThreads.java
java modules/19-virtual-threads/examples/Benchmark.java
java modules/19-virtual-threads/examples/PinningToday.java
java modules/19-virtual-threads/examples/ScopedValues.java
java modules/19-virtual-threads/examples/CommonMisuses.java

./scripts/verify-examples.sh modules/19-virtual-threads
```

Kıyaslama yaklaşık on beş saniye sürer, çoğu bilerek platform havuzu durumlarında beklemekle geçer.

Sık Yapılan Hatalar

Sanal iş parçacıklarını havuzlamak. Görev başına bir tane; tekrar kullanılacak bir şey yok.

İstemeden bir hız sınırını kaybetmek. Eşzamanlılığı `Semaphore` ile açıkça sınırlayın.

Onları CPU ağırlıklı iş için kullanmak. Sınır hâlâ çekirdekler.

Ölçekte `ThreadLocal` tutmak. İş parçacığı başına bir girdi, ve artık milyonlarca var.

`synchronized` kullanmaktan kaçınma tavsiyesine uymak. Bu JDK 24 öncesinde doğrudu, şimdi değil. Varsaymak yerine ölçün.

Eşzamanlılık hatalarını düzelttiklerini sanmak. İş parçacığı *maliyetini* düzeltirler. Yarışlar ve görünürlük el değmeden durur.

Bir çocuk iş parçacığının `ScopedValue` miras almasını beklemek. Yapılandırılmış eşzamanlılık olmadan almaz.

Özet Çıkarımlar

- **Sanal iş parçacığı bloke olduğunda iner**, taşıyıcısını serbest bırakarak. Tüm fikir bu.
- **Havuz değil, görev başına bir iş parçacığı.**
- **JEP 491, JDK 24'te `synchronized` sabitletmesini kaldırdı.** Yerel çağrılar hâlâ sabitler.
- **Sabitlenme taşıyıcı harcar; çekişme zaman harcar.** Farklı problemler.
- **`ScopedValue` değişmez ve kapsamla sınırlıdır**, temizlenecek hiçbir şey yok.
- **Bloke olmayı ucuzlatırlar, paylaşmayı güvenli kılmazlar.** Modül 18 tümüyle geçerli kalır.

Ödev

[homework/README.md](#)

Farkı kendiniz ölçün, sonra kazara kaldırdığınız hız sınırını bulun. Referans çözüm [solutions/19-virtual-threads/](#) altında.

Ödev 19: Bir İndirme Servisi

Farkı kendiniz ölçün, sonra fark etmeden kaldırdığınız hız sınırını bulun.

Birinci Bölüm: Ölçün

`DownloadService.java` dosyasını klasik biçimde oluşturun.

Her biri 100ms bloke olan 2.000 simüle indirmeyi üç biçimde çalıştırın:

1. `newVirtualThreadPerTaskExecutor()`
2. `newFixedThreadPool(100)`
3. `newFixedThreadPool(25)`

Üç zamanlamayı da yazdırın.

Sonra iki platform sayısını bakmadan **tahmin edin**. Formül basit ve aynı anda kaç görevin bloke olabileceğinden çıkıyor.

Tahmin ettiğiniz alt sınırları ölçümlerin yanında yazdırın.

Açıklayın

Şunları kapsayan bir yorum:

- Platform sayılarının neden `(görev / havuzBoyutu) × gecikme` olduğu
- Sanal sayının içinde neden `havuzBoyutu` olmadığı
- Asıl nokta.** Platform zamanlamaları tahmin etmek zorunda kaldığınız bir sayıya bağlı. Onu çok düşük tahmin etmenin neye, çok yüksek tahmin etmenin neye mal olduğunu söyleyin.

İkinci Bölüm: Kaldırdığınız Sınır

Alt servis **5 eşzamanlı çağrıya** dayanıyor. Eski kodunuz `newFixedThreadPool(5)` kullanıyordu, dolayısıyla o sınır kazara uygulanıyordu.

Aynı anda uçuşta olan çağrıların **tepe** sayısını kaydeden bir `ConcurrencyTracker` yazın. Sonra 40 çağrıyı üç biçimde çalıştırıp her seferinde tepeyi yazdırın:

1. 5'lik platform havuzu
2. Sanal iş parçacıkları, **başka hiçbir değişiklik olmadan**
3. Sanal iş parçacıkları artı açık bir `Semaphore(5)`

İkincisi 40'a çıkacak. Hiçbir şey başarısız olmayacak, hiçbir şey uyarmayacak.

Sayacınızın tepe değeri kendisi de yarışsız olmalı. Aynı anda tepeyi yükselten iki iş parçacığı bir güncellemeyi kaybetmemeli, ve bunu nasıl yapacağınızı çözmek alıştırmamızın bir parçası.

Açıklayın

Şunları kapsayan bir yorum:

- Eski havuz boyutunun aynı anda hangi iki ayrı kararı verdiği
- İkisi de 5 verse bile `Semaphore` seçeneğinin havuzdan neden daha iyi olduğu
- `release()` **nereye gitmeli, ve bir istisnada atlanırsa ne olur.** Başarısızlık biçimi konusunda ayrıntılı olun

Üçüncü Bölüm: Kapsam Değerleri

Bir `ScopedValue` ile bir istek kimliği bağlayın, sonra onu parametre olarak geçirmeden **iki çerçeve derinde** okuyun.

Kapsamın dışında bağlı olmadığını gösterin.

Sonra kapsamın içinde bir çocuk iş parçacığı başlatın ve bağlamayı görüp görmediğini gösterin.

Açıklayın

Şunlar hakkında bir yorum:

- Bir `ThreadLocal` ile karşılaştırıldığında **yazmak zorunda kalmadığınız** şey
- Bir `ThreadLocal` metodunun istek bittikten sonra hâlâ neyi tutacağı, ve iş parçacığı tekrar kullanıldığında bunun neden önemli olduğu
- Bir çocuk iş parçacığının bağlamayı neden miras almadığı, ve alması için neyin gerektiği

Dördüncü Bölüm: Değişmeyen Şey

Düz bir `int` alanı her biri 100.000 kez artıran dört sanal iş parçacığı çalıştırın.

Beklenen ile gerçekleşeni yazdırın.

Sanal iş parçacıklarının burada neden yardımcı olmadığını, ve on bin iş parçacığı oluşturmak artık kolay olduğuna göre bu hatanın riskinin **arttığını mı azaldığını mı** bir yorumda yazın.

Kabul Kriterleri

- ☐ `java DownloadService.java` ile çalışıyor
- ☐ Üç zamanlama, tahmin edilen alt sınırlarla birlikte yazdırılıyor

- ☐ Açıklama havuz boyutunu yanlış tahmin etmenin her iki yönünü de kapsıyor
- ☐ `ConcurrencyTracker` bir tepe kaydediyor ve kendisi yarışsız
- ☐ Üç tepe değeri 5, 40 ve 5
- ☐ Bir yorum havuz boyutunun birleştirdiği iki kararı isimlendiriyor
- ☐ `release()` bir `finally` içinde, değilse oluşacak sızıntıyı anlatan yorumla
- ☐ Bir kapsam değeri, parametre geçirmeden iki çerçeve derinde okunuyor
- ☐ Kapsam dışında bağlı olmadığı gösteriliyor
- ☐ Çocuk iş parçacığı davranışı gösteriliyor ve açıklanıyor
- ☐ Güvensiz sayacın sanal iş parçacıkları altında artış kaybettiği gösteriliyor

İleri Seviye

`Semaphore` yerine sınırlı bir kuyruk koyun: iş gönderen bir üretici ve onu boşaltan sabit sayıda sanal iş parçacığı tüketicisi.

Sonra ikisini karşılaştıran bir yorum yazın. Hangisi gönderene geri basınç uyguluyor? İş, işlenebileceğinden hızlı geldiğinde her birinde ne oluyor, ve hangi başarısızlığı hata ayıklamayı tercih edersiniz?

İpucu, sayacınızın tepe değeri düşük görünüyorsa

`peak = Math.max(peak, current)` bir oku-değiştir-yaz işlemidir, ki bu Modül 18'in yarışının farklı değişken adlarıyla hâlidir. `AtomicInteger` sınıfında kazanana kadar yeniden deneyen bir metot var; bulun.

Modül 20: Paketler ve Modüller

Paketler adları düzenler. Sınıf yolu JVM'e nereye bakacağını söyler. Modüller ikisinin üstüne zorlama ekler.

Burada ihtiyacınız olanın çoğu pratik: bir build dosyasını okumaya, bir başlangıç hatasını teşhis etmeye ve

`--add-opens` seçeneğinin neden var olduğunu anlamaya yetecek kadar.

Neler Öğreneceksiniz

- Paketten dizine kural, ve gerçek bir erişim seviyesi olarak paket-özel
- Sınıf yolu, ve aynı şey olmayan iki başlangıç hatası
- Modül içe aktarma bildirimleri, ve ne zaman kullanılmayacağı
- Bir jar kurmak, ve bir modülün jar'ın yapamadığı neyi eklediği

Başka Bir Dilden Geliyorsanız

	Python	Java
Ad alanı	modül = dosya, paket = izin	dosyada bildirilen paket
Düzen zorunluluğu	import yolu ile	derleyici tarafından
Arama yolu	<code>sys.path</code>	sınıf yolu ya da modül yolu
Her şeyi içe aktarma	<code>from x import *</code>	<code>import x.*</code> ya da <code>import module m</code>
Dosyalar arası gizlilik	alt çizgi geleneği	paket-özel, zorlanan
Dağıtım birimi	wheel	jar

İnsanları yakalayan satır zorunlu düzen. Java'da `package` ifadesi ile izin yolu eşleşmek **zorundadır**, ve derleyici bunu size söyler.

Ders

Paketler

```
package com.example.util;    ->    com/example/util/Greeter.java
```

Derleyici dizini, kendisine verilmeyen kaynakları bulmak için kullanır; JVM de çalışma zamanında sınıfları bulmak için.

`Packages.java` geçici bir dizinde küçük iki paketli bir proje kurar, gerçek `javac` ile derler ve çalıştırır:

```
javac exit code: 0
produced: [classes/com/example/app/Main.class, classes/com/example/util/Greeter.class]
<< Hello, world >>
```

Adlandırma: ters alan adı, tamamı küçük harf. Ters çevirme, iki kuruluşun çakışmaması için var.

Gerçek kod için asla varsayılan paketi kullanmayın. `package` ifadesi olmayan bir sınıf, paketi olan hiçbir sınıf tarafından içe aktarılamaz, dolayısıyla düzenli hiçbir yerden kullanılamaz.

Paket-özel gerçek bir erişim seviyesidir, ve insanların unuttuğu olanıdır:

Seviye	Erişim
<code>private</code>	yalnızca bu sınıf
(hiçbiri)	bu paket
<code>protected</code>	bu paket artı her yerdeki alt sınıflar
<code>public</code>	herkes

İşbirliği yapan sınıfların, bir şeyi dünyaya açmadan paylaşma biçimi budur.

Sınıf yolu

Dizin ve jar dosyalarından oluşan, **sırayla aranan** bir liste:

```
java -cp classes:lib/one.jar:lib/two.jar com.example.Main
java -cp "classes;lib/*" com.example.Main (Windows ; kullanır)
```

İki hata aynı şey değildir. `TheClasspath.java` dosyasından:

ClassNotFoundException denetlenen bir istisnadır. Bir şey çalışma zamanında bir sınıfı *ismen* istedi ve bulunamadı. Genellikle yansıma, bir JDBC sürücüsü ya da yapılandırılmış bir sınıfı yükleyen bir çatı.

NoClassDefFoundError bir Error'dur. Sınıf derleme zamanında mevcuttu ve şimdi eksik ya da kullanılamaz durumda. Ve insanların yanlış teşhis ettiği durum şu:

```
first attempt: ExceptionInInitializerError, caused by java.lang.IllegalStateException:
the static initialiser failed
second attempt: NoClassDefFoundError: Could not initialize class TheClasspath$Broken
```

Statik ilkclendiricisi fırlatan bir sınıf gerçek sebebi **bir kez** bildirir. Sonraki her deneme hiçbir ipucu olmadan `NoClassDefFoundError` der.

Sınıf yolunda olduğundan emin olduğunuz bir sınıf için `NoClassDefFoundError` görüyorsanız, logda **yukarı** kaydırın. Gerçek başarısızlık daha önce bildirildi ve ilgisiz görünüyordu.

Kimsenin beklemediği başarısızlık biçimi: aynı sınıfı içeren iki jar. Sınıf yolu sırayla aranır, ilki kazanır, ikincisi sessizce yok sayılır. Belirtiler, var olmayan bir metot ya da değiştirdiğinizi sandığınız bir sürümden gelen davranış. Modül sisteminin çözmek için kurulduğu problem budur.

Modül içe aktarma bildirimleri

JEP 511, JDK 25'te kesinleşti:

```
import module java.base;
```

Tek satır, o modülün dışa açtığı her public paketi içe aktarır. `ModuleImports.java` dosyasından, başka hiçbir import olmadan:

```
names:    [ada, grace, alan]
path:     /tmp/example.txt
time:     2026-09-23
```

Bu bir **derleme zamanı kolaylığıdır**. Bytecode birebir aynıdır.

Script'ler, tek dosyalık programlar ve öğretim örnekleri için **kullanın**. Gerçek bir kod tabanında **kullanmayın**: açık import'lar bir dosyanın neye bağlı olduğunu belgeler, böylece inceleyen kişi bir dosyanın yeni bir yere uzanmaya başladığını görebilir. Ayrıca iki modül aynı basit adı dışa açtığında belirsizlik mümkün hâle gelir.

Jar'lar ve modüller

`JarsAndModules.java` dosyasından:

```
jar --create --file tool.jar --main-class com.example.tool.Tool -C classes .
```

```
META-INF/MANIFEST.MF
com/example/tool/Tool.class
```

`MANIFEST.MF`, `--main-class` seçeneğinin yazdığı şeydir, ve `java -jar` komutunun nereden başlayacağını bilmesinin sebebi budur.

Bir **modül** bir bildirim ekler:

```
module com.example.tool {
    requires java.logging;    // what it needs
    exports com.example.tool; // what others may use
}
```

Düz bir jar'ın yapamadığı üç şey:

1. **Güçlü kapsülleme**. Dışa açılmamış bir pakete dışarıdan erişilemez, yansıma ile bile. `public` artık evrensel olarak erişilebilir anlamına gelmez.

2. **Başlangıçta denetlenen bağımlılıklar.** Eksik bir `requires`, üretimde bir saat sonra `NoClassDefFoundError` olarak değil, JVM başlarken hata olarak ortaya çıkar.
3. **Bölünmüş paket yok.** İki modül aynı paketi içeremez, ki bu tam olarak yukarıda anlatılan sessiz gölgelemedir.

Çoğu projenin hâlâ neden sınıf yolu kullandığı: modülerleştirmek, her bağımlılığın da modüler olmasını ya da otomatik modül sayılmasını gerektirir, ki bu size kapsülleme olmadan ad denetimi verir.

Çoğu geliştiricinin modül sistemiyle gerçekte karşılaştığı yer: bir kütüphanenin `java.base` içine yansıma yapmasından kaynaklanan `InaccessibleObjectException`, `--add-opens` ile çözülen. Pratik asgari şart o bayrağın neden var olduğunu bilmek: `java.base` içini size dışa açmıyor, ve `--add-opens` bunu geçersiz kılıyor.

Araçlar

```
jar --list --file x.jar           # içine bakın
jar --describe-module --file x.jar # modüler mi
jdeps --list-deps x.jar          # neye ihtiyacı var
javadoc -d docs src/**/*.java   # dokümantasyon üretin
```

Çalıştırın

```
java modules/20-packages-and-modules/examples/Packages.java
java modules/20-packages-and-modules/examples/TheClasspath.java
java modules/20-packages-and-modules/examples/ModuleImports.java
java modules/20-packages-and-modules/examples/JarsAndModules.java

./scripts/verify-examples.sh modules/20-packages-and-modules
```

Bunlardan ikisi geçici bir dizinde gerçek bir paketli proje kurar, `javac` ile derler ve sonra siler. Geride hiçbir şey kalmaz.

Sık Yapılan Hatalar

Paket ifadesinin dizinle eşleşmemesi. Derleyici bunu söylüyor; onunla kavga etmek yerine inanın.

Varsayılan paketi kullanmak. Paketi olan hiçbir şey sizi içe aktaramaz.

Refleksle her şeyi `public` yapmak. Paket-özel var ve işbirliği yapan sınıflar için genellikle doğru olan o.

`NoClassDefFoundError` hatasını **eksik jar diye teşhis etmek**, oysa daha önce bir statik ilklendirici başarısız olmuş.

Sınıf yolunda bir kütüphanenin iki sürümü. İlki kazanır, sessizce.

Üretim kodunda `import module` . Bir dosyanın neye bağlı olduğunu gizler.

`--add-opens` seçeneğini anlamadan eklemek. Başka bir modülün kapsüllemesinde açılmış bir deliktir. Bazen gerekli, asla bedava.

Özet Çıkarımlar

- **Paket ifadesi ile dizin eşleşmelidir**, ve derleyici bunu zorlar.
- **Paket-özel varsayılan erişim seviyesidir**, bir seviyenin yokluğu değil.
- **Sınıf yolu sırayla aranır**, dolayısıyla tekrarlanan bir sınıf sessizce gölgelenir.
- `ClassNotFoundException` ismen istendi demektir; `NoClassDefFoundError` derleme zamanında oradaydı demektir. Başarısız bir statik ilklendirici ikincisine sebep olur ve birincisini gizler.
- `import module` script'ler için bir derleme zamanı kolaylığıdır, kod tabanları için değil.
- **Modüller kapsülleme, başlangıç zamanında bağımlılık denetimi ve bölünmüş paket yasağı** ekler, ki `--add-opens` bu yüzden var.

Ödev

[homework/README.md](#)

Sıfırdan iki paketli bir proje kurun, jar'layın, sonra üç başlangıç hatasını teşhis edin. Referans çözüm [solutions/20-packages-and-modules/](#) altında.

Ödev 20: Kendiniz Kurun

Sıfırdan gerçek bir iki paketli proje kurun, paketleyin, sonra üç başlangıç hatasını yeniden üretin ki her birini bilerek görmüş olun.

Her şey, program çıkmadan önce silinen geçici bir dizinde gerçekleşmeli.

Birinci Bölüm: İki Paket

`ProjectBuilder.java` dosyasını klasik biçimde oluşturun.

Şu düzene sahip küçük bir projeyi **yazsın, derlesin ve çalıştırsın**:

```
src/com/example/model/Item.java
src/com/example/app/Inventory.java
```

`Item`, ad ve miktar içeren bir record. Şunlara sahip olmalı:

- Diğer paketlerin çağırdığı bir **public** metot
- Public metodun kullandığı bir **paket-özel** yardımcı

`Inventory` sınıfı `Item` tipini içe aktarır, birkaç tane kurar ve yazdırır.

`ToolProvider` üzerinden `javac` ile derleyin, üretilen sınıf dosyalarını listeleyin, sonra çıktı dizinine yönlendirilmiş bir `URLClassLoader` ile `Inventory` sınıfını yükleyip çalıştırın.

`Inventory` içine, paket-özel yardımcıyı çağırmaya kalkarsa **derlenmeyecek** satırı gerçek hata metniyle birlikte gösteren bir yorum ekleyin. Kontrol edin.

Açıklayın

Dizin yolunun paket ifadesini neden yansıtmak zorunda olduğunu bir yorumda yazın. `javac` metodunun onu ne için, JVM'in ne için kullandığını söyleyin. Aynı sebep değil.

İkinci Bölüm: Paketleyin

`--main-class` ayarlanmış bir jar kurun, sonra içeriğini listeleyin.

Ayrıca ona `jar --describe-module` çalıştırın ve geleni yazdırın.

Açıklayın

İki yorum:

1. `--main-class` **gerçekte ne yaptı.** İçine yazdığı dosyayı isimlendirin ve o olmadan neyin hâlâ çalışacağını söyleyin.
2. `--describe-module` **ne bildirdi.** Bir `module-info.java` yazmadınız, peki araç neyi, nereden türetti, ve bu neden gerçek modülerlik değil?

Üçüncü Bölüm: Üç Başlangıç Hatası

Her birini yeniden üretin, yakaladığınızı yazdırın ve açıklayın:

1. **Hiç var olmamış bir sınıf için** `ClassNotFoundException`. Derleyicinin sizi neden uyaramayacağını söyleyin ve buna yol açan iki gerçek durumu isimlendirin.

2. **Var olan ama yükleyicinin yolunda olmayan bir sınıf için** `ClassNotFoundException`. Boş bir `URLClassLoader` kurun ve ondan az önce derlediğiniz bir sınıfı isteyin.

Sonra ilginç yorumu yazın: belirtilen birinci hatayla birebir aynı ve sebep tamamen farklı. Bunun gerçek bir dağıtımda neye karşılık geldiğini söyleyin.

3. **Başarısız bir statik ilklendiriciden** `NoClassDefFoundError`. Sınıfa iki kez dokununuz ve her iki sonucu da yazdırın.

Doğru anlaşılmaya değer olan bu. Şunları kapsayan bir yorum yazın:

- İlk denemenin ne bildirdiği ve gerçek sebebin nerede saklandığı
- İkinci denemenin neden daha az kullanışlı bir şey bildirdiği
- **Teşhis kuralı.** Orada olduğundan emin olduğunuz bir sınıf için `NoClassDefFoundError` gördüğünüzde ne yapmalısınız?

Dördüncü Bölüm: Pratikte Modüller

Etiketlerle birlikte şunları yazdırın:

- `String` sınıfının sınıf yükleyicisi ve oradaki `null` değerinin ne anlama geldiği
- Kendi sınıfınızın yükleyicisi
- `String` sınıfının modül adı
- Kendi modülünüz ve adlandırılmış olup olmadığı

Açıklayın

Şunlar hakkında bir yorum:

- **Adsız modülün** ne olduğu, neyi okuduğu ve neyi dışa açtığı
- `--add-opens` **neden var.** Ayrıntılı olun: `java.base` neyi yapmıyor, bir kütüphane bunun sonucunda hangi istisnayı alıyor, ve bayrak neyi değiştiriyor?

Kabul Kriterleri

- ☐ `java ProjectBuilder.java` ile çalışıyor ve geçici dizinini siliyor
- ☐ İki paket yazılıyor, derleniyor ve çalıştırılıyor
- ☐ `Item` hem public hem paket-özel bir üyeye sahip
- ☐ Derlenmeyen çağrı gerçek hata metniyle belgelenmiş
- ☐ Bir yorum izin kuralını hem `javac` hem JVM tarafından açıklıyor
- ☐ `--main-class` ile bir jar kuruluyor ve içeriği listeleniyor
- ☐ `--describe-module` çıktısı yazdırılıyor ve açıklanıyor
- ☐ Üç başarısızlık da gerçek mesajlarıyla yeniden üretiliyor
- ☐ Birinci ve ikinci başarısızlıklar belirtile değil sebeple ayırt ediliyor
- ☐ Statik ilklendirici durumu her iki denemeyi de gösteriyor
- ☐ `NoClassDefFoundError` için bir teşhis kuralı belirtiliyor
- ☐ Adsız modül ve `--add-opens` ikisi de açıklanıyor

İleri Seviye

Yalnızca `com.example.model` paketini dışa açan, `com.example.app` paketini **açmayan** gerçek bir `module-info.java` ekleyin. Modül olarak derleyin ve `--module-path` ile çalıştırın.

Sonra dışarıdan, dışa açılmamış pakete yansıma yapmayı deneyin ve ne aldığınızı kaydedin.

Güçlü kapsüllemenin ikinci bölümdeki jar'ın vermediği neyi verdiğini ve karşılığında neye mal olduğunu bir yorumda yazın.

İpucu, ikinci başarısızlık başarısız olmuyorsa

Sıradan bir `new URLClassLoader(urls)` önce ebeveynine devreder, ve ebeveyn muhtemelen sınıfınızı bulabilir. Devretmeyi durdurmak için ebeveyn olarak `null` geçirin, o zaman yükleyicinin gerçekten bakacak hiçbir yeri kalmaz.

Modül 21: Kodunuzu Test Etmek

Bu müfredatın sıfır bağımlılık kuralı var, ve JUnit bir bağımlılık. Dolayısıyla bu modül testi çatısız anlatıyor, ki bu zaten daha kullanışlı ders olduğu ortaya çıkıyor: bir test çatısı sihir değildir, ve çalışan bir tanesinin tamamını görmek gerçek olanları anlamayı kolaylaştırır.

Ayrıca `assert` konusunu kapsıyor, çoğunlukla onun neden bir test aracı **olmadığını** bilmeniz için.

Neler Öğreneceksiniz

- Programınızdaki her `assert` ifadesinin neden varsayılan olarak hiçbir şey yapmadığı
- `assert` ifadesinin yanlış kullanıldığı iki yol, biri gerçek bir üretim hatası
- 39 satırda eksiksiz bir test çatısı
- Bir çatının, makul biçimde elle yazamayacağınız neyi kattığı

Başka Bir Dilden Geliyorsanız

	Python	Java
Yerleşik test koşucusu	<code>unittest</code> , <code>pytest</code>	JDK'da yok
Testlerde <code>assert</code>	normal yol	varsayılan olarak kapalı
Keşif	adlandırma geleneğiyle	anotasyonla, bir çatı üzerinden
Standart tercih	<code>pytest</code>	JUnit 5, fiilen evrensel

Önemli olan satır: Python'un `assert` ifadesi aksini istemedikçe açıktır. Java'nınki istemedikçe kapalıdır, ve neredeyse kimse istemez.

Ders

Assert'ler kapalı

`AssertionsAreOff.java` dosyasından:

```
-ea olmadan:  assertions enabled: false
-ea ile:      assertions enabled: true
```

Tespit, tanınmaya değer bir numara kullanıyor:

```
boolean enabled = false;
assert enabled = true;
```

Atama yalnızca assert'ler etkinse çalışır, çünkü aksi hâlde tüm ifade atlanır. Bir `assert` içindeki yan etkinin doğru olduğu çok az yerden biridir.

Buradan iki kural çıkar.

Bir assert içine asla mantık koymayın. Bu gerçek bir hatadır:

```
assert list.remove(item);
```

`-ea` ile testlerde çalışır ve üretimde sessizce hiçbir şey silmemeye başlar.

Çağırılardan gelen argümanları doğrulamak için asla `assert` kullanmayın. Her zaman açık olan bir istisna kullanın:

```
if (n < 0) throw new IllegalArgumentException("n must not be negative");
```

`assert`, **yanlış olamayacağına inandığınız iç değişmezler** içindir: erişilemez bir switch default'u, kodunuzun az önce kurduğu bir durum. `UsingAssertions.java` ikisini de gösteriyor ve mesajlı biçimi kullanıyor, ki bu tek bir string'e mal olur ve mesajsız bir `AssertionError` hatasını kendini açıklayan birine çevirir.

Bu depo hakkında bir not: doğrulama scripti örnekleri düz `java <dosya>` ile çalıştırır, dolayısıyla assert'ler kapalıdır. `UsingAssertions.java` dosyası `// EXPECT: assertions` işaretleyicisi taşır, böylece script onu `-ea` ile çalıştırır. Bu olmadan `assert` öğreten bir örnek hiçbir şey yapmadan geçerdi.

39 satırda bir test çatısı

`MicroHarness.java` eksiksiz bir `Tests` sınıfı içerir. Gerçek çıktısı:

```
PASS    addition works
PASS    string concatenation
PASS    list size
FAIL    this one is wrong on purpose
        returned false
FAIL    this one throws
        IllegalStateException: boom
PASS    equality with a useful message
FAIL    and one that fails
        expected <5> but was <4>

7 run, 4 passed, 3 failed
```

Fark edilmeye değer üç tasarım kararı, çünkü her gerçek çatı aynılarını verir:

1. **Fırlatan bir test, çalıştırmanın çökmesi değil bir başarısızlıktır.** Çatı `Throwable` yakalar, ki bu bunun doğru olduğu az sayıda yerden biridir: bir testin ona yapabileceği her şeyden sağ çıkmak zorundadır.

2. **Başarısızlıklar bir ayrıntı taşır**, dolayısıyla çıplak bir `FAIL` yerine `expected <5> but was <4>` görünür.
3. **Gerçek bir çatı, bir şey başarısız olduğunda sıfır dışı çıkar**, böylece CI fark eder. Bu çatı çıkmıyor, çünkü bu deponun kendi doğrulamasından temiz geçmek zorunda.

Bir çatının neler kattığı

`WhatJUnitAdds.java` dosyasından:

1. Keşif. Mikro çatı her testin elle kaydedilmesini gerektirir. Bir satır unutun ve test sessizce var olmaz, ve hiçbir şey hiç kaydedilmemiş bir testi bildirmez. JUnit `@Test` anotasyonunu tarar ve o başarısızlık biçimi kaybolur.

2. Yalıtım. Her `@Test` yeni bir örnek alır, dolayısıyla durum testler arasında sızamaz. Onsuz:

```
test one added an item; list size is now 1
test two starts with size 1, not 0
```

Yalnızca önceki çalıştığı için geçen bir test, hiç test olmamasından daha kötüdür, çünkü biri sıralamayı değiştirdiğinde başarısız olur.

3. Assert mesajları. On alanlı bir nesnede farklı alanı gösteren `assertEquals`, hata ayıklama oturumunun tamamıdır.

4. Elle yazmaya yer olmayan şeyler: parametrelili testler, `assertThrows`, zaman aşımaları, etiketleme, IDE entegrasyonu ve CI'nın okuyabileceği standart bir rapor biçimi.

```
class CalculatorTest {
    @Test
    void addsTwoNumbers() {
        assertEquals(4, Calculator.add(2, 2));
    }

    @Test
    void rejectsNegativeInput() {
        assertThrows(IllegalArgumentException.class, () -> Calculator.sqrt(-1));
    }
}
```

Gerçek bir projede ilk gün JUnit 5 ekleyin. Her yerde varsayılan ve onunla ilgili tartışmalı hiçbir şey yok. Bu modülden taşınan şey çatı değil: bir testin bir şeyi denetleyip bildiren bir metod olduğunu, testler arası yalıtımın bilinçli bir özellik olduğunu ve `assert` ifadesinin bir test aracı olmadığını bilmek.

Çalıştırın

```
java modules/21-testing-your-code/examples/AssertionsAreOff.java
java -ea modules/21-testing-your-code/examples/AssertionsAreOff.java

java -ea modules/21-testing-your-code/examples/UsingAssertions.java
java modules/21-testing-your-code/examples/MicroHarness.java
java modules/21-testing-your-code/examples/WhatJUnitAdds.java

./scripts/verify-examples.sh modules/21-testing-your-code
```

Birincisini her iki biçimde de çalıştırın. Fark, dersin kendisi.

Sık Yapılan Hatalar

`assert` ifadesinin çalıştığını varsaymak. Biri `-ea` geçirmedikçe çalışmaz.

Bir `assert` içinde yan etkiler. `assert list.remove(x)` üretimde çalışmayı durdurur.

Çağırının argümanlarını `assert` ile doğrulamak. Bir istisna kullanın.

Birbirine bağımlı testler. İkincisi birincisi çalıştığı için geçer, ta ki biri sıralamayı değiştirene kadar.

Başarısızlıkta ayrıntısı olmayan bir test. "FAIL" size hiçbir şey söylemez; beklenen ve gerçekleşen değerler her şeyi söyler.

İlk istisnada ölen bir çatı. Tek bir kötü test çalıştırmanın tamamını düşürmemeli.

Gerçek bir proje için kendi çatınızı yazmak. Yukarıdaki her şey JUnit kullanmanın lehine argümandır, aleyhine değil.

Özet Çıkarımlar

- **Assert'ler varsayılan olarak kapalıdır**, dolayısıyla `assert` bir test mekanizması değildir.
- Bir `assert` içine asla mantık ya da argüman doğrulaması koymayın. Yalnızca iç değişmezler.
- Bir test çatısı, bir şeyi denetleyip bildiren bir metottur. Otuz dokuz satır bunu kanıtlamaya yeter.
- Bir çatıda `Throwable` yakalayın ki tek bir kötü test çalıştırmayı bitirmesin.
- Başarısızlıklar beklenen ve gerçekleşeni taşımalı, yoksa hata ayıklama hiçbir yerden başlar.
- Gerçek projelerde JUnit kullanın. Bu modül onun neden var olduğudur, yerine geçeni değil.

Ödev

[homework/README.md](#)

Çatıyı bir framework'ün size vereceği özelliklerle genişletin, sonra `assert` ifadesinin gizlediği hatayı bulun. Referans çözüm solutions/21-testing-your-code/ altında.

Ödev 21: Aksi Hâlde İçerik Aktaracağınız Çatıyı Kendiniz Kurun

Mikro çatıyı, bir framework'ün yaptığı şeyleri yapana kadar genişletin, sonra `assert` ifadesinin üretimde gizlediği hatayı bulun.

Sıfır üçüncü parti bağımlılık. Buradaki her şey yalnızca JDK.

Birinci Bölüm: Daha İyi Bir Çatı

`TestKit.java` dosyasını klasik biçimde oluşturun.

Şunları destekleyen bir `Suite` sınıfı yazın:

- `test(String name, Runnable body)` — bir geçme ya da başarısızlık kaydeder
- `assertEquals(expected, actual)` — başarısızlık ayrıntısı **her iki** değeri de göstermeli
- `assertThrows(Class<? extends Throwable>, Runnable)` — yalnızca o tam tip fırlatılırsa geçer

`assertThrows` metodunun bir değil **iki** başarısızlık durumu var. İkisini de farklı mesajlarla ele alın:

- Hiçbir şey fırlatılmadı
- Bir şey fırlatıldı, ama yanlış tip

Sonra en az şunları içeren bir suite çalıştırın: iki geçme, başarısız bir `assertEquals`, her iki `assertThrows` başarısızlık biçimi, ve beklenmedik bir şey fırlatan bir test.

Parametrelili testler ekleyin: bir döngüde üretilen dört satır girdiye karşı denetlenen tek bir davranış. Her durumu girdileriyle adlandırın, ve ona `case 3` demenin neden daha kötü olacağını bir yorumda söyleyin.

Bir özet satırı ve gerçek bir çatının döndüreceği çıkış kodunu yazdırın.

İkinci Bölüm: Yalıtım

Küçük, değiştirilebilir bir sınıf yazın, sonra ona karşı iki denetimi **iki kez** çalıştırın:

- Aralarında tek bir örnek paylaşarak
- Her birine kendi örneğini vererek

İkinci denetimin paylaşılan sürümde başarısız olduğunu, taze sürümde geçtiğini gösterin.

Açıklayın

Sıraya bağımlı bir testin neden **hiç test olmamasından daha kötü** olduğunu bir yorumda yazın.

Cevabınız sonradan ne olduğunu, kime olduğunu ve bir testin yokluğunun neden daha dürüst olduğunu

söylemeli.

Üçüncü Bölüm: `assert` İfadesinin Gizlediği Hata

İçinde şu bulunan bir metoda sahip bir sınıf yazın:

```
assert items.remove(item) : "expected " + item + " to be present";
```

Programınızı `-ea` **olmadan** çalıştırın ve koleksiyon boyutunu öncesinde ve sonrasında yazdırın. Sonra doğru sürümü yazın ve çalıştığını gösterin.

Açıklayın

Şunları kapsayan bir yorum:

- Bunun, onun için yazacağınız her testten neden geçtiği
- Üretimde ne olduğu, ve başarısızlığın neden bir çökme gibi görünmediği
- **Bunu önleyen kural**, tek cümlede

Ayrıca tespit numarasını kullanarak `assert`'lerin etkin olup olmadığını yazdırın, böylece çıktı her iki durumda da kendini açıklasın.

Dördüncü Bölüm: Hangi Denetim, Nereye

İki metot yazın:

1. Bir **public çağırının argümanını** doğrulayan biri
2. Kendi kodunuzun az önce kurduğu bir **iç değişmezi** öne süren biri

Birincisi için istisna, ikincisi için `assert` kullanın.

Açıklayın

Aralarında karar vermek için uygulayacağınız testi bir yorumda verin. Dal başına bir cümle, ve klavye başında gerçekten cevaplayabileceğiniz bir soru olmalı.

Kabul Kriterleri

- ☐ `java TestKit.java` ile çalışıyor, üçüncü parti import yok
- ☐ `Suite` sınıfı `test`, `assertEquals` ve `assertThrows` destekliyor
- ☐ `assertThrows` "hiçbir şey fırlatılmadı" ile "yanlış tip fırlatıldı" durumlarını ayırıyor
- ☐ Beklenmedik biçimde fırlatan bir test, çalıştırmanın çökmesi değil bir başarısızlık
- ☐ Parametrelili durumlar girdileriyle adlandırılmış

- ☐ Bir yorum `case 3` adının neden daha kötü olacağını söylüyor
- ☐ Özet sayıları, artı gerçek bir çatinın kullanacağı çıkış kodunu gösteriyor
- ☐ Paylaşılan durum ikinci denetimi başarısız kılıyor; taze durum geçiriyor
- ☐ Bir yorum sıraya bağımlı testlerin neden hiç testten kötü olduğunu açıklıyor
- ☐ Assert içine gizlenmiş silme, `-ea` olmadan **gerçekleşmediği** gösteriliyor
- ☐ Düzeltilmiş sürümün çalıştığı gösteriliyor
- ☐ Tek cümlelik kural belirtiliyor
- ☐ İstisna ve assert her biri ait olduğu yerde kullanılıyor, karar testi yazılmış

İleri Seviye

`test` metoduna bir zaman aşımı ekleyin, böylece asılı kalan bir gövde çalıştırmayı asmak yerine başarısızlık olarak bildirilsin.

Sonra bunu çalıştırmak için neyi devreye sokmak zorunda kaldığınızı ve bir zaman aşımının neden görüldüğünden daha zor olduğunu bir yorumda yazın. Bildirdikten sonra asılı testi çalıştıran iş parçacığına ne olduğunu düşünün.

İpucu, üçüncü bölümde silme gerçekleşiyorsa

Nasıl çalıştırdığınıza bakın. `-ea` ile assert çalışır ve silme işler, ki bu hatanın testten sağ çıkmasının tam sebebi. Üretimin çalıştığı gibi, bayraksız çalıştırın.

Modül 22: Yenilikler, Dürüstçe

Bu modül, müfredatın kendi başlığı konusunda dürüst olduğu yer.

Modül 01, JDK 27'nin çalışma ortamı ve araç zinciri olduğunu, öğrendiğiniz dilin ise JDK 25'e kadar birikmiş kararlı Java olduğunu iddia etmişti. Yirmi bir modül sonra, işte kanıtı.

Aşağıdaki her sürüm iddiası kaynağına bağlanıyor. Bu alışkanlık bu modülün asıl konusu, çünkü içindeki diğer her şey güncelliğini yitirecek.

Neler Öğreneceksiniz

- JDK 26 ve 27'nin gerçekte ne teslim ettiği, sürüm sayfalarından
- Altı aylık ritim, LTS, ve hangi numaranın ne için önemli olduğu
- Bir JEP nasıl okunur, ve preview tur numarasının size ne söylediği
- Yapılandırılmış eşzamanlılık ve ilkel tip desenleri, `--enable-preview` arkasında çalıştırılmış

Başka Bir Dilden Geliyorsanız

	Python	Node.js	Java
Yayın ritmi	~yıllık	~altı ayda bir majör	altı ayda bir, sabit tarih
Uzun dönem destek	minör başına ~5 yıl	çift numaralı majörler, 30 ay	iki yılda bir LTS
Bitmemiş özellikleri denemek	<code>from __future__ import</code>	bayraklar ya da bir yayın kanalı	<code>--enable-preview</code>
Spesifikasyonun yaşadığı yer	PEP'ler	TC39 önerileri	JEP'ler

En yakın benzeri TC39'un aşamalı önerileri. Bir Java preview'ı kabaca aşama 3 önerisine denk: belirtilmiş, uygulanmış, bayrak arkasında yayımlanmış, ve yerleşmeden önce hâlâ değişebilir.

Bilinmeye değer fark: Python'un `__future__` import'u ileri uyumludur, Java'da ise bir **preview sınıf dosyası** farklı bir sürüm tarafından, bayrakla bile doğrudan reddedilir. Bu tek olgu, bu modülün tavsiye ettiklerinin çoğunu belirliyor.

Ders

Gerçekte ne yayımlandı

JDK 26, GA 17 Mart 2026:

JEP	Başlık	Durum
500	Prepare to Make Final Mean Final	Final
504	Remove the Applet API	Final
516	Ahead-of-Time Object Caching with Any GC	Final
517	HTTP/3 for the HTTP Client API	Final
522	G1 GC: Improve Throughput by Reducing Synchronization	Final
524	PEM Encodings of Cryptographic Objects	İkinci Preview
525	Structured Concurrency	Altıncı Preview
526	Lazy Constants	İkinci Preview
529	Vector API	On Birinci İnkübatör
530	Primitive Types in Patterns	Dördüncü Preview

JDK 27, GA 15 Eylül 2026:

JEP	Başlık	Durum
523	Make G1 the Default Garbage Collector in All Environments	Final
527	Post-Quantum Hybrid Key Exchange for TLS 1.3	Final
531	Lazy Constants	Üçüncü Preview
532	Primitive Types in Patterns	Beşinci Preview
533	Structured Concurrency	Yedinci Preview
534	Compact Object Headers by Default	Final
536	JFR In-Process Data Redaction	Final
537	Vector API	On İkinci İnkübatör
538	PEM Encodings of Cryptographic Objects	Üçüncü Preview

JDK 27'de Java yazma biçiminizi değiştiren hiçbir şey yok. Kesinleşen beş JEP'inden dördü çalışma zamanı ve güvenlik işi: bir çöp toplayıcı varsayılını, küçültülmüş nesne başlıkları, TLS anahtar değişimi, uçuş kaydedici maskeleye. Dördünden de tek satır düzenlemeden faydalanıyorsunuz. Dil biçimli her şey hâlâ preview, bazıları yedinci ardışık sürüm boyunca.

"Java 27'nin yeni özellikleri" vaat eden bir kurs ya listeyi şişirmek ya da preview özellikleri yerleşmiş gibi anlatmak zorunda kalırdı. İkisi de böyle söylemekten kötü.

JDK 26'da bilinmeye değer iki tanesi: [JEP 517](#) yerleşik `HttpClient` sınıfına HTTP/3 kazandırıyor, bir bağımlılık eklemek yerine bir sürüm isteği değiştirerek aldığınız gerçek bir yetenek. [JEP 500](#) ise kodun

yansımayla bir `final` alanı deęiřtirmesi durumunda, bunu yasaklamaya hazırlık olarak uyarıyor; hedefi bazı serileřtirme ve taklit kütüphaneleri.

Sürümler ve LTS

Altı ayda bir, Mart ve Eylül'de, bir řey hazır olsun olmasın sabit bir tarihte bir özellik sürümü. 2017 öncesinde sürümler özellik odaklıydı ve iki üç yılda bir geliyordu.

Sürüm	Tanım	Tarih
Java 17	LTS	Eylül 2021
Java 21	LTS	Eylül 2023
Java 25	LTS	Eylül 2025
Java 26		Mart 2026
Java 27		Eylül 2026

LTS olmayan bir sürüm, bir sonraki sürüm onun yerini alana kadar güvenlik yaması alır. Altı ay, sonra hiçbir řey. Bir LTS ise yıllarca alır, üretim sistemlerinin neredeyse hepsinin birini çalıştırmasının sebebi budur.

"Java 27'de ne yeni" çoęu ekip için yanlış sorudur. Doğrusu "Java 21'den beri ne yeni", çünkü gerçekte bulundukları yer orası.

Durum	Kullanın
Öğrenme	en yeni sürüm. Özellikler önce burada kesinleşir.
Üretim	güncel LTS, bugün Java 25
Bir kütüphane	desteklemek zorunda olduğunuz en eski LTS

Kimsenin fark etmedięi birikim. Herhangi bir tek sürüm cılız görünür. Java 21 ile Java 25 arasında bunların hepsi geldi, ve her biri bu müfredatta yer alıyor:

JEP	Özellik	Modül
512	Kompakt kaynak dosyaları ve örnek main metotları	01
511	Modül içe aktarma bildirimleri	20
513	Esnek kurucu gövdeleri	05
506	Kapsam deęerleri	19
485	Stream gatherer'ları	14
491	Sanal iş parçacıklarını sabitlemeden senkronize etme	19

Hangi sürümde olduğunuzu denetlemek için `java.version` metnini değil `Runtime.version()` metodunu kullanın. O metni ayırtmak, Java 9 biçimi `1.8.0_301` yerine `9.0.1` yaptığında herkes için bozuldu.

Bir JEP nasıl okunur

Java hakkında okuduğunuz her şey güncelliğini yitirir, bu dosya dahil. JEP'ler yitirmez, çünkü onlar spesifikasyonun **ta kendisidir**.

- openjdk.org/jeps/0 dizindir
- `openjdk.org/jeps/<numara>` tek bir JEP'tir
- `openjdk.org/projects/jdk/<sürüm>` bir sürümde ne yayımlandığıdır

Alanlar, kullanışlılık sırasına göre:

Alan	Nedeni
Status	yalnızca <i>Closed/Delivered</i> yayımlandı demektir
Release	Release alanı yoksa hiçbir şey için planlanmamıştır
Summary	tek paragraf, çoğu zaman ihtiyacınız olan tek şey
Motivation	<i>neden</i> var olduğu. İnsanların atladığı ve okumaya değer bölüm.
Alternatives	neyin reddedildiği. "Neden şunu yapmadılar ki" sorularının çoğunu cevaplar.

Sözcük dağarcığı:

Terim	Anlamı
Preview	eksiksiz ve belirtilmiş, henüz kalıcı değil. <code>--enable-preview</code> ister. Değişebilir ya da geri çekilebilir.
İnkübatör	bir <code>jdk.incubator</code> modülündeki API. <code>--add-modules</code> ister. Preview'dan daha az yerleşmiş.
Deneysel	<code>-XX:+UnlockExperimentalVMOptions</code> arkasındaki bir JVM özelliği
Final	kalıcı. Kaldırılmayacak.

Preview tur numarası pratik bir işarettir, ilginç bir ayrıntı değil. Yapılandırılmış eşzamanlılık yedincisinde. Switch için desen eşleme dört tur aldı. Record'lar iki. Yüksek bir numara tasarımın hâlâ tartışıldığı ve API'nin genellikle turlar arasında değiştiği anlamına gelir, ki burada tam olarak bu oldu: daha önceki yapılandırılmış eşzamanlılık preview'larına göre yazılmış kod bu sürümde derlenmiyor.

Herhangi bir özellik hakkında sorulacak üç soru:

1. Final mi? Değilse hangi tur, ve öncesinde kaç tane vardı?
2. Hangi sürüm teslim etti, ve o bir LTS mi?
3. Desteklemek zorunda olduğunuz asgari sürüm onu gerçekten kullanabilir mi?

Preview özellikler, çalıştırılmış

Aşağıdaki her iki örnek de `// EXPECT: preview` işaretleyicisi taşır, dolayısıyla doğrulama scripti onları `-enable-preview --source 27` ile çalıştırır.

Yapılandırılmış eşzamanlılık, yedinci preview. Modül 18'in `ExecutorService` sınıfı iki görev gönderip birini unutmanıza izin verir. Hiçbir şey onları birbirine bağlamaz, ve kimsenin incelemediği bir `Future` istisnasını yutar.

```
try (var scope = StructuredTaskScope.open()) {
    var user = scope.fork(() -> loadUser());
    var orders = scope.fork(() -> loadOrders());
    scope.join();
    return combine(user.get(), orders.get());
}
```

```
user:ada and orders:3

scope failed: ExecutionException
cause: the fast one failed
```

İkinci durum anında dönüyor. Hızlı görev fırlattığı anda iki saniyelik bir kardeş görev, boşuna beklenmek yerine **iptal edildi**.

Ve Modül 19'dan bir halkayı kapatıyor:

```
child sees: req-7
```

Modül 19, düz bir çocuk iş parçacığının `ScopedValue` miras almadığını ve miras almanın yapılandırılmış eşzamanlılık gerektirdiğini söylemişti. Yukarıdaki satır o. Çalışıyor çünkü kapsam, ebeveynin çocuklarından uzun yaşayacağını garanti ediyor, dolayısıyla bir çocuk okurken bağlama sona eremiyor.

İlkel tip desenleri, beşinci preview.

```
42 instanceof byte: true
300 instanceof byte: false
```

Referans tipi için `instanceof` "bu nesne o tipte mi" diye sorar. İlkel tip için farklı bir şey sorar: **bu değer kayıpsız dönüştürülebilir mi**.

Bugün o daraltma bir dönüşümdür, ve `(byte) 300` hiçbir uyarı olmadan `44` verir. Bu Modül 02'nin taşmasıdır. Desen biçimi sarmak yerine reddeder.

İkisini de üretimde kullanmalı mısınız? Hayır. Bir preview sınıf dosyası, bayrakla bile farklı bir sürüm tarafından reddedilir, dolayısıyla her JDK yükseltmesi derlemenizi kırabilir. Okuyun, deneyin, ve kesinleşecekleri turu JEP üzerinden izleyin.

Çalıştırın

```
java modules/22-whats-new/examples/WhatChanged.java
java modules/22-whats-new/examples/ReleasesAndLts.java
java modules/22-whats-new/examples/HowToReadAJep.java

java --enable-preview --source 27 modules/22-whats-
new/examples/PreviewStructuredConcurrency.java
java --enable-preview --source 27 modules/22-whats-
new/examples/PreviewPrimitivePatterns.java

./scripts/verify-examples.sh modules/22-whats-new
```

Sık Yapılan Hatalar

Üretimde preview özellikleri etkinleştirmek. Bir preview sınıf dosyası, bayrakla bile farklı bir JDK sürümü tarafından reddedilir.

Üretim sistemi için en yeni sürümün peşinden koşmak. Altı ay yama, sonra hiçbir şey.

JEP yerine bir blog yazısı okumak. O yazı farklı bir preview turuna göre yazılmıştı.

`java.version` ayarlamak. `Runtime.version()` kullanın.

Bir preview API'sinin çalıştığı için kararlı olduğunu varsaymak. Yedi tur, değişmiş olması için yedi şans demektir.

Bir sürümü JEP sayısına göre değerlendirmek. İlginç soru, son LTS'ten beri neyin biriktiğidir.

Özet Çıkarımlar

- **JDK 27, Java yazma biçiminizi değiştiren hiçbir şey teslim etmedi.** Kesinleşen işi çalışma zamanı ve güvenlik.
- **Üretim için önemli olan numara LTS'tir;** öğrenme için önemli olan en yeni sürümdür.
- **Değer,** herhangi bir tek sürümün içinde değil **LTS sürümleri arasında birikir.**
- **Bir JEP spesifikasyondur ve hakkındaki her makaleden uzun yaşar.** Motivation bölümünü okuyun.
- **Preview tur numarası size bir tasarımın ne kadar yerleştiğini söyler.**
- **Preview özellikleri asla yayımlamayın.** Sınıf dosyaları diğer sürümler tarafından reddedilir.

Ödev

[homework/README.md](#)

Bir özelliđi birincil kaynaklardan arařtırın ve kullanıp kullanamayacađınıza karar verin. Referans çözüm [solutions/22-whats-new/](#) altında.

Ödev 22: Kendiniz Araştırın

Bu müfredatın son ödevi Java yazmakla ilgili değil. "Bunu kullanabilir miyim" sorusunu birincil kaynaklardan cevaplamakla ilgili, çünkü buradaki diğer her becerinin raf ömrü var ve bunun yok.

Birinci Bölüm: Üç Özellik, Araştırılmış

`FeatureResearch.java` dosyasını klasik biçimde oluşturun.

Bu müfredatta geçen üç özellik seçin, en az biri **Final** ve en az biri **Preview** olsun. Her biri için openjdk.org üzerinden şunları bulup kaydedin:

Alan	Nerede bulunur
JEP numarası	JEP dizini ya da sürüm sayfası
Durum	JEP'in kendi başlığı
Teslim eden sürüm	Release alanı
O sürüm LTS mi?	sürüm takvimi

Bunları bir tablo olarak yazdırın, ve altına **kullandığınız bağlantıları yazdırın**. Kaynağı olmayan bir iddia burada sayılmaz.

İkinci Bölüm: Üç Soru

Modülün kontrol listesini bir özelliğe uygulayan bir metot yazın:

1. Final mi? Değilse hangi tür?
2. Hangi sürüm teslim etti, ve o bir LTS mi?
3. Desteklemek zorunda olduğunuz asgari sürüm nedir?

Bir karar yazdırmalı. **JDK 21** desteklemek zorunda olduğunuzu varsayarak bir Final ve bir Preview özelliğe karşı çalıştırın.

İlginç Sonuç

İkinizden biri Final olacak, bir LTS'te yayımlanmış olacak, ve **yine de kullanılamaz** olacak.

Nedenini açıklayan bir yorum yazın. Cevabınız bir özelliğin **güvenli** olmasıyla **erişilebilir** olması arasında ayırım yapmalı, çünkü bu ikisini karıştırmak birinin kimsenin bağımlı olamayacağı bir kütüphane yayımlama biçimidir.

Bir uygulama notu. Sürümleri metin olarak değil sayı olarak alın. `"JDK 21"` ile `"JDK 25"` metinlerini karşılaştırmak tesadüfen çalışır; `"JDK 9"` ile `"JDK 25"` karşılaştırması çalışmaz. Bunun neden

`java.version` ayırtırmakla aynı hata olduğunu bir yorumda söyleyin.

Üçüncü Bölüm: Bir Geçmiş Bölümü Okuyun

İkiden fazla preview turu geçmiş bir özellik seçin. JEP'ini açın ve **History** bölümünü bulun.

Her turu listeleyin: JEP numarası, sürüm, ve inkübatör mü preview mü olduğu.

Sonra Cevaplayın

- Inkübatör turları dahil toplam kaç tur var?
- JEP **numarası** turlar arasında değişti mi, ve bu size tekrarlanan bir numaranın söylemeyeceği neyi söylüyor?
- Okumadan önce varsaydığınıza kıyasla geçmişte sizi şaşırtan bir şey oldu mu?

Son soru alıştırmanın asıl amacı. Ne beklediğinizi ve ne bulduğunuzu yazın.

Sonra hızla kesinleşmiş bir özellikle karşılaştırın ve gelecekte kullanacağınız kestirme kuralı belirtin.

Dördüncü Bölüm: Sürümü Düzgün Denetleyin

`Runtime.version()`, `feature()` ve `java.version` özelliğini yazdırın.

Sonra asgari gerekli sürüm için bir denetim yazın.

Açıklayın

`java.version` ayırtırmanın neden tuzak olduğunu bir yorumda yazın. Ayrıntılı olun: Java 9 öncesi biçim neydi, şimdi ne, her birinde sürüm numarasını hangi bileşen ifade ediyordu, ve noktalara göre bölüp 1. indeksi alan koda ne oldu?

Kabul Kriterleri

- ☐ `java FeatureResearch.java` ile çalışıyor
- ☐ JEP numarası, durum, sürüm ve LTS bilgisiyle üç özellik
- ☐ Kaynak bağlantıları yazdırılıyor
- ☐ Üç soru metodu bir Final ve bir Preview özellik için karar yazdırıyor
- ☐ Bir özellik Final, bir LTS'te ve yine de kullanılamaz, sebebi açıklanmış
- ☐ Güvenli ile erişilebilir ayrımı açıkça belirtilmiş
- ☐ Sürümler sayı olarak karşılaştırılıyor, nedenini söyleyen yorumla
- ☐ Inkübatör turları dahil tam bir preview geçmişi listelenmiş
- ☐ JEP numarasının değişmesi işareti açıklanmış

- ☐ Ne beklediğiniz ile ne bulduğunuz yazılmış
- ☐ Gelecekteki özellikler için bir kestirme kural belirtilmiş
- ☐ `Runtime.version()` kullanılıyor, `java.version` tuzağı açıklanmış

İleri Seviye

Preview aşamasından sonra kesinleşmek yerine **geri çekilmiş** bir özellik bulun.

JEP'inin nedenini ne söylediğini, ve bunun preview sırasında onu kullanan kod yayımlamış biri için ne ima ettiğini bir yorumda yazın.

İpucu, History bölümünü bulamıyorsanız

JEP'in alt kısmına yakın, Alternatives ve Risks bölümlerinin altındadır. Belgenin en kullanışlı ve kimsenin kaydırıp gitmediği kısımdır.

Uygulama Planı: Java 27 Öğrenim Müfredatı

Aşama: Specify → Plan → Tasks → Implement dizisinin 2. aşaması (Plan) **Spesifikasyon:** [SPEC.md](#)

Tarih: 2026-09-23

Doğrulanmış Ön Koşullar

Denetim	Sonuç
JDK 27 kurulu	<code>java version "27" 2026-09-15</code> , derleme 27+35-2325
JDK 27 varsayılan	Evet, <code>/Library/Java/JavaVirtualMachines/jdk-27.jdk</code>
JEP 512 kompakt kaynak dosyaları bayraksız çalışıyor	Doğrulandı, <code>java HelloWorld.java</code> çıktısı <code>Hello, World!</code> , çıkış 0
<code>gh</code> CLI yetkili	Evet, hesap <code>aliturgutbozkurt</code> , <code>repo</code> kapsamı mevcut

Modül 01'in giriş yolu varsayılmadı, gerçek araç zincirinde kanıtlandı.

Açık Sorularda Alınan Kararlar

Spesifikasyondaki 2 ve 3 numaralı açık sorular açıkça cevaplanmadı, dolayısıyla spesifikasyonun kendi önerileri geçerli:

- Test (Modül 21):** sıfır bağımlılık. `-ea` ile çıplak `assert` artı elle yazılmış bir mikro çatı, ve JUnit'in ne kattığını, gerçek projelerin onu neden kullandığını açıklayan bir bölüm. Bağımlılık yasağı korunuyor.
- Preview özellikler:** yalnızca Modül 22'de, JEP numarası ve preview turuyla açıkça etiketlenmiş, `--enable-preview --source 27` arkasında çalıştırılmış. Kaldırılmadılar, çünkü bir öğrenci kararlı ile preview arasındaki farkı bilmeli, ama karantinaya alındılar ki daha öncesindeki hiçbir şey onlara bağlı olmasın.

İkisinden biri değişmeli ise söyleyin.

Bileşenler

Bileşen	Sorumluluk	Bağımlılığı
scaffold	Depo iskeleti, doğrulama scripti, gitignore, README taslağı	—
template	Modül formatının referans uygulaması olarak Modül 01	scaffold
core-language	Modül 02–10, Java'ya yeni gelenin önce ihtiyaç duyduğu sözdizimi ve nesne modeli	template
type-system	Modül 11–12, jenerikler ve koleksiyonlar	core-language
functional	Modül 13–15, lambda'lar, stream'ler, Optional	type-system
platform	Modül 16–21, IO, standart kütüphane, eşzamanlılık, modüller, test	functional
release-literacy	Modül 22, 26 ve 27'nin ne değiştirdiği, JEP'ler, preview ile kararlı	platform
integration	Kök README içindekiler tablosu, tam depo doğrulama geçişi	hepsi

Kurulum sırası: scaffold → template → core-language → type-system → functional → platform → release-literacy → integration

Bu Sıra Neden

Sıralama teknik değil pedagojik. Şablon var olduktan sonra modül yazımı ilkesel olarak herhangi bir sırada olabilirdi, ama her modülün metni öğrencinin öncekileri okuduğunu varsayıyor. Sırayla yazmak çapraz göndermeleri dürist tutan şey; örneğin Modül 14'ün stream'lerde Modül 13'te tanıtilan fonksiyonel arayüzlere geri atıf yapması.

Not edilmeye değer iki istisna:

- scaffold önce ve eksiksiz gelmeli. Doğrulama scripti sonraki her örneğin çalıştığını kanıtlayan şey, dolayısıyla o var olmadan modül yazmak doğrulanmamış içerik biriktirmek demek.
- template yalnızca ilk modül değil, bir kapı. Modül 01 yedi bölümlük README yapısını, yorum stilini, ödev biçimini ve çözüm düzenini sabitliyor. Yanlış yapmak 21 modülü yeniden çalışmak demek, dolayısıyla Modül 02 başlamadan önce inceleniyor.

Riskler ve Önlemler

Risk	Etki	Önlem
Preview özellikler kesinleşmeden değişir	Modül 22 değişen bir şeyi öğretir	Modül 22'ye karantina, her preview açıkça etiketli, JEP numarası ve preview turu belirtilerek okuyucunun güncel durumu denetleyebilmesi sağlanır
Uydurulmuş sürüm iddiaları	Müfredat yanlışları kendinden emin biçimde öğretir	Spesifikasyon sınırı bunu yasaklıyor. Her JDK iddiası openjdk.org'a dayanıyor. Zaten uygulandı: JDK 27 JEP tablosu hafızadan değil sürüm sayfasından geldi
22 modül boyunca kapsam yorgunluğu	Proje yarım kalır	Issue başına bir modül, her biri bağımsız olarak teslim edilebilir ve doğrulanabilir. Doğrulanmış 11 modüllük yarım bir müfredat bile öğretir
Örnekler gelecekteki bir JDK'da sessizce bozulur	Çürüme	<code>verify-examples.sh</code> CI'da çalıştırılabilir ve her örneği ve çözümü kapsar
Modüller yazıldıktan sonra şablon değişimi	Pahalı yeniden çalışma	Modül 01, Modül 02 öncesinde açık bir inceleme kapısı
Öğrencinin JDK sürümü 27'den farklı	Örnekler kafa karıştırıcı biçimde başarısız olur	Kök README gerekli JDK'yı belirtiyor, doğrulama scripti <code>java -version</code> denetliyor ve uyumsuzlukta gürültülü biçimde başarısız oluyor

Paralel ve Ardışık

- **Ardışık:** scaffold → template → diğer her şey. Ayrıca içerik bağımlılığı nedeniyle Modül 13'ten önce 14, ve 08'den önce 09.
- **Paralleleştirilebilir:** `platform` içinde Modül 16, 17, 20 ve 21 birbirine içerik bağımlılığı taşımıyor, herhangi bir sırada ya da farklı kişilerce aynı anda yazılabilir.

Doğrulama Kontrol Noktaları

Sonrasında	Denetim
scaffold	<code>./scripts/verify-examples.sh</code> boş ağaçta 0 ile çıkıyor ve bilerek bozuk bir dosya verildiğinde gürültülü biçimde başarısız oluyor
template	Modül 01 doğrulamadan geçiyor ve README'si yedi bölümü de içeriyor
her modül	<code>./scripts/verify-examples.sh modules/<id></code> 0 ile çıkıyor, ödev metni mevcut, referans çözüm temiz çalışıyor
integration	Tam depo doğrulaması 0 ile çıkıyor, ve sekiz spesifikasyon başarı kriterinin hepsi tek tek işaretleniyor

Görev Listesi: Java 27 Öğrenim Müfredatı

Aşama: Specify → Plan → Tasks → Implement dizisinin 3. aşaması (Tasks) **Spesifikasyon:** [SPEC.md](#) ·

Plan: [plan.md](#)

Aşağıdaki her görev en fazla 5 dosyaya dokunuyor ve bağımsız olarak doğrulanabilir. Her görev bir GitHub issue'suna karşılık geliyor.

Tüm 24 görev tamamlandı ve issue'ları kapatıldı.

Çeviri notu. Görev metinleri İngilizce özgünlerinden bırakıldı. Dosya adlarını, modül kimliklerini, JEP numaralarını ve kabul kriterlerini isimlendiriyorlar; bunların İngilizce kaynakla birebir eşleşmesi gerekiyor. Çevrilmiş olanlar bölüm başlıkları ve alan etiketleri.

scaffold — iskelet

✓ T01 — Repository scaffolding and verification script · #1

- Kabul: `scripts/verify-examples.sh` walks `modules/**/examples/*.java` and `solutions/**/*.*.java`, runs each with JDK 27, honours the `// EXPECT: compile-error` and `// EXPECT: preview` headers, fails on any unexpected non-zero exit, and refuses to run if `java -version` is not 27. Root `README.md` stub states the JDK requirement and how to run an example. `.gitignore` excludes `*.class` and editor cruft.
- Doğrulama: `./scripts/verify-examples.sh` exits 0 on the empty tree; planting a deliberately broken `.java` file makes it exit non-zero with a readable message; removing it restores exit 0.
- Dosyalar: `scripts/verify-examples.sh`, `README.md`, `.gitignore`

template — format referansı

✓ T02 — Module 01 getting-started, the reference implementation · #2

- Kabul: Covers JDK 27 setup, the JVM/JRE/JDK distinction, compact source files (JEP 512, final in 25), `IO.println`, and what compiling actually does. README has all seven template sections. At least two runnable examples. Homework brief with acceptance criteria. Reference solution under `solutions/01-getting-started/`. This module's structure is the template every later module copies.
- Doğrulama: `./scripts/verify-examples.sh modules/01-getting-started` exits 0; solution runs clean; README section headings match the spec template exactly.
- Dosyalar: `modules/01-getting-started/README.md`, `modules/01-getting-started/examples/*.java`, `modules/01-getting-started/homework/README.md`, `solutions/01-getting-started/Solution.java`

- Kapi: T03 başlamadan önce formatın insan tarafından incelenmesi.

core-language — çekirdek dil

✓ T03 — Module 02 variables-and-types · #3

- Kabul: Primitives vs. references, `var` and where it helps or hurts, autoboxing traps including the Integer cache, String immutability, text blocks. Coming-from-another-language section contrasts Python/JS dynamic typing.
- Doğrulama: `./scripts/verify-examples.sh modules/02-variables-and-types` exits 0.
- Dosyalar: `modules/02-variables-and-types/**`, `solutions/02-variables-and-types/**`

✓ T04 — Module 03 control-flow · #4

- Kabul: Operators, `if / else`, switch expressions with arrow form and `yield`, all loop forms, enhanced `for`, labelled break. Contrasts switch expressions against the old fall-through statement form.
- Doğrulama: `./scripts/verify-examples.sh modules/03-control-flow` exits 0.
- Dosyalar: `modules/03-control-flow/**`, `solutions/03-control-flow/**`

✓ T05 — Module 04 methods · #5

- Kabul: Declaration, overloading and its resolution rules, varargs, static vs. instance, and an explicit treatment of Java being pass-by-value including for references, which is the single most misunderstood point for newcomers.
- Doğrulama: `./scripts/verify-examples.sh modules/04-methods` exits 0.
- Dosyalar: `modules/04-methods/**`, `solutions/04-methods/**`

✓ T06 — Module 05 classes-and-objects · #6

- Kabul: Classes, fields, constructors, `this`, encapsulation, static members, flexible constructor bodies (JEP 513, final in 25). Graduates the learner from compact source files to the classic `public class` plus `public static void main(String[] args)` and explains why every real codebase uses it.
- Doğrulama: `./scripts/verify-examples.sh modules/05-classes-and-objects` exits 0.
- Dosyalar: `modules/05-classes-and-objects/**`, `solutions/05-classes-and-objects/**`

✓ T07 — Module 06 inheritance-and-polymorphism · #7

- Kabul: `extends`, `super`, `@Override`, `abstract`, `final`, dynamic dispatch, and the `equals / hashCode / toString` contract with a worked example of getting it wrong.
- Doğrulama: `./scripts/verify-examples.sh modules/06-inheritance-and-polymorphism` exits 0.
- Dosyalar: `modules/06-inheritance-and-polymorphism/**`, `solutions/06-inheritance-and-polymorphism/**`

✓ T08 — Module 07 interfaces · #8

- Kabul: Interfaces vs. abstract classes, default/static/private methods, functional interfaces introduced as a concept ahead of Module 13, and why Java favours composition here.
- Doğrulama: `./scripts/verify-examples.sh modules/07-interfaces` exits 0.
- Dosyalar: `modules/07-interfaces/**`, `solutions/07-interfaces/**`

✓ T09 — Module 08 records-and-enums · #9

- Kabul: Records, compact constructors, what records generate for you, enums carrying behaviour and state, and a decision guide for class vs. record vs. enum.
- Doğrulama: `./scripts/verify-examples.sh modules/08-records-and-enums` exits 0.
- Dosyalar: `modules/08-records-and-enums/**`, `solutions/08-records-and-enums/**`

✓ T10 — Module 09 sealed-and-pattern-matching · #10

- Kabul: Sealed types, `instanceof` patterns, switch pattern matching, record patterns, and exhaustiveness checking. Depends on records from Module 08.
- Doğrulama: `./scripts/verify-examples.sh modules/09-sealed-and-pattern-matching` exits 0.
- Dosyalar: `modules/09-sealed-and-pattern-matching/**`, `solutions/09-sealed-and-pattern-matching/**`

✓ T11 — Module 10 exceptions · #11

- Kabul: Checked vs. unchecked and why Java is alone in this, try/catch/finally, try-with-resources, custom exceptions, and an explicit list of anti-patterns such as swallowing exceptions.
- Doğrulama: `./scripts/verify-examples.sh modules/10-exceptions` exits 0.
- Dosyalar: `modules/10-exceptions/**`, `solutions/10-exceptions/**`

type-system — tip sistemi

✓ T12 — Module 11 generics · #12

- Kabul: Generic classes and methods, bounded types, wildcards with PECS, type erasure and the concrete limitations it imposes.
- Doğrulama: `./scripts/verify-examples.sh modules/11-generics` exits 0.
- Dosyalar: `modules/11-generics/**`, `solutions/11-generics/**`

✓ T13 — Module 12 collections · #13

- Kabul: `List`, `Set`, `Map`, `Deque`, sequenced collections, choosing the right implementation, the equals/hashCode contract applied to hash-based collections, and immutable factory methods.
- Doğrulama: `./scripts/verify-examples.sh modules/12-collections` exits 0.

- Dosyalar: `modules/12-collections/**`, `solutions/12-collections/**`

functional — fonksiyonel

✓ T14 — Module 13 lambdas · #14

- Kabul: Lambda syntax, method references in all four forms, the standard functional interfaces, and variable capture including the effectively-final rule.
- Doğrulama: `./scripts/verify-examples.sh modules/13-lambdas` exits 0.
- Dosyalar: `modules/13-lambdas/**`, `solutions/13-lambdas/**`

✓ T15 — Module 14 streams · #15

- Kabul: Pipeline anatomy, intermediate vs. terminal operations, laziness, collectors, gatherers, and an honest treatment of when parallel streams help and when they hurt. Depends on Module 13.
- Doğrulama: `./scripts/verify-examples.sh modules/14-streams` exits 0.
- Dosyalar: `modules/14-streams/**`, `solutions/14-streams/**`

✓ T16 — Module 15 optional · #16

- Kabul: `Optional` as a return type, the idiomatic operations, and the anti-patterns that make it worse than plain null such as calling `get` unguarded or using it as a field.
- Doğrulama: `./scripts/verify-examples.sh modules/15-optional` exits 0.
- Dosyalar: `modules/15-optional/**`, `solutions/15-optional/**`

platform — platform

✓ T17 — Module 16 files-and-io · #17

- Kabul: `Path` and `Files`, reading and writing text, streaming large files, try-with-resources applied for real, and why the legacy `File` API is avoided.
- Doğrulama: `./scripts/verify-examples.sh modules/16-files-and-io` exits 0; examples clean up any files they create.
- Dosyalar: `modules/16-files-and-io/**`, `solutions/16-files-and-io/**`

✓ T18 — Module 17 standard-library-tour · #18

- Kabul: String methods and formatting, `java.time` in depth enough to avoid the classic date bugs, `Math`, random number generation, and comparators.
- Doğrulama: `./scripts/verify-examples.sh modules/17-standard-library-tour` exits 0; no example depends on the current wall-clock time for its assertions.
- Dosyalar: `modules/17-standard-library-tour/**`, `solutions/17-standard-library-tour/**`

✓ T19 — Module 18 concurrency-basics · #19

- Kabul: Threads, a demonstrable race condition, `synchronized`, atomics, `ExecutorService`, and the memory-visibility problem stated plainly.
- Doğrulama: `./scripts/verify-examples.sh modules/18-concurrency-basics` exits 0; examples are deterministic enough not to flake, or state their nondeterminism and still exit 0.
- Dosyalar: `modules/18-concurrency-basics/**`, `solutions/18-concurrency-basics/**`

✓ T20 — Module 19 virtual-threads · #20

- Kabul: Virtual threads (final in 21), why blocking became cheap again, scoped values (JEP 506, final in 25), pinning pitfalls, and a benchmark example contrasting platform and virtual threads.
- Doğrulama: `./scripts/verify-examples.sh modules/19-virtual-threads` exits 0.
- Dosyalar: `modules/19-virtual-threads/**`, `solutions/19-virtual-threads/**`

✓ T21 — Module 20 packages-and-modules · #21

- Kabul: Packages, classpath, the module system at a practical level, module import declarations (JEP 511, final in 25), plus `jar` and `javadoc` usage.
- Doğrulama: `./scripts/verify-examples.sh modules/20-packages-and-modules` exits 0; any multi-file example documents its exact compile command.
- Dosyalar: `modules/20-packages-and-modules/**`, `solutions/20-packages-and-modules/**`

✓ T22 — Module 21 testing-your-code · #22

- Kabul: `assert` with `-ea`, a hand-rolled micro test harness kept under 50 lines, what a real JUnit setup adds and why projects adopt it. Zero third-party dependencies, per the spec boundary.
- Doğrulama: `./scripts/verify-examples.sh modules/21-testing-your-code` exits 0; harness demonstrates both a passing and a failing assertion without failing the overall run.
- Dosyalar: `modules/21-testing-your-code/**`, `solutions/21-testing-your-code/**`

release-literacy — sürüm okuryazarlığı

✓ T23 — Module 22 whats-new · #23

- Kabul: What JDK 26 and 27 actually changed, sourced from `openjdk.org` rather than memory. LTS vs. non-LTS and why 25 matters more than 27 for production. How to read a JEP. Preview and incubator features explained, with structured concurrency (JEP 533, seventh preview) and primitive patterns (JEP 532, fifth preview) demonstrated behind `--enable-preview`, each labelled with its JEP number and preview round.
- Doğrulama: `./scripts/verify-examples.sh modules/22-whats-new` exits 0 including the preview-flagged examples; every version claim in the README carries a link to its `openjdk.org` source.
- Dosyalar: `modules/22-whats-new/**`, `solutions/22-whats-new/**`

integration — bütünleştirme

✅ T24 — Root README and final verification pass · #24

- Kabul: Root `README.md` links all 22 modules in order with a one-line description each, documents JDK 27 setup per platform, and explains how to use the solutions tree. All eight spec success criteria checked off explicitly.
- Doğrulama: `./scripts/verify-examples.sh` exits 0 across the whole repository; every module folder confirmed to contain README, examples, and homework; no non-English content found; no third-party dependency anywhere.
- Dosyalar: `README.md` , `SPEC.md` (success criteria checklist)